

國立臺灣大學電機資訊學院電子工程學研究所



碩士論文

Graduate Institute of Electronics Engineering
College of Electrical Engineering and Computer Science
National Taiwan University
Master's Thesis

三元權重變形器神經網路加速電路之設計與晶片實現
Ternary-Weight Transformer Acceleration Circuit Design and
Chip Implementation

陳法諭

Fa-Yu Chen

指導教授：闕志達 博士

Advisor: Tzi-Dar Chiueh, Ph.D.

中華民國 114 年 7 月

July, 2025

國立臺灣大學碩士學位論文

口試委員會審定書

MASTER'S THESIS ACCEPTANCE CERTIFICATE

NATIONAL TAIWAN UNIVERSITY

三元權重變形器神經網路加速電路之設計與晶片實現

Ternary-Weight Transformer Acceleration Circuit Design and Chip Implementation

本論文係陳法諭(R12943015)在國立臺灣大學電子工程學研究所完成之碩士學位論文，於民國 114 年 7 月 30 日承下列考試委員審查通過及口試及格，特此證明。

The undersigned, appointed by the Graduate Institute of Electronics Engineering on 30 July 2025, have examined a Master's Thesis entitled above presented by Fa-Yu Chen (R12943015) candidate and hereby certify that it is worthy of acceptance.

口試委員 Oral examination committee:

謝志遠

馬席彬

楊家驊

(指導教授 Advisor)

劉東溪

系(所、學位學程)主管 Director:

江介宏

致謝



在實驗室的兩年餘研究生涯，隨著碩士論文的完成也即將迎來結束，在不久的將來便會離開熟悉的校園環境，前往未知的就業場域展開新的探索。這兩年中，不僅在學術研究上努力精進自我、克服困難，也邂逅了許多各領域的貴人與高手，並獲得他們的幫助，才得以順利完成論文。

首先最發自內心感謝的便是指導教授，闕志達教授。教授在這兩年的研究進程中，不斷地給予我們新的刺激與建議，並無時無刻提醒著我們，要不斷思考現有方案哪裡有改進的空間，更重要的是，永遠不要放棄創新的可能性。最令我欽佩的，是在我的研究遭遇困難導致進度無法順利推進時，教授往往能一眼看出當下的問題本質，以畫龍點睛的建議讓我茅塞頓開，能夠用新的角度去看待這些問題，並找出原先沒有想到的解決方式。另外，老師也時常提醒我們對於學術研究應抱持嚴謹的態度與思維，對於自身的產出都應審慎看待，並為之負責。相信老師「嚴謹、嚴格」的研究態度，定會對未來我們在職場上看待自身事務的心境產生扎實的影響，讓我們更能駕馭即將到來的挑戰。也感謝教授在我們表達技巧的幫助，不論是平時周會的報告，或是如金砂獎或智慧創新大賞等競賽的提報，老師都給予了非常多回饋與指教，也讓我們順利在各競賽中獲得佳績。

此外，也要感謝闕志達教授實驗室的所有成員。感謝俊瑋學長、呈顯學長、嘉宏學長、嘉芯學姐、惟真學姐、國偉學長、丕全學長，在不管是研究相關的知識與經驗，抑或是對於未來人生規劃的建議，都給予我很大的幫助，讓這兩年的研究之旅更為順遂。也要感謝同屆的實驗室同學，駿驊、詠翔、帛恩、家任和慶玲，非常謝謝大家在不管是研究還是生活上的分享，在與大家討論的過

程中，往往能激盪出不同的創意與火花。祝諸位未來都能一帆風順，在各領域創造佳績。另外還要感謝實驗室的學弟們，庭毅、沛翰、奇泓、侑宸、睿謙、懷頡，謝謝大家對於實驗室事務的幫助，讓我可以專注地進行研究，也祝你們研究可以順順利利。



另外也特別感謝楊家驤教授實驗室的同學，由於在晶片下線流程中有許多需要實作經驗才能處理的問題，多虧有這幾位同學，才能順利將過程中的問題解決。非常感謝你們的協助，祝你們研究順利。

最後，最感謝的便是一路走來，陪伴我的家人們，感謝你們在我求學路上，鼓勵我追逐自己的興趣與理想，並支持我的決定，也謝謝你們在我遭遇挫折時，不斷關心我的情況，並給予我最大的支持，真的很謝謝你們。

謹以此篇論文獻給求學路上幫助我的家人、同學、朋友們，非常感謝諸位的協助，才能順利完成畢業研究，謝謝你們。祝你們一切順利、平安。

陳法諭 謹誌

中華民國 114 年 9 月 30 日

摘要



近年來，變形器神經網路(Transformer)由於其優異的表現，被廣泛應用於電腦視覺、自然語言處理、語音辨識等各式應用領域，並基於最初的 transformer 模型，針對各式任務的需求延伸出多種變體模型。儘管 transformer 在多個應用領域都有著超越傳統非 transformer 模型的表現，但高昂的硬體建置成本與模型推論時的能耗需求，都使得 transformer 系列模型在邊緣裝置與嵌入式系統等對於算力跟能耗有高度限制場域的部署成為一大挑戰。因此，在 transformer 模型表現日新月異的同時，許多研究也著眼於尋找在模型效能與硬體成本間的平衡點，其中，如何降低模型參數量與推論時的能耗，以便於邊緣運算裝置的部署，便是我們希望能解決的問題。

本文以維持 transformer 模型效能為首要目標，並以此為基礎追求大幅降低模型參數量、硬體成本、推論能耗的解決方案。針對 transformer 模型中的運算，提出了包括三元(Ternary)權重量化、非線性函數(包括 Softmax 與 GELU)近似、加法近似等技巧，並完成相對應硬體加速器晶片的設計與實現。並透過量化感知訓練的方式，確保在施加上述近似技巧後，模型仍盡可能保持原有效能。

首先是三元權重量化，我們將 transformer 模型的權重(Weight)量化至-1, 0 與 1，而輸入(Activation)則是採用 INT-4 的量化精度。這樣的精度設定相比二元(Binary)權重加上 INT-4 輸入的組合經實驗證明可以有效降低模型的效能損失，同時不會顯著增加硬體的面積與功耗。

在非線性函數近似的方面，我們針對 Softmax 函數提出了硬體友善的近似版本，Multimax，透過僅計算單一矩陣 row 中最大的 N 個數值並賦予相對應比

例，可以大幅降低面積與功耗。以 Multimax 計算電路本身而言，相比於 32 位元單精度浮點數的 Softmax 電路，面積與功耗分別可以降低 95.8%與 97.2%。本晶片可依照軟體模擬時的參數設定，支援 Multimax 的 N 為 1, 3 或 5。另一個非線性函數，GELU，我們則是提出以分段二項式進行近似的 QGELU，並將二項式中的參數設計為硬體僅需 Shift & Add 便可以完成的運算，讓整體成本進一步降低。以 QGELU 計算電路本身而言，相比於 32 位元單精度浮點數的 GELU 電路，面積與功耗分別可以降低 98.3%與 99.9%。

最後是加法電路的近似，本文提出多模態加法近似，可以支援精確運算、LSB 近似加法以及末兩位近似加法運算，可以在對於運算精確度較不敏感的 transformer 運算中採用較大程度的近似，以節省推論時的功耗，並在對於精確度較敏感的部分維持高精確度的加法運算以維持模型效能。近似加法器我們採用 OAI21 與 AAI21 的混和雙層近似加法器架構，相比普通加法器，面積與功耗分別可以下降 86.2%與 90.8%。若以單一一個 PE Cube 進行分析，開啟 LSB 近似加法與末兩位近似加法運算時分別可以節省 4.3%與 21.4%的功耗。

本研究所提出之加速晶片於 40 奈米製程與 200MHz 的工作頻率下，可以達到 32.3 TOPS/W 的能源效率與 1.50 TOPS/mm^2 的面積效率，超越其他研究所提出之神經網路加速晶片。而使用 ViT-Base 進行 inference 時，運算效能可達 1003.1 Token/J。

關鍵字：三元權重、Multimax、QGELU、高能效、加法近似電路

Abstract



In recent years, Transformer neural networks have achieved remarkable success in computer vision, natural language processing, and speech recognition, resulting in numerous task-specific variants. Despite their superior performance, Transformers face challenges of high hardware cost and energy consumption, hindering deployment in resource-constrained environments such as edge devices. As performance continues to improve, research increasingly focuses on balancing accuracy and efficiency. This work addresses the key challenge of reducing parameters and inference power consumption to enable deployment on edge platforms.

This work focuses on preserving Transformer performance while reducing parameters, hardware cost, and inference energy consumption. We introduce optimization techniques targeting key computations, including ternary weight quantization, nonlinear function approximations (Softmax, GELU), and approximate adders. A hardware accelerator chip is designed to support these methods, with quantization-aware training ensuring minimal accuracy loss.

We first apply ternary weight quantization, mapping weights to $[-1, 0, 1]$ with activations in 4-bit precision (INT4). Compared to binary weights with INT4 activations, this setting reduces performance degradation while incurring minimal hardware and power overhead.

For nonlinear function approximation, we propose Multimax, a hardware-friendly alternative to Softmax that selects the top-N values and assigns proportional weights. Compared to 32-bit Softmax, Multimax reduces area by 95.8% and power by 97.2%, with configurable N (1, 3, or 5).

Regarding the GELU activation function, we introduce QGELU, a segmented polynomial approximation designed for efficient hardware implementation. The polynomial coefficients are carefully chosen to allow computation using only shift-and-add operations, further reducing complexity. Compared to a standard 32-bit floating-point GELU circuit, QGELU achieves a 98.3% reduction in area and a 99.9% reduction in power consumption.

Lastly, we propose a multi-mode approximate adder supporting exact, LSB, and two-bit approximations to reduce inference energy. By applying lower-precision modes

to less accuracy-sensitive stages, the design balances efficiency and performance. Implemented with a two-layer OAI21–AAI21 hybrid structure, it achieves 86.2% area and 90.8% power savings over conventional adders. At the PE cube level, LSB and two-bit modes yield 4.3% and 21.4% power savings, respectively.

The accelerator chip proposed in this study, implemented using a 40nm process and operating at a frequency of 200 MHz, achieves an energy efficiency of 32.3 TOPS/W and an area efficiency of 1.50 TOPS/mm², outperforming other neural network accelerators reported in previous works. When performing inference with ViT-Base, the efficiency reaches 1003.1 token/J.

Keywords: Ternary Weights, Multimax, QGELU, Energy Efficiency, Approximate Adder Circuits

目次



致謝.....	i
摘要.....	iii
Abstract.....	v
目次.....	viii
圖次.....	xii
表次.....	xvi
1 第一章 緒論	1
1.1 研究背景.....	1
1.2 研究動機與目標.....	3
1.3 論文組織與貢獻.....	4
2 第二章 變形器神經網路(Transformer)介紹	6
2.1 多頭注意力(Multi-Head Attention, MHA).....	7
2.2 多層感知器(Multi-Layer Perceptron, MLP).....	11
2.3 層正則化(Layer Normalization)	14
2.4 第二章總結.....	16
3 第三章 三元權重與近似運算神經網路訓練與推論	18
3.1 訓練模型與資料集.....	18
3.2 影像辨識任務.....	18
3.2.1 ImageNet 資料集	18
3.2.2 Vision Transformer 簡介	19
3.2.2.1 ViT 與 DeiT	20
3.2.2.2 DaViT.....	21
3.2.2.3 Swin	23

3.3 文字生成任務.....	25
3.3.1 OLM & WikiText-2 資料集	26
3.3.2 GPT-2 簡介	27
3.4 語音轉文字任務.....	27
3.4.1 LibriSpeech 資料集	28
3.4.2 Whisper 簡介	28
3.5 三元權重神經網路簡介與實驗結果.....	29
3.5.1 神經網路量化.....	29
3.5.1.1 三元權重量化.....	33
3.5.1.2 輸入量化.....	34
3.5.2 三元權重神經網路訓練.....	36
3.6 複雜函數近似.....	38
3.6.1 Multimax.....	39
3.6.1.1 Softmax 硬體實作技巧比較	39
3.6.1.2 構想與實現.....	40
3.6.2 QGELU.....	42
3.6.2.1 GELU 硬體實作技巧比較	43
3.6.2.2 構想與實現.....	43
3.7 加法近似運算.....	46
3.7.1 加法近似技巧文獻回顧.....	47
3.7.2 OAI21+AAI21 雙層近似加法器	50
3.7.3 近似加法硬體實作結果分析.....	53
3.8 模型訓練結果彙整.....	57
3.8.1 影像辨識任務訓練結果.....	57
3.8.2 文字生成任務訓練結果.....	59
3.8.3 語音轉文字任務訓練結果.....	60

3.8.4 稀疏性分析.....	60
3.8.5 加法近似訓練結果.....	60
3.8.6 模型輸出結果比較.....	64
3.9 第三章總結.....	65
4 第四章 晶片架構設計	67
4.1 晶片架構總覽.....	67
4.1.1 運算電路概覽.....	67
4.1.2 記憶體配置.....	68
4.2 PE(Processing Element) Cube	68
4.2.1 PE Unit.....	69
4.2.2 PE Cube	71
4.3 PPU(Post-Processing Unit).....	74
4.3.1 架構介紹.....	74
4.3.2 Accumulator.....	75
4.3.3 Requantizer	77
4.3.4 QGELU	78
4.3.5 Pre-Sort	80
4.4 Multimax Calculation Unit	81
4.5 Self-Test Circuit.....	84
4.6 晶片架構.....	85
4.7 晶片計算流程.....	86
4.8 晶片支援運算總結.....	88
4.9 第四章總結.....	90
5 第五章 晶片實現	93
5.1 晶片設計流程.....	93
5.2 晶片驗證流程.....	94

5.3 晶片參數.....	95
5.3.1 晶片布局.....	95
5.3.2 晶片數據分析.....	97
5.3.3 晶片比較表.....	104
5.4 效能與功耗分析.....	108
5.5 晶片量測.....	111
5.6 第五章總結.....	114
6 第六章 研究結語與展望	116
參考文獻.....	119

圖次



圖 2.1 Transformer 神經網路架構[1]	7
圖 2.2 h=4 Multi-Head Attention 運算流程	8
圖 2.3 Single-Head Attention 運算流程	9
圖 2.4 MLP 運算流程	12
圖 2.5 Activation Function 非線性區間比較	14
圖 2.6 Transformer block 核心架構圖	17
圖 3.1 ImageNet 資料集[8]	19
圖 3.2 ViT 模型架構圖	20
圖 3.3 DaViT 中 Dual Attention 示意圖[11]	22
圖 3.4 Dual Attention Block 架構圖[11]	22
圖 3.5 Channel Group Self-Attention 結果範例[11]	23
圖 3.6 ViT 系列模型之 Feature Map 比較[11]	23
圖 3.7 Swin 與 ViT 之 MHA 計算範圍差異[12]	24
圖 3.8 Swin 模型採用之 Shifted Window 作法[12]	24
圖 3.9 Cycle-shift 與 masked-MSA[12]	25
圖 3.10 Swin 模型架構[12]	25
圖 3.11 Whisper 核心架構與訓練流程	29
圖 3.12 BitNet 架構[20]	30
圖 3.13 BitNet a4.8[23]	31
圖 3.14 Progressive Quantization	37
圖 3.15 Progressive Mixed Precision	39
圖 3.16 Softmax 輸出範例	40
圖 3.17 Softmax 與 Multimax N=1, 3, 5 結果比較	42

圖 3.18 GELU vs. FGELU vs. QGELU 繪圖	44
圖 3.19 QGELU 局部曲線放大圖	45
圖 3.20 ViT 模型 GELU 輸入分布	46
圖 3.21 NAND gate + NOR gate 單層近似加法器與雙層近似加法器架構[38]	48
圖 3.22 OR gate 近似加法器架構[39]	48
圖 3.23 OR gate + Input Swapping 近似加法器架構[40]	49
圖 3.24 OAI21 Dual-Mode Adder 近似加法器架構[35]	49
圖 3.25 OAI21 邏輯閘架構	50
圖 3.26 AAI21 邏輯閘架構	51
圖 3.27 OAI21 + AAI21 雙層近似加法器架構	53
圖 3.28 TW4A Swin-tiny Linear Layer 之權重與輸入分布	54
圖 3.29 近似加法測試資料實際加總結果分布	54
圖 3.30 近似加法器面積與功耗比較	55
圖 3.31 近似加法器誤差分布圖	56
圖 3.32 近似加法器 MSE、Area、Power 比較繪圖	57
圖 3.33 ViT-Base 模型輸出比較	64
圖 4.1 PE Unit 電路架構	70
圖 4.2 PE Unit 資料流	71
圖 4.3 PE Cube 架構	71
圖 4.4 PE Plane 運算流程	72
圖 4.5 PE Cube 運算流程	73
圖 4.6 矩陣乘法 block 讀取示意圖	73
圖 4.7 PE Cube 與 PPU 連接關係	75
圖 4.8 PPU Unit 電路架構	75
圖 4.9 Accumulator 電路架構	76
圖 4.10 MatMul 運算流程	76

圖 4.11 Requantizer 電路架構.....	77
圖 4.12 Linear Layer 運算流程	78
圖 4.13 QGELU 電路架構	79
圖 4.14 QGELU 運算流程	79
圖 4.15 GELU vs. QGELU 硬體面積功耗比較	80
圖 4.16 Pre-Sort 電路架構	81
圖 4.17 Multimax Calculation Unit 電路架構	82
圖 4.18 Multimax 運算流程	83
圖 4.19 Softmax vs. Multimax 硬體面積功耗比較	84
圖 4.20 Self-Test Circuit 輸出訊號波形圖	84
圖 4.21 Self-Test Circuit 硬體架構	85
圖 4.22 晶片整體架構圖	86
圖 4.23 單一 PE Cube 運算過程	87
圖 4.24 四組 PE Cube 運算過程	87
圖 4.25 加速晶片運算支援範圍	89
圖 4.26 各模式資料傳遞過程與啟動模組示意圖	90
圖 5.1 晶片設計流程	94
圖 5.2 晶片驗證流程	95
圖 5.3 晶片布局圖	96
圖 5.4 晶片電壓衰退分布圖與晶片 layout	97
圖 5.5 面積佔比圖	102
圖 5.6 矩陣運算模式功耗佔比圖與模組啟動示意圖	103
圖 5.7 量化模式功耗佔比圖與模組啟動示意圖	103
圖 5.8 QGELU 模式功耗佔比圖與模組啟動示意圖	104
圖 5.9 Multimax 模式功耗佔比圖與模組啟動示意圖	104
圖 5.10 晶片量測流程圖	112

圖 5.11 範例量測結果 shmoo plot.....	113
-------------------------------	-----



表次



表 3.1 BitNet b1.58 於 LLaMA LLM 訓練結果[21]	30
表 3.2 BitNet 系列模型訓練結果[24]	32
表 3.3 ViT-b1.58 訓練結果[25]	32
表 3.4 QGELU 區間分割與二次函數係數	44
表 3.5 GELU vs. QGELU vs. FGELU 誤差分析	46
表 3.6 OAI21 真值表與近似加法結果比較	51
表 3.7 AAI21 真值表與近似加法結果比較	52
表 3.8 近似加法器面積與功耗比較	55
表 3.9 近似加法器 MSE 比較	56
表 3.10 影像辨識任務訓練結果	58
表 3.11 TW4A vs. 1W4A 影像辨識任務訓練結果比較	59
表 3.12 文字生成任務訓練結果	59
表 3.13 語音轉文字任務訓練結果	60
表 3.14 各模型稀疏性分析	60
表 3.15 Swin-tiny 加法近似訓練結果(baseline=TW4A + Multimax + QGELU)	63
表 3.16 Swin-tiny 加法近似訓練設定(baseline=TW4A + Multimax + QGELU)	63
表 3.17 Whisper 加法近似訓練結果(baseline=TW4A + Multimax + QGELU)	64
表 3.18 GPT-2 模型輸出比較	65
表 3.19 Whisper 模型輸出比較	65
表 4.1 On-chip memory 配置	68
表 4.2 PE Cube 架構近似加法架構比較表	74
表 4.3 GELU vs. QGELU 硬體比較表	80
表 4.4 Softmax vs. Multimax 硬體比較表	83

表 5.1 ViT 架構各加速運算模式佔比.....	99
表 5.2 晶片(Die)參數表	100
表 5.3 核心運算電路(Core)參數表	101
表 5.4 晶片比較表.....	107
表 5.5 運算平台設置細節.....	108
表 5.6 各運算平台平均功耗.....	109
表 5.7 各運算平台能量效率計算.....	109
表 5.8 ASIC 與 LP CPU inference 數據比較.....	110
表 5.9 ViT 系列模型參數細節.....	110
表 5.10 量化近似 ViT 模型與同系列較小模型 accuracy 比較.....	111
表 5.11 各運算平台效率計算結果.....	111
表 5.12 PVT corner 設定與晶片模擬結果	114

第一章 緒論



1.1 研究背景

隨著資料量的急遽增加與運算能力的提升，機器學習(Machine Learning)已逐步成為各種智慧應用的核心技術。早期的機器學習演算法多仰賴特徵工程與領域知識來建構模型，雖能達成特定任務，卻難以處理複雜且高維度的非結構化資料。隨著深度學習(Deep Learning)技術的興起，特別是以卷積神經網路(CNN)與遞迴神經網路(RNN)為代表的模型架構，人工智慧開始在圖像識別、語音辨識、自然語言處理等領域取得突破性的成果。

在此基礎上，[1]提出的 Transformer 模型，徹底改變了深度學習的發展方向。該架構摒棄了傳統 RNN 的序列性限制，透過自注意力機制(Self-Attention Mechanism)實現了並行計算與全局關聯建模的能力，不僅顯著提升了模型性能，也大幅加速了訓練過程。Transformer 及其後續變體(如 BERT、GPT 系列、T5、ViT 等)迅速在語言、視覺、跨模態等多個領域取得領先地位，成為當代深度學習研究與應用的主流架構。然而，Transformer 模型強大的表現能力也伴隨著巨大的參數規模與計算複雜度。推論過程中所需的運算量與記憶體頻寬已遠遠超出傳統處理器的承載範圍。這對運行效率與能耗造成了極大挑戰，特別是在資源有限的邊緣裝置或即時應用場景中，顯得更加明顯。

為了應對深度學習模型日益成長的運算需求，運算硬體架構亦同步發展。早期的機器學習任務多仰賴中央處理器(Central Processing Unit, CPU)執行，雖具通用性，但在面對大規模矩陣運算時效率有限。隨後圖形處理器(Graphics Processing Unit, GPU)因具備高度平行化的運算處理能力而被廣泛應用於深度學

習領域，成為主流訓練與推論平台。然而，GPU 本質上仍為通用型架構，其能耗與成本在特定應用情境中可能不具優勢。



為進一步追求高效能與低功耗的運算平台，專用積體電路(Application-Specific Integrated Circuit, ASIC)以及可程式化邏輯裝置(Field-Programmable Gate Array, FPGA)逐漸成為新一代人工智慧硬體設計的重點方向。與通用架構相比，ASIC 能針對特定神經網路運算特性進行最佳化設計，大幅提升硬體利用率與能源效率。例如，Google 推出的 Tensor Processing Unit(TPU)[2]即是一個針對深度神經網路優化的 ASIC 設計，已廣泛應用於其雲端 AI 服務中。儘管如此，Transformer 架構本身的高動態性與複雜資料存取模式，仍對硬體設計造成諸多挑戰。

在眾多提升硬體效率的技術中，模型量化(Model Quantization)被認為是最具潛力的解決方案之一。量化技術透過將模型中使用的浮點數據轉換為較低位元的整數表示(如 INT8、INT4 甚至 Binary)，可有效降低儲存空間與記憶體存取頻寬，並簡化運算單元的實作。結合適當的量化策略與硬體架構調整，能在僅微幅影響模型準確度的情況下，顯著提升整體運算效率與能源效益。對於需部署於資源受限或需高即時性的系統而言，量化技術與專用硬體的結合，具備極高的實用性與產業價值。

除了模型量化之外，函數近似(Approximation)也是現代 AI 加速器設計中廣泛採用的重要技術之一。許多深度學習模型中涉及大量非線性激活函數與數學運算，如 Softmax、GELU(Gaussian Error Linear Unit)、Sigmoid、Tanh 等，這些函數在數學上通常包含指數、平方根、誤差函數等複雜運算，若直接以浮點邏輯實現，將導致硬體面積龐大、計算延遲增加，並大幅提升功耗，嚴重限制模

型於邊緣端或嵌入式平台的應用效能。為了降低這類非線性函數的硬體實作複雜度，常見的作法包括使用數學近似(如泰勒展開、雙曲線近似)、LUT、區段線性近似(Piecewise Linear Approximation)或混合策略等方式，將原始函數轉換為硬體可高效運行的近似表示，大幅簡化乘除法與冗長的指數運算邏輯。

1.2 研究動機與目標

隨著人工智慧技術的蓬勃發展，Transformer 模型憑藉其在多項任務中展現出的卓越效能，已成為現今深度學習領域的主流架構，特別是在生成式 AI 應用中表現尤為突出。然而，在 Transformer 模型優越效能的背後伴隨著極高的運算成本與記憶體需求，進而衍生出多項挑戰。

首先，Transformer 模型在訓練與推論階段皆需仰賴大量矩陣運算與非線性變換，對硬體資源需求極為龐大，導致整體功耗居高不下。其次，隨著模型規模不斷擴張，參數量與運算過程中的特徵維度也隨之倍增，使得在記憶體資源有限的邊緣裝置或嵌入式平台上部署此類模型變得日益困難。此外，Transformer 架構中所廣泛使用的非線性函數，例如 GELU 激活函數與 Softmax 注意力權重函數，其計算涉及複雜數學運算，進一步增加實作複雜度與硬體負擔。

因此，本研究旨在從軟硬體協同設計的角度，針對 Transformer 模型提出整體性加速解決方案，並聚焦於以下兩個研究目標：

1. 模型層面的效能優化：

模型模擬部分，我們希望以模型效能為首要目標，提出合適的數值量化精度與硬體運算複雜的函數之近似技巧，並可以在經過重新訓練後盡可能維持原

有模型效能，同時能夠大幅降低模型推論所需的硬體成本與記憶體需求。並在各項 transformer 模型常見應用領域(如影像辨識、文字生成、語音辨識等)進行驗證，確保我們提出的技巧在不同種類與架構的 transformer 模型中接能夠維持良好的推論效果。

2. 硬體層面的加速器晶片設計：

加速晶片設計部分，希望能夠支援 transformer 模型中主要運算的加速，並針對非線性函數採用硬體友善的方式進行實作。此外，本設計將採用 40 奈米製程進行前後端晶片設計與下線，並針對晶片面積、功耗效率與運算量等關鍵指標進行分析與評估，以驗證本研究在硬體層面之實際效益。所設計之硬體加速器希望得以超越現有加速器之面積與功耗效率。

透過上述軟硬整合之設計策略，本研究期望能為 Transformer 模型於低功耗、高效能應用場景中的落地部署提供具體可行的硬體支援，進而促進 AI 技術於邊緣運算、生醫感測、智慧裝置等領域之廣泛應用。

1.3 論文組織與貢獻

本文共分為六個章節。第一章說明本研究之背景脈絡、研究動機與主要研究目標。第二章針對 Transformer 模型架構進行詳細說明，涵蓋其關鍵模組與運作原理，以奠定後續方法與設計討論的基礎。第三章介紹本研究提出之模型優化方式，包括量化演算法、函數近似方式，並詳述所使用之模型架構與資料集，搭配訓練結果分析。第四章則聚焦於加速晶片的設計，說明整體系統架構、各模組的硬體實作方式與資料運算流程。第五章進一步探討晶片實現之相關細節，包括設計參數、驗證流程以及與既有設計之效能比較分析，並聚焦晶片的效能

與功耗分析。第六章則對本研究成果進行總結，並提出未來可延伸之研究方向與應用展望。



本篇論文的三個主要貢獻可歸納如下：

1. 本研究將 Transformer 模型中的權重與輸入分別量化為 Ternary 及 INT-4 格式，成功降低了乘法與加法電路的複雜度，進而有效減少整體運算電路的硬體成本與記憶體資源使用。此一設計大幅縮減了加速晶片所需之邏輯面積與記憶體空間，提升了部署於資源受限裝置時的可行性。
2. 針對 Transformer 架構中計算量較高之非線性函數，Softmax 與 GELU，本文分別提出了硬體友善的近似運算方法：Multimax 與 QGELU。相較於傳統使用 FP32 精度的原始函數，Multimax 可使面積與功耗分別下降 95.8% 與 97.2%，而 QGELU 則分別達到 98.3%與 99.9%的顯著下降。此外，透過模型層級的軟體模擬驗證，證實上述近似函數可於維持模型準確度的同時，有效取代原有高複雜度運算。
3. 本研究提出混合式雙層近似加法器架構，結合 OAI21 與 AAI21 基本邏輯閘單元，並於運算電路中設計三種近似程度加法運算模式，分別是精確模式、LSB 近似與末兩位近似，使加法運算可根據不同運算階段對精度的需求進行調節，以節省模型推論時的功耗。實驗結果顯示，針對單一 PE Cube，採用 LSB 近似與末兩位近似模式相較於精確模式，可分別降低功耗達 4.3%與 21.4%，有效提升模型推論階段的能效表現。

第二章 變形器神經網路 (Transformer)介紹



Transformer 神經網路由 Google Brain 團隊於 2017 年提出 [1]。與傳統神經網路主要依賴遞迴結構(Recurrent Neural Networks, RNNs)或捲積結構(Convolutional Neural Networks, CNNs)作為基礎不同，Transformer 則以注意力機制(Attention Mechanism)作為運算核心，能夠有效捕捉序列中不同位置間的全域依賴關係。此外，Transformer 架構中引入了多頭注意力(Multi-Head Attention, MHA)以平行化計算多個注意力分布，並搭配位置編碼(Positional Encoding)以保留序列順序資訊。由於其高度並行化與優異的特徵擷取能力，Transformer 在各項應用領域，包括自然語言處理中的文字生成與翻譯、電腦視覺中的影像辨識，以及語音處理等任務上，均展現出遠超過傳統神經網路架構的表現。

Transformer 架構主要由編碼器(Encoder)與解碼器(Decoder)兩部分所組成，其中每一層皆包含三個主要基礎模塊：多頭注意力、前饋神經網路(Feed Forward Network, FFN)、以及 Add & Norm 結構(包含殘差連接 Residual Connection 與層正規化 Layer Normalization)。其中，多頭注意力模塊負責捕捉序列中不同位置之間的關聯性，前饋神經網路則進行非線性變換以增強特徵表現能力，而 Add & Norm 結構則有助於穩定深層網路的梯度傳遞。本篇論文所提出之量化演算法訓練以及後續之加速電路設計，主要聚焦於多頭注意力與前饋神經網路這兩項核心運算模塊的最佳化與實作。

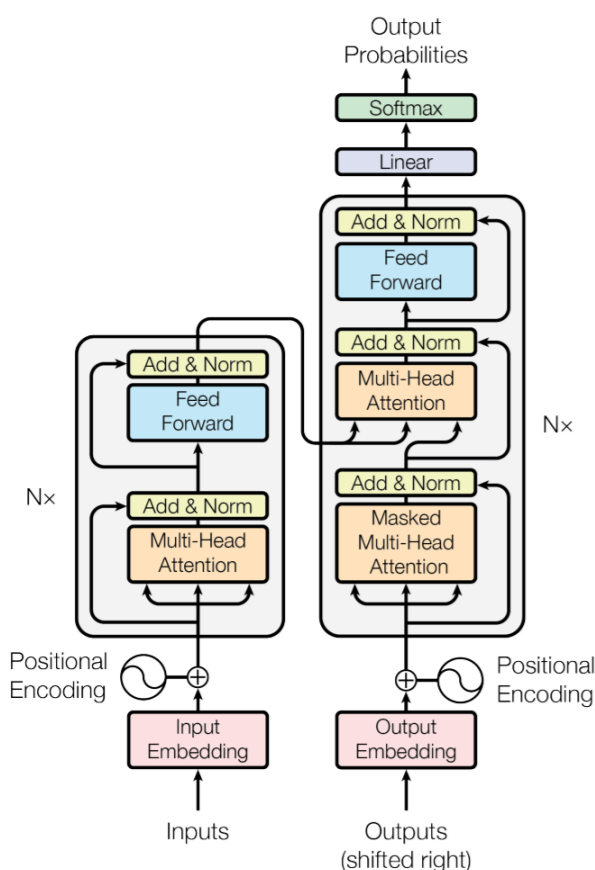


圖 2.1 Transformer 神經網路架構[1]

2.1 多頭注意力(Multi-Head Attention, MHA)

多頭注意力是 Transformer 架構中的核心機制，其設計理念為：將輸入的表示向量(input embeddings)投影至多個子空間，並在每個子空間中分別進行自注意力運算。每一個注意力頭(Head)都能從不同的角度捕捉輸入之間的關聯性，進而提取多樣化的語義資訊。最終，來自各頭的注意力輸出會被串接起來，並經由一個線性投影層，整合成最終的輸出表示。

使用多個注意力頭的主要優勢在於，每個頭可以專注於輸入序列中的特定關係模式。例如，在自然語言處理中的文字生成任務中，各個頭可能分別學習到不同位置間 token 的語法結構、語意依賴或上下文邏輯等互動關係。透過並行

的多頭計算，模型能更全面地理解輸入序列中潛藏的語義結構，進一步提升表示能力與預測準確性。儘管多頭注意力能有效擴充模型的表示能力，其代價為計算與記憶體資源的顯著增加，特別是在注意力頭數增加的情況下。因此，實際應用中需在效能與資源成本間取得平衡。

圖 2.2 展示了當注意力頭數 $h=4$ 時，Multi-Head Attention 的整體運算流程。

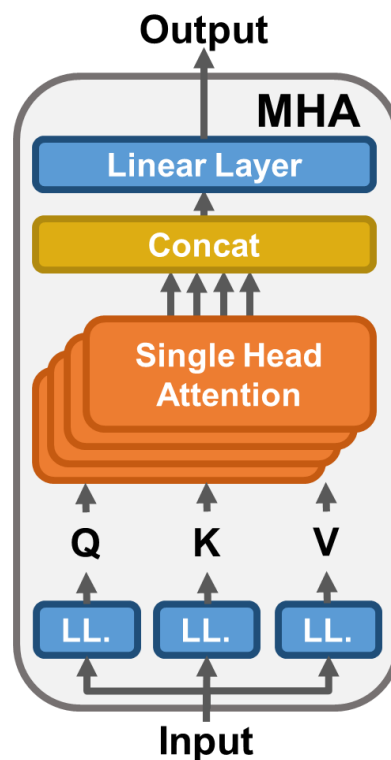


圖 2.2 $h=4$ Multi-Head Attention 運算流程

此外，圖 2.3 亦放大呈現了單一注意力頭的運算過程，便於詳細觀察其內部機制。

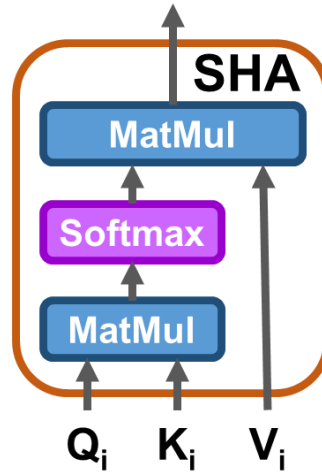


圖 2.3 Single-Head Attention 運算流程

以第 i 個 Head 為例，輸入向量首先會分別通過三個線性投影層，產生三組表示矩陣： Q_i (Query)矩陣、 K_i (Key)矩陣、與 V_i (Value)矩陣。這三個矩陣分別是由模型學習得來的權重參數 W_i^Q 、 W_i^K 、 W_i^V 所生成。接下來，計算注意力權重時， Q_i 與 K_i^T 矩陣相乘後，會除已一個縮放參數 $\sqrt{d_k}$ 。此操作可穩定梯度，防止隨維度擴增而造成過大的內積值，進而影響 Softmax 的學習效果。這部分的結果會通過 Softmax，Softmax 函數會對每一行的得分向量進行正規化，使所有注意力權重落在 0~1 區間內，且總和為 1。這些權重表示每個輸入位置對當前輸出位置的重要程度。Softmax 的數學函數如(式 2.1)所示。最後會乘上 V_i 矩陣，即可得到單一頭的注意力輸出。

$$\text{Softmax}(X)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

(式 2.1)

完成各個 Head 的運算後，各組結果會被串接後乘上權重矩陣 W^O ，至此完成 Multi-Head Attention 的完整運算。計算流程可以參考(式 2.2)的數學公式。



$$\begin{aligned}
 Q_i &= I \times W_i^Q \\
 K_i &= I \times W_i^K \\
 V_i &= I \times W_i^V \\
 head_i &= Attention(Q_i, K_i, V_i) = Softmax\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) \times V_i \\
 Output &= Concat(head_1, head_2, \dots, head_h) \times W^O
 \end{aligned}$$

(式 2.2)

由上述介紹可觀察到，各個注意力頭之間的運算過程彼此獨立，因此在硬體資源允許的情況下，Multi-Head Attention 機制能充分發揮平行計算的優勢。特別是在以 GPU 進行模型訓練或推論的應用場景中，這種並行特性可有效提升運算效率與加速性能，進而縮短訓練時間並改善模型部署效能。

然而需注意的是，Attention 機制本身對於序列資料中的相對順序並不具備敏感性。也就是說，它在運算過程中僅考慮向量間的相似度，卻未直接納入輸入元素在序列中出現的位置信息。這對於某些對輸入排列順序高度敏感的任務而言(如語言翻譯、語言建模與文字生成等)，可能導致模型無法正確理解語義。例如，若同一個名詞出現在句首與句末，其語意可能截然不同。若模型無法辨識這種位置差異，則預測結果將受到影響。

為解決上述問題，Transformer 架構中引入了位置編碼(Positional Encoding)技術，旨在將詞語或 token 的位置信息嵌入原始輸入表示中，使模型能夠學習位置與語義間的關聯。透過位置編碼，Attention 機制得以在保持並行計算特性的同時，捕捉序列中各元素的相對與絕對位置資訊。

實作位置編碼的方法主要可分為兩大類型：



1. 給定數值型(Fixed Positional Encoding)：

此方法使用預定義的數學公式為每個位置分配固定的向量，例如原始 Transformer 論文中[1]所提出的三角函數編碼方式(式 2.3)，即透過正弦與餘弦函數於不同頻率下生成位置表示，提供模型一種能推論相對距離的連續表示方式。

2. 自主學習型(Learnable Positional Encoding)：

此類方法則將位置編碼向量設為可訓練參數[3]，與詞向量一同更新，讓模型根據下游任務需求，自行學習適合的位置資訊表示方式，提升靈活性與表現潛力。

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

(式 2.3)[1]

2.2 多層感知器(Multi-Layer Perceptron, MLP)

多層感知器，亦稱為前饋神經網路(Feed-Forward Neural Network, FNN)，是 Transformer 架構中關鍵的組成元件之一，主要負責對多頭注意力(Multi-Head Attention)機制的輸出進行非線性轉換，進一步強化模型的表示能力與擬合複雜模式的能力。

在標準 Transformer 架構中，MLP 模組通常由兩層 Linear Layers 組成，中間穿插一層非線性 Activation Function，如圖 2.4 所示。



Multilayer Perceptron (MLP)

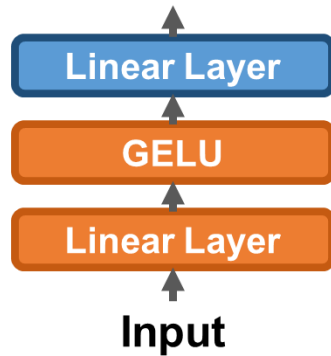


圖 2.4 MLP 運算流程

Linear Layer 主要負責線性的變換，數學公式如(式 2.4)所示。其中 W 為權重(Weight)矩陣、 B 為偏置(Bias)項矩陣，二者皆為模型訓練時學習而來。

$$\text{Linear Layer}(X) = X \times W + B$$

(式 2.4)

Activation Function 則負責非線性轉換的運算，使模型得以學習更為複雜的映射關係，在 Transformer 模型中一般使用 GELU 函數作為 Activation Function。GELU 於[4]中被提出，希望能提出相較於 ReLU 函數更平滑、連續且更具表現力的非線性轉換，數學公式如(式 2.5)所示，(式 2.5)所展現的形式為最初提出的版本。

$$GELU(x) = x \times \Phi(x) = \frac{1}{2} \left[1 + \operatorname{erf} \left(\frac{x}{\sqrt{2}} \right) \right]$$



(式 2.5)

其中 $\Phi(x)$ 為標準高斯分佈的累積分佈函數，此形式在理論上具有良好的特性，但在實務應用上，其計算相對複雜。因此，在實作層面上，GELU 通常會採用一種基於 Tanh 的近似公式(式 2.6)來替代原始表達式，以平衡運算效率與函數平滑性。

$$GELU(x) \approx \frac{x}{2} \left[1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right]$$

(式 2.6)

圖 2.5 繪製了三種常見啟動函數：GELU、ReLU 以及 Sigmoid 的數學曲線作為參考。從圖中可以觀察到，ReLU 在輸入大於零時對應斜率為 1 的線性增長，而在輸入小於零時則直接輸出 0，呈現出明顯的線段性質。雖然 ReLU 具備計算效率高與抑制梯度消失等優點，但其在負半軸的梯度為零，導致某些神經元在訓練過程中可能出現 dead neurons 的問題，進而限制模型的表現能力。

相較之下，GELU 在原點附近展現出更細緻的非線性曲線形態，輸出值隨輸入平滑變化。這種非線性特性可使模型學習更加複雜的資訊關係與非線性模式，有助於提升語言建模等高階任務的效果。尤其值得注意的是，GELU 函數在整個定義域上均為連續且可微分，與 ReLU 在原點不可微的特性不同，這有助於在反向傳播時維持梯度的穩定性，進而提升模型訓練的收斂效率與穩定性。綜合而言，GELU 函數結合了 ReLU 的部分線性優勢與 Sigmoid 的平滑特性，在現代 Transformer 架構中成為常用的激活函數選擇。

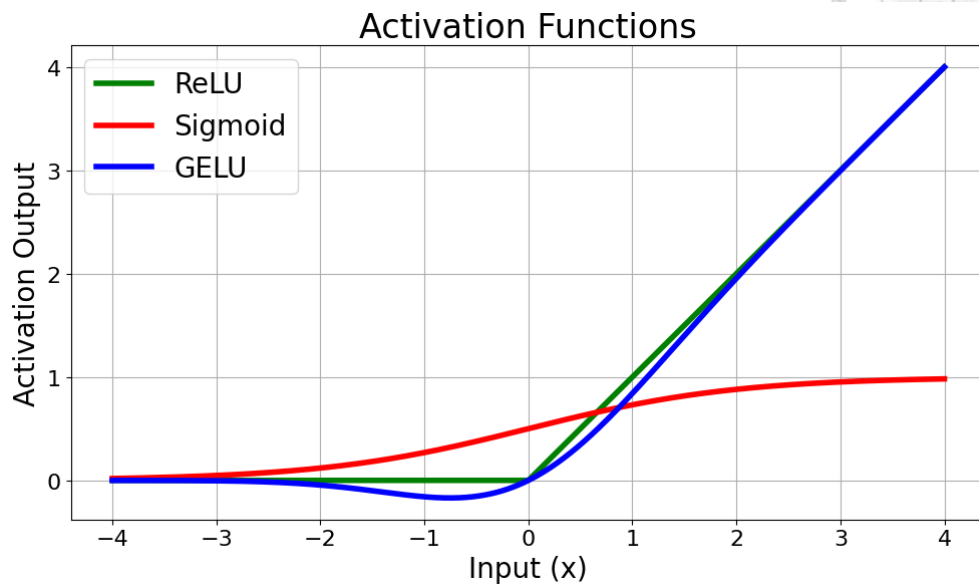


圖 2.5 Activation Function 非線性區間比較

2.3 層正則化(Layer Normalization)

隨著深度神經網路的層數增加，在模型訓練時容易因運算數值不穩定導致梯度消失(Gradient Vanishing)或梯度爆炸(Gradient Explosion)的情況，使訓練難以收斂或結果不理想。為了緩解上述問題，研究者在模型中引入各種正則化技術，透過將階段性運算結果的數值控制在特定數值範圍內，可以穩定模型內部數值的分布，加速模型訓練收斂。

在傳統神經網路中，常使用批標準化(Batch Normalization)作為正則化的技巧，Batch Normalization 是以 mini-batch 為單位，針對每個特徵維度計算均值與變異數，並進行標準化。不過由於 Batch Normalization 的運算需要橫跨 mini-batch 的所有維度，對輸入序列的長度與 batch size 有一定依賴，但在 Transformer 模型中序列的長度可能不一致，這導致模型在推論時的數值分布可能與訓練時的有所差異，導致模型表現不穩定。



因此，[5]中在 Transformer 中使用 Layer Normalization 來實作，也就是架構中的 Add & Norm 部分，相比於 Batch Normalization，Layer Normalization 是針對每個樣本輸入進行獨立的正則化，不會受 batch size 變動的影響。經訓練實驗也成功證明，Layer Normalization 於 Transformer 模型中能夠更好的協助訓練過程的穩定，獲得更理想的訓練結果。而 Add & Norm 中的 Add 則是殘差連接運算(式 2.7)，目的是透過讓輸入繞過核心運算部分，直接連接到輸出，以緩解梯度消失問題。

$$Output = LayerNorm(x + Sublayer(x)) \quad (式 2.7)$$

Layer Normalization 的數學運算如(式 2.8)所示，首先計算輸入的平均值(Mean)，接著計算變異數(Variance)，最後進行正則化運算，在最終計算正則化時會於分母加上一個小常數 ϵ ，以避免變異數過小導致數值不穩定。

$$\begin{aligned} x &= [x_1, x_2, \dots, x_H] \\ \mu &= \frac{1}{H} \sum_{i=1}^H x_i \\ \sigma^2 &= \frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2 \\ \hat{x}_i &= \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \end{aligned} \quad (式 2.8)$$

Layer Normalization 作為一種有效的正則化技術，特別適用於以 Transformer 為代表的序列模型。其透過對神經網路中各層輸出進行標準化處理，穩定了中間層的數值分布，有助於提升模型的訓練效率與最終性能表現。



2.4 第二章總結

本章節介紹了 Transformer 模型的核心架構，並對其關鍵模組進行了詳細說明與分析。當中最具創新性的設計為 Attention 機制，其透過動態計算輸入序列中各位置間的關聯程度，使模型能有效捕捉輸入資料中的長距依賴與語義關係，顯著提升了序列解析的能力。

此外，作為激活函數所採用的 GELU 函數，提供了平滑且具備非線性特性的轉換能力，相較於傳統 ReLU 函數，更有助於模型學習複雜的表示結構，進一步增強表現力。

最後，為確保模型在訓練過程中的穩定性，Transformer 採用了 Add & Norm 模組，其中的 Layer Normalization 技術已被實證能有效緩解梯度不穩問題，加速收斂並提升整體訓練效果。Transformer 中 block 的核心架構圖如圖 2.6 所示。

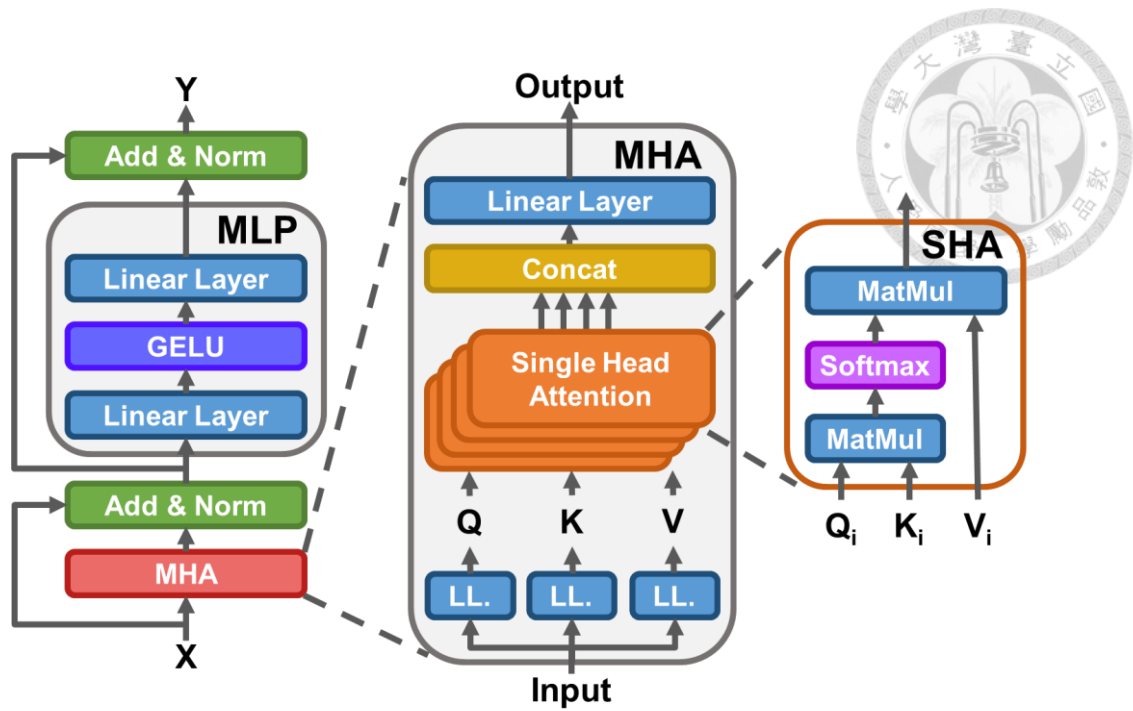


圖 2.6 Transformer block 核心架構圖

綜上所述，這些關鍵模組共同構成了 Transformer 架構的核心基礎，使其成為當前自然語言處理與其他序列建模任務中極具影響力的模型之一。

第三章 三元權重與近似 運算神經網路訓練與推論

3.1 訓練模型與資料集

本研究為驗證所提出的技巧與演算法能否適用於各式 transformer 架構衍伸之模型與任務，因此於三種常見應用：影像辨識(Image Classification)任務、文字生成(Text Generation)任務、語音翻譯(Speech Translation)任務上進行量化感知訓練，並評估模型量化後的效能。

本章節會針對各任務所使用之模型與資料集分別進行介紹。

3.2 影像辨識任務

影像辨識是電腦視覺領域基本且具代表性的任務之一，此任務目標為將輸入影像分類至預先定義的類別中，應用涵蓋醫學影像分析、自動駕駛等領域。傳統的影像辨識多使用卷積神經網路(Convolutional Neural Networks, CNNs)[6]進行實作。本研究使用 Vision Transformer 及其衍伸模型，於 ImageNet 資料集上進行訓練。

3.2.1 ImageNet 資料集

本研究採用 ImageNet 資料集進行影像辨識任務的模型訓練與推論。ImageNet 資料集是目前最廣泛運用且最具代表性的影像辨識任務標註資料集之一，ImageNet 包含超過 1400 萬張影像，並涵蓋超過 2 萬個標註類別。ImageNet

project 自 2010 年始每年舉辦 ImageNet Large Scale Visual Recognition Challenge(ILSVRC)影像分類競賽，該競賽的資料集從眾多類別中選擇 1000 個不重複類別供參賽者使用。



在電腦視覺研究領域中，最常見的資料集便是 ImageNet-1K，此資料集代指的是 ILSVRC 2012-2017 image classification and localization dataset[7]，是在 2012 年的 ILSVRC 競賽中使用的 1000 個類別的資料集。此資料集包含超過 128 萬張訓練(Training)影像、5 萬張驗證(Validation)影像以及 10 萬張測試(Test)影像。且隨著影像辨識模型參數量上升，訓練所需的成本也隨之增加，因此部分研究使用 ImageNet-1K 的子資料集 ImageNet-100 進行訓練與驗證，在考量訓練時間與硬體成本後，本段落中也以 ImageNet-100 資料集進行 ViT 模型的訓練與驗證。



圖 3.1 ImageNet 資料集[8]

影像辨識的訓練資料前處理說明如下，在讀取 ImageNet 的 raw data 後，會進行一系列的影像處理 data augmentation 步驟，包括影像縮放、水平翻轉、垂直翻轉、顏色微調(color jitter)、正則化等處理，並且使用的是隨機 augmentation 策略，以增強模型對抗雜訊的能力。其中影像縮放的部分會採用 bicubic interpolation 策略使內插的像素點更加平滑。之後便會送入模型中執行 patch embedding 將輸入影像轉為模型可處理的 token。

3.2.2 Vision Transformer 簡介

最初 Transformer 架構被提出時，主要用於處理自然語言任務，然而其優秀表現也促使研究者將其用於其他領域，包括影像辨識等電腦視覺任務，而使用

transformer 架構的模型也展現出超越傳統 CNN 模型的表現。隨著研究不斷進行，也出現許多基於 transformer 架構，並針對影像辨識任務進行運算優化處理的模型，這類模型便被通稱為 Vision Transformer。包括最早被提出的 ViT[9]，以及後續衍伸出的 DeiT[10]、DaViT[11]、Swin[12]等，後續會對這些模型架構分別進行介紹。

3.2.2.1 ViT 與 DeiT

Vision Transformer(ViT)[9]是首個將完整 Transformer 架構應用於電腦視覺相關任務的模型，ViT 的架構圖如圖 3.2 所示，保持原有 Transformer 架構，包含 Multi-Head Attention、MLP、LayerNorm 以及各層間的 Residual Connection 等，且可以透過增加 Transformer block 的數量來換取更高的效能。而輸入資訊的處理方式也與 Transformer 在進行自然語言處理輸入的方式相似，透過將輸入影像分割為多個區塊(Patch)，透過 Linear Projection 將 Patch 轉為固定長度之輸入，並加上位置編碼(Position Embedding)後送入 Transformer 模型中。

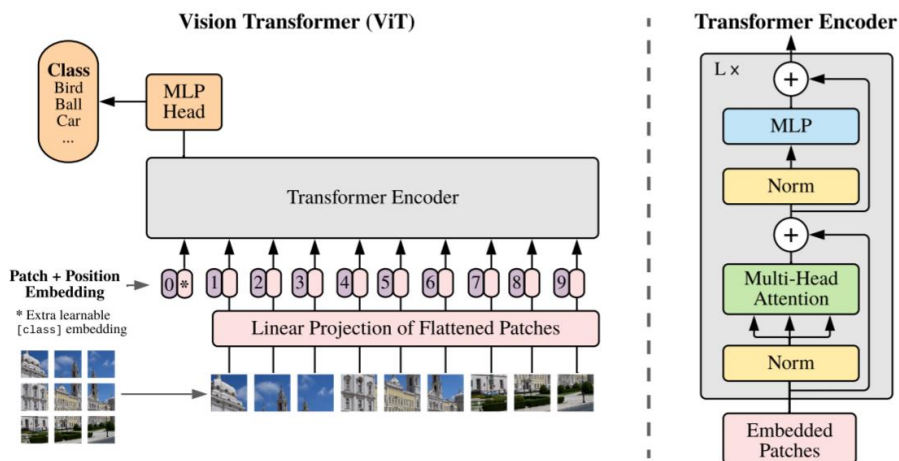


圖 3.2 ViT 模型架構圖

整體運算過程如(式 3.1)所示，其中(H, W)是原始 image 的解析度，(P, P)是每個 image patch 的解析度，C 是 image 的 channel 數量， $N = \frac{HW}{P^2}$ 是分割後的 patch 數量。 $\mathbf{x}_{class}$ 是一個用於總結輸入資訊的特殊向量，與 BERT 的做法相似，會插入於 embedded patch 的前方，用於歸納圖像的特徵，以利後續分類的步驟。 $\mathbf{E}_{pos}$ 則是位置編碼的矩陣，ViT 使用模型自主學習的位置編碼進行實作。

$$\begin{aligned}
 \mathbf{z}_0 &= [\mathbf{x}_{class}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{pos}, & \mathbf{E} &\in \mathbb{R}^{(P^2 C) \times D}, \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D} \\
 \mathbf{z}'_\ell &= MSA(LN(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, & \ell &= 1 \dots L \\
 \mathbf{z}_\ell &= MSA(LN(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, & \ell &= 1 \dots L \\
 \mathbf{y} &= LN(\mathbf{z}_L^0)
 \end{aligned}$$

(式 3.1)[9]

3.2.2.2 DaViT

DaViT 模型的全名為 Dual Attention Vision Transformer，顧名思義相比於原始 Transformer 使用單一 Multi-Head Attention，作者在 DaViT 模型中使用了 Dual Attention 的技巧，分別是處理空間維度資訊的 Spatial Window Multihead Self-Attention 以及處理通道維度資訊的 Channel Group Self-Attention。Spatial Window Multihead Self-Attention 的目的在於改善 Transformer 的傳統 Attention 方式無法有效獲取圖片裡各區域間關係的資訊的缺點，而 Channel Group Self-Attention 透過讓各個 channel 交流各自獲取的特徵，可以在將圖片分割成多個 patch 的情況下仍能掌握全局的特徵。

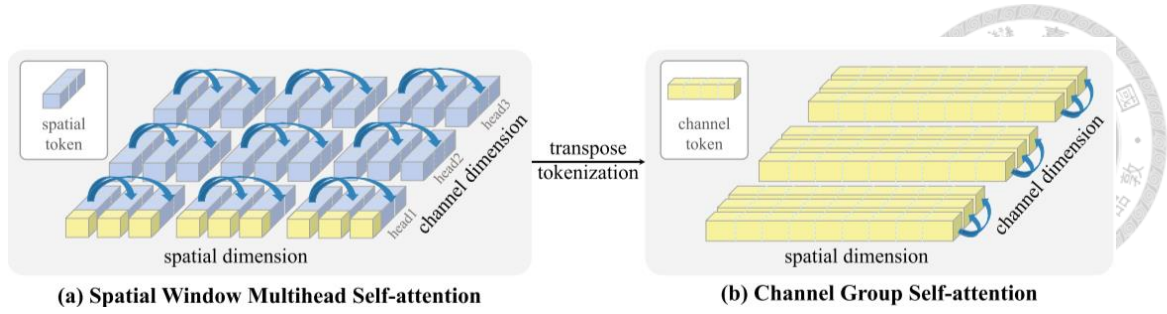


圖 3.3 DaViT 中 Dual Attention 示意圖[11]

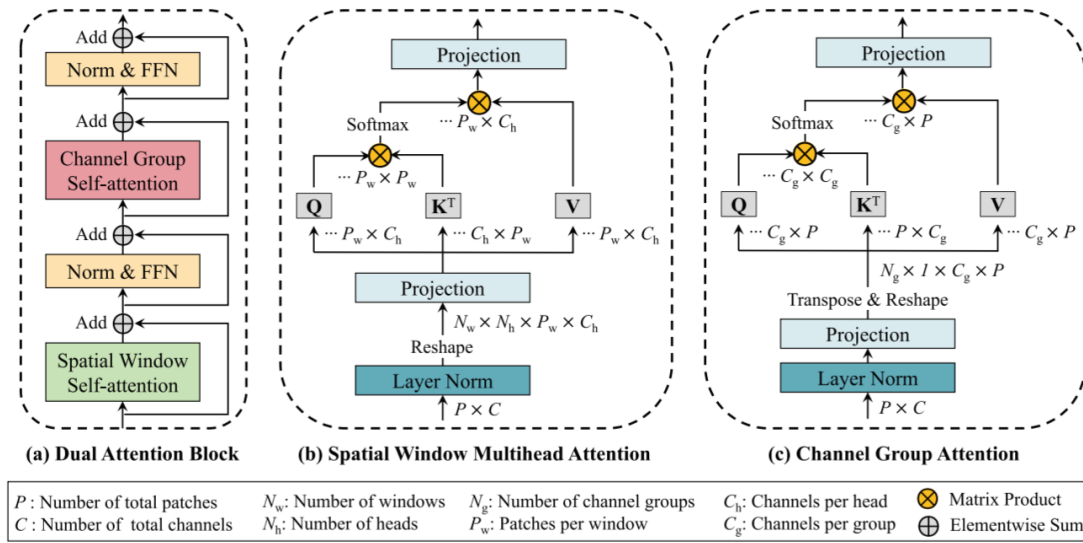


圖 3.4 Dual Attention Block 架構圖[11]

圖 3.5 中可以觀察 Channel Group Self-Attention 的行為，右側七個 column 展示的是經過 Spatial Window Multihead Self-Attention 的 attention 結果分布圖，將這些資訊輸入 Channel Group Self-Attention 後，可以匯集各區域的 attention 資訊，整理成包括全域資訊以及區域資訊的 attention 結果，如第二個 column 所展示。此結果可以驗證 Channel Group Self-Attention 可以有效彙整影像中，散佈於影像各處的全局重要資訊。

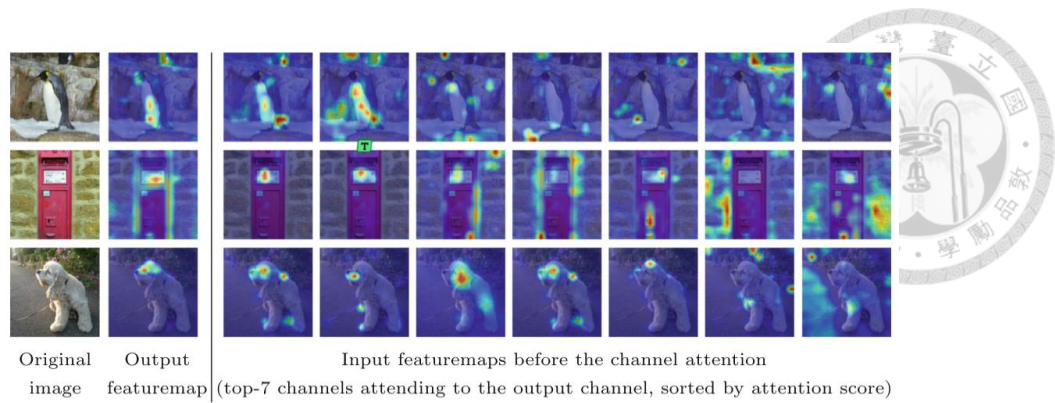


圖 3.5 Channel Group Self-Attention 結果範例[11]

由圖 3.6 中 Feature Map 的視覺圖像可以發現相比於其他 ViT 模型，DaViT 由於使用 Dual-Attention 的技巧，使得影像重要資訊的提取更為明確也集中。論文中也使用 ImageNet-1K 作為訓練資料進行模型訓練，證明 DaViT 擁有超越其餘 ViT 模型的表現。

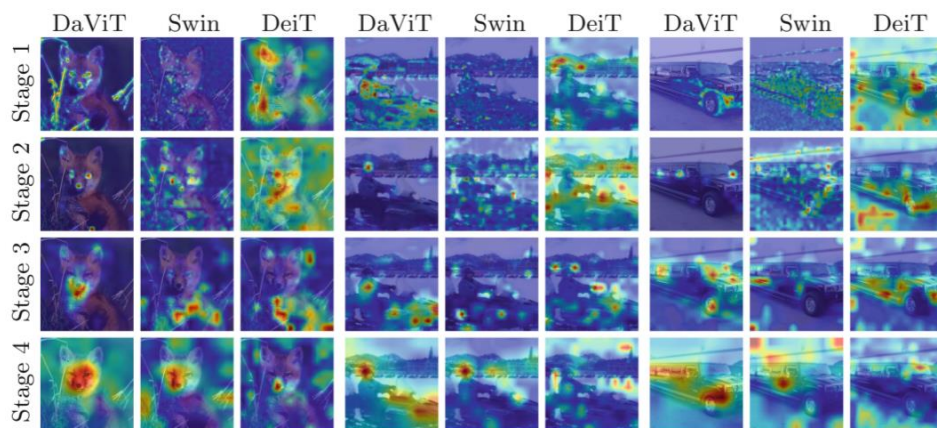


圖 3.6 ViT 系列模型之 Feature Map 比較[11]

3.2.2.3 Swin

Swin[12]提出 Windows Multi-Head Self-Attention(W-MSA)的技巧，相比於 ViT 會對整個特徵圖計算 MHA，Swin 僅會對各個 patch 內部的數值進行計算，可以有效降低運算量。

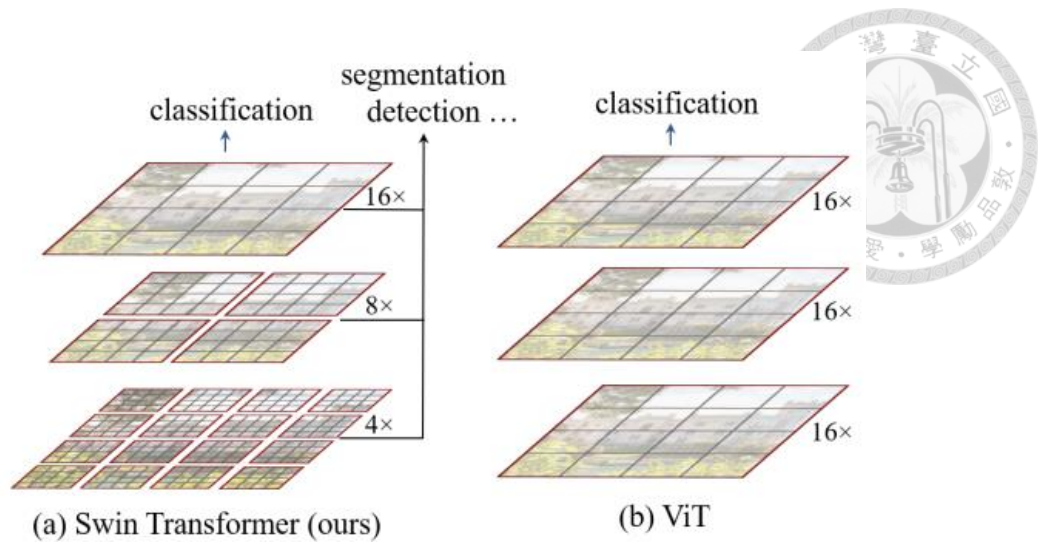


圖 3.7 Swin 與 ViT 之 MHA 計算範圍差異[12]

不過由於若只對各個 patch 單獨計算 MHA，可能出現 patch 間資訊不流通導致模型提取特徵的能力下降的情況，因此該研究團隊提出了 Shifted Windows Multi-Head Self-Attention(SW-MSA)的技巧，透過偏移 patch 的位置，來達到 patch 間資訊交換的目的。

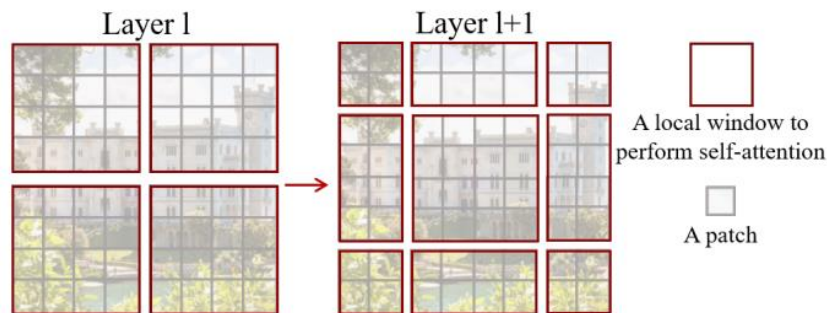


圖 3.8 Swin 模型採用之 Shifted Window 作法[12]

不過由圖 3.8 可以觀察到，採用 SW-MSA 偏移過後的 patch 變成了九個區塊，這使得 SW-MSA 的計算複雜度上升。為了同時保留 W-MSA 計算量低的優勢與 SW-MSA 訊息交換的特性，[12]提出 cycle-shift 與 masked-MSA 的技巧，首先透過 cycle-shift 把較小區塊的 patch 與大區塊合併，使結果回到 4 個 patch。

Masked-MSA 則是為了解決位移後的資料與相鄰資料間不連續的問題，在 cycle-shift 後會重新劃分 patch 範圍，而 MSA 仍僅對各個 patch 進行計算，因此不會因為 shift 後的資料造成混淆。

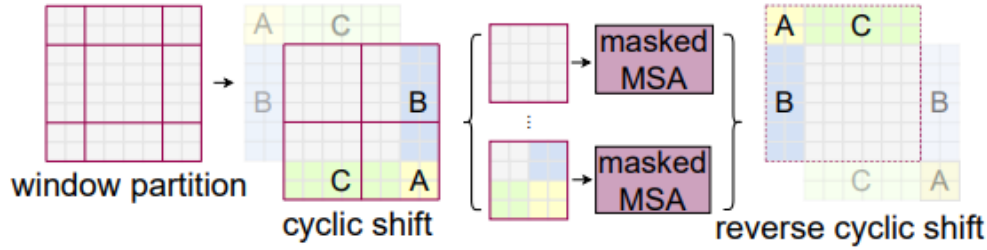


圖 3.9 Cycle-shift 與 masked-MSA[12]

圖 3.10 為 Swin 的整體模型架構，由圖中右側的 block diagram 可以觀察到使用 W-MSA 與 SW-MSA 的 block 是交替出現的，以兩兩一組的形式組成 swin 的核心架構。經實驗證明此篇研究提出之作法可以有效降低整體模型運算量，同時提升影像辨識任務的效能。

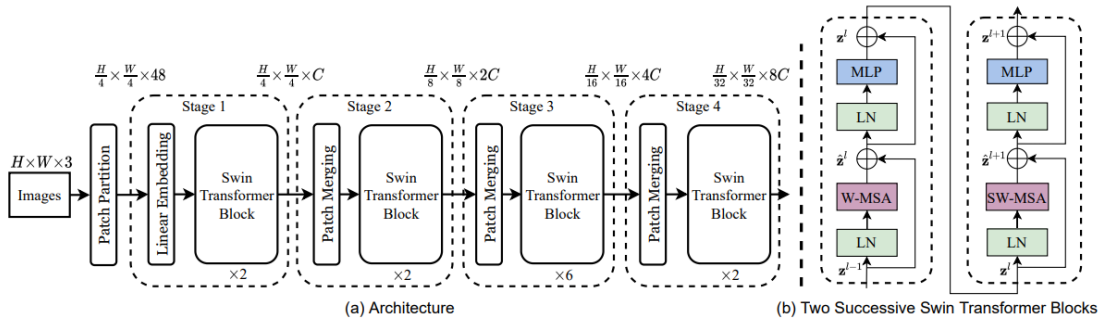


圖 3.10 Swin 模型架構[12]

3.3 文字生成任務

文字生成任務是自然語言處理中的核心任務之一，目標是根據輸入的文字或其他資訊，生成語意通順、文法正確且符合上下文文意的文字輸出。應用涵蓋文章撰寫、自然語言對話系統等應用場域，是 transformer 模型最常見的應用

之一。這類任務常見的模型有 OpenAI 提出的 GPT[13]系列、Google 提出的 T5[14]、Meta 提出的 LLaMA[15]系列等大型語言模型，在本研究中，考量到運算資源與訓練時間，我們選用 GPT-2[16]來進行量化模型的訓練與推論。



3.3.1 OLM & WikiText-2 資料集

Online Language Modeling(OLM)是機器學習領域中常見的文本資料集，由定期抓取的線上文本資料與 wikipedia 中的文字資訊組成。在此研究中，我們使用 olm/olm-december-2022-tokenized-1024 這個 OLM 的子資料集進行模型 pretrain 訓練。在 finetune 部分，我們使用 wikitext-2 資料集，這是一個由維基百科提取的資料集，包含約 20 萬個語句與 60 萬個詞彙，為文字生成類模型常用之訓練資料集。

文字生成類模型效能常見的衡量標準為困惑度(perplexity)，困惑度可以理解為模型輸出與測試資料之 cross entropy 的 exponential 結果，因此當 perplexity 越低時，往往表示該模型擁有更好的效能。

資料前處理部分，在讀取 raw data 後，首先要使用 tokenizer 將文字轉為模型可以處理的 token，接著要確認 tokenizer 的詞彙數量是否大於模型本身的 embedding 長度，如果詞彙數量大於 embedding 維度，則需要擴充模型的 embedding 維度以符合 tokenizer 的編碼。否則會出現 index out of range 的情形。接著，依照 block size 將字串分為多個 chunk，並設定訓練資料的 label，不過由於文字生成模型訓練時的預測目標就是訓練字串的下一個字元，因此 label 其實就是 input id 位移一位之後的結果(下一個字元的 id)。完成後便獲得能夠輸入模型進行訓練的資料集。



3.3.2 GPT-2 簡介

GPT(Generative Pre-trained Transformer)是由 OpenAI 提出的 transformer 模型系列，採用 transformer 架構中的 decoder 部分進行實作，為主要應用於生成式任務的語言模型。由最初的 GPT-1[13]開始，目前已演進至 GPT-4[17]。GPT-1 已證明該模型架構能在各項生成式任務的指標中階達到 SOTA 的成果，GPT-2 則進一步增加模型的參數量與訓練資料量，由 GPT-1 的 1.17 億的模型參數量提升至 GPT-2 的 15 億，訓練資料量也提升十倍之多。且經過訓練可以在無需要針對特定任務進行微調的前提下，便能在多項語言任務上展現強大的 zero-shot 能力。

GPT 系列的訓練方式為先進行模型的預訓練(pretrain)，接著進行模型的微調(finetune)，預訓練時使用的訓練資料多為大量截取自網路的文本資料，例如上個段落提到的 OLM 資料集，而微調訓練階段則多使用與目標任務相關的資料集進行訓練以充分發揮模型的效能。

3.4 語音轉文字任務

語音轉文字(Automatic Speech Recognition, ASR)是一種將輸入語音訊號轉換為對應文字序列的技術，其核心目標在於讓機器能夠聽懂人類語言，並以文字形式準確地重建語者的語意內容。傳統 ASR 系統通常由三大部分構成：聲學模型、語言模型，以及發音詞典，分別負責將語音特徵對映至音素、判斷語句合理性與流暢度，以及建立語音與文字之間的對應關係。然而，近年隨著深度學習技術的蓬勃發展，特別是序列到序列(Seq2Seq)模型與 Transformer 架構的引入，端到端 ASR 系統已成為主流，將上述各模組整合於單一神經網路架構中，簡化系統流程並提升辨識準確率。

隨著訓練資料集規模的不斷擴增，以及模型架構的持續創新，ASR 技術正朝向多語言、多任務與低資源語言的廣泛應用邁進，成為人機互動與智慧語音系統中不可或缺的關鍵技術。



3.4.1 LibriSpeech 資料集

LibriSpeech[18]是一個大型英語朗讀語音資料集，提供高品質的約 1000 小時的英文朗讀音訊資料。LibriSpeech 已成為語音辨識領域的標準基準資料集，並經過文本標註，許多模型皆在其上進行訓練與比較，並常以詞錯率 (Word Error Rate, WER) 作為主要評估指標。WER 的計算方式是以單一個詞為單位計算，可以用來評估一個句子中需要針對多少詞進行修正才能與測試資料相同。

資料前處理的部份我們會先確保訓練資料的採樣率與訓練時的 feature extractor 的採樣率相同，若有差異則會將訓練資料以 feature extractor 的採樣率重新採樣以確保整體資料的採樣率相同。接著將音訊轉為用於模型輸入的特徵，此外也對音訊的文字標註進行處理(例如將其轉為小寫以避免因 case 造成的比對誤差)，並使用 tokenizer 將字串轉為 label id。接著我們會依照音訊長度篩選掉長度過長的輸入以保持模型效能與訓練梯度穩定。最後將 feature 與 label 都 pad 到相同長度以利模型輸入。

3.4.2 Whisper 簡介

Whisper[19]由 OpenAI 提出，是一個通用語音辨識模型。Whisper 採用 Encoder-Decoder Transformer 架構，整體結構類似於序列到序列(seq2seq)模型。Whisper 主要創新為採用大規模的弱監督學習(weakly supervised pretraining)來達到最佳的模型效能，並且使用多工(Multitask)訓練資料進行訓練，能夠在不同任務與不同語言上進行翻譯、語言檢測等任務。模型的訓練過程如圖 3.11 所示。

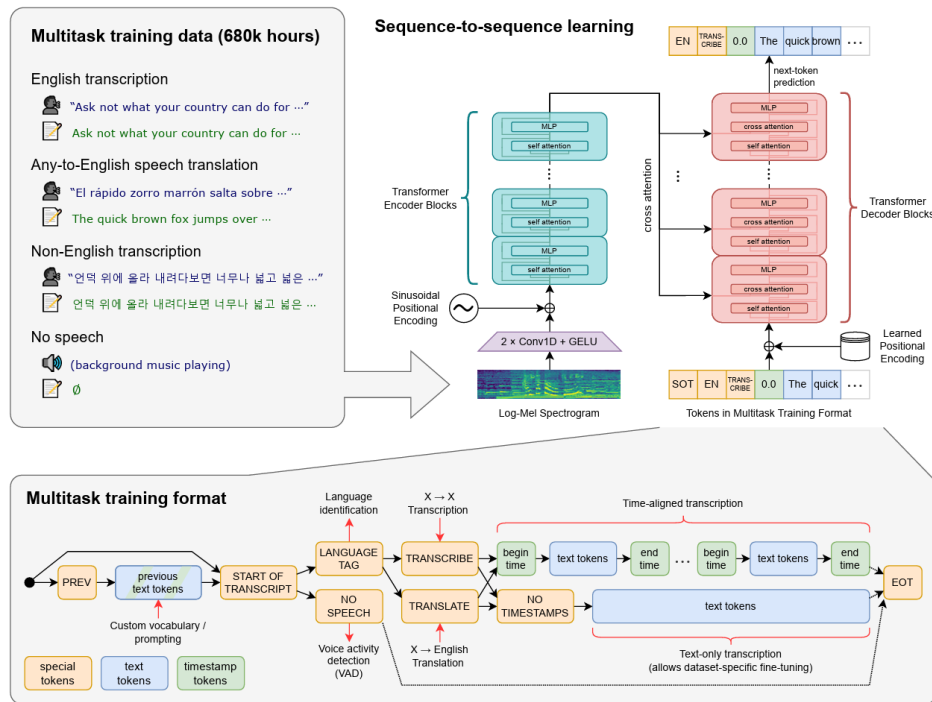


圖 3.11 Whisper 核心架構與訓練流程

3.5 三元權重神經網路簡介與實驗結果

此章節中會進行神經網路參數量化的介紹，包括模型權重的三元量化，以及輸入的 INT-4 量化。

3.5.1 神經網路量化

為了降低 Transformer 模型在矩陣乘法運算上所帶來的龐大計算負載，本研究使用簡稱為 TW4A 的量化方法，即於模型所有運算過程中採用三元權重 (Weight) 與 INT4 輸入 (Activation)，應用於多頭注意力與前饋神經網路這兩個運算最密集的核心模組。透過 TW4A 低精度運算，可以大幅減少乘加運算的硬體運算複雜度與資源消耗，同時維持與全精度模型相近的效能表現。

在現有研究文獻中，[20]提出了以 1-bit 權重為核心的 BitNet，替換 Transformer 架構中的 linear layer，並於大型語言模型(LLM)上進行訓練，架構如圖 3.12。證明低精度的權重在 LLM 的各式 task 中仍能保有模型效能。

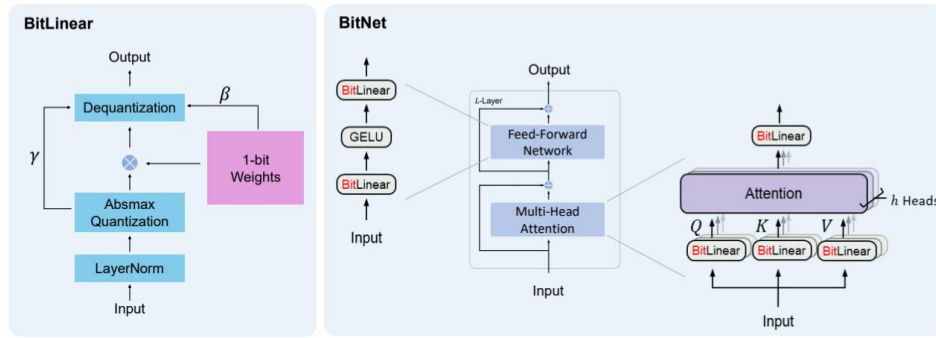


圖 3.12 BitNet 架構[20]

爾後更是提出 BitNet 的改良版本 BitNet b1.58[21, 22]，透過 ternary 權重與 8-bit 輸入的結合，能進一步提升 LLM 的效能，且 ternary 包含 0 的特性使得模型可以做到 feature filtering，避免 binary 權重下的干擾，並證明 ternary 權重能在 LLM 中有著優異的表現，如表 3.1。由表中可以觀察到 ternary weight 使得 memory 的需求大幅下降，並在不同參數量的模型中都有明顯的 inference 速度提升，同時模型效能(PPL)與原版模型相近甚或微幅提升。

表 3.1 BitNet b1.58 於 LLaMA LLM 訓練結果[21]

Models	Size	Memory (GB)↓	Latency (ms)↓	PPL↓
LLaMA LLM	700M	2.08 (1.00x)	1.18 (1.00x)	12.33
BitNet b1.58	700M	0.80 (2.60x)	0.96 (1.23x)	12.87
LLaMA LLM	1.3B	3.34 (1.00x)	1.62 (1.00x)	11.25
BitNet b1.58	1.3B	1.14 (2.93x)	0.97 (1.67x)	11.29
LLaMA LLM	3B	7.89 (1.00x)	5.07 (1.00x)	10.04
BitNet b1.58	3B	2.22 (3.55x)	1.87 (2.71x)	9.91
BitNet b1.58	3.9B	2.38 (3.32x)	2.11 (2.40x)	9.62

BitNet 研究團隊也進一步提出 BitNet a4.8[23]，將原先 8-bit 的 activation 降為 4-bit 與 8-bit 混合使用的精度，在 attention 與 MLP 的輸入端使用 4-bit，輸出端則使用 8-bit 以維持結果精度，而權重則維持 ternary 的 1.58-bit 權重，BitNet a4.8 架構如圖 3.13。實驗結果證明 BitNet a4.8 與 BitNet b1.58 有著相近的模型效能。

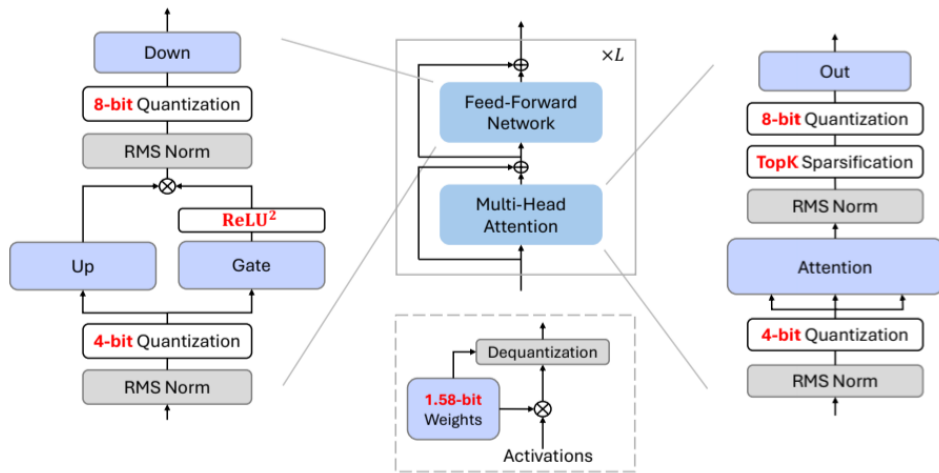


圖 3.13 BitNet a4.8[23]

最後，BitNet 團隊提出的 BitNet v2[24]中採用了結合 Hadamard Transform 的 H-BitLinear，並將 activation 完全降低至 4-bit(BitNet v2 a4)，仍能保有與 ternary weight + INT8 activation 相近的正確率，BitNet 各版本的訓練結果如表 3.2，BitNet v2 a4 在不同參數量 LLM 的訓練結果足以證明即使是在 LLM 的範疇，TW4A 的精度設定在大幅降低硬體需求的同時，也能保有優異的模型表現。

表 3.2 BitNet 系列模型訓練結果[24]

Models	Size	PPL↓	ARCC↑	ARCE↑	HS↑	PQ↑	WGe↑	LBA↑	Avg↑
BitNet b1.58	400M	13.37	24.32	43.01	39.51	64.91	51.93	45.51	44.87
BitNet a4.8		13.61	24.15	41.75	39.48	65.18	53.59	44.34	44.75
BitNet v2 (a8)		13.50	23.29	43.06	39.06	64.74	50.59	45.26	44.33
BitNet v2 (a4)		13.78	23.29	41.46	38.33	65.45	50.59	44.56	43.95
BitNet b1.58	1.3B	11.02	27.90	49.58	48.85	69.80	55.80	54.12	51.01
BitNet a4.8		11.15	27.47	49.20	48.72	69.64	56.51	53.85	50.90
BitNet v2 (a8)		11.14	27.90	49.96	48.37	69.42	57.22	54.14	51.17
BitNet v2 (a4)		11.33	27.56	49.58	48.00	68.23	55.49	53.58	50.41
BitNet b1.58	3B	9.71	28.84	54.80	56.39	71.44	59.35	60.47	55.22
BitNet a4.8		9.80	29.01	55.01	55.92	71.76	59.59	59.85	55.19
BitNet v2 (a8)		9.72	30.55	55.56	57.19	71.33	58.72	60.90	55.71
BitNet v2 (a4)		9.85	28.92	55.01	56.59	71.65	59.67	60.74	55.43
BitNet b1.58	7B	9.09	31.74	59.51	61.49	74.37	59.98	61.63	58.12
BitNet a4.8		9.16	31.91	59.09	61.06	74.16	59.67	61.54	57.91
BitNet v2 (a8)		9.14	32.94	58.54	61.08	74.10	61.48	64.22	58.73
BitNet v2 (a4)		9.24	32.42	58.00	60.71	74.27	60.85	63.52	58.30

除了 LLM 外，[25]也於 ViT 上證明 ternary 權重的有效性，該論文使用的是 ternary 權重加上 8-bit 輸入的組合，經過訓練發現可以超越全 4-bit 的 accuracy，與 16-bit ViT 也約僅有 1% 的 top-1 accuracy loss。訓練結果如表 3.3 所示。

表 3.3 ViT-b1.58 訓練結果[25]

Method	Memory Cost	CIFAR-10		ImageNet-1k	
		train loss	test acc.	Top-1	Top-3
ViT-L	1.14G	0.024	76.28	76.54	90.23
16bit Inference ViT-L	585M	-	74.61	75.29	87.44
8bit Inference ViT-L	293M	-	72.20	74.11	85.26
4bit Inference ViT-L	146M	-	70.69	70.88	82.50
ViT-1.58b-L (ours)	57M	0.026	72.27	74.25	85.78

與傳統的二元權重量化(Binary Quantization)相比，三元權重具有以下幾項重要優勢：



1. 更高的表現能力：三元權重允許權重賦值三種數值(-1,0,1)，比起僅能取值-1, 1 的二元權重提供了更高的數值表達精度與自由度，能夠捕捉更細緻的特徵差異，進而提升模型的學習與推論效能。
2. 乘法運算簡化：由於三元權重中包含 0，對應的乘法運算可以直接跳過或將結果置為零，而與-1 與 1 相乘的運算也可透過簡易硬體實現，沒有傳統乘法器的硬體需求，大幅簡化硬體設計與運算路徑。
3. 自然稀疏性：經過訓練的三元權重模型自然地引入了權重稀疏性(sparsity)，在硬體上若搭配稀疏感知(sparsity-aware)計算技術與資料存取架構，能有效降低記憶體讀取數據量與運算能耗，進一步縮短推論延遲並降低功率消耗，而不會對模型精度造成明顯影響。

綜上所述，TW4A 提供了一種兼顧運算效率、硬體成本以及模型表現的模型量化方案，適用於追求低功耗、高 throughput 與小面積部署需求的深度學習加速系統設計。

3.5.1.1 三元權重量化

本研究將全精度 weight 量化為三元權重的方法，參考 TNN[26]所提出的量化策略，實作的數學公式可參考(式 3.2)。此方法的核心概念為：透過設定一個閾值(threshold)參數 Δ ，判斷每個權重值應被量化為+1、0 或-1，同時引入一個縮放因子 α ，以有效減少量化所帶來的誤差，維持模型的表現能力。

$$\Delta = 0.7 \times \frac{1}{N} \sum_{i=1}^N |w_i|$$



$$\tilde{w}_i = \begin{cases} +1 & \text{if } w_i > \Delta \\ 0 & \text{if } |w_i| \leq \Delta \\ -1 & \text{if } w_i < -\Delta \end{cases}$$

$$mask_i = \mathbb{I}(|w_i| > \Delta)$$

$$\alpha = \frac{\sum_i^N |w_i| \cdot mask_i}{\sum_i^N mask_i}$$

(式 3.2)[26]

3.5.1.2 輸入量化

本研究中，activation 的數值表示採用 INT4 精度，此選擇為經由模型訓練實驗驗證所得結果，綜合考量硬體運算成本與模型準確率後，所達成的 tradeoff。低精度的數值格式能有效降低乘法與加法等基本運算的硬體複雜度，進而節省晶片面積與功耗，特別適用於資源受限的推論環境中。

為提升量化模型的效能表現，我們於訓練階段採用[27]所提出之 Learned Step Size Quantization+(LSQ+)技巧，其核心量化公式如(式 3.3)所示，量化數值還原的計算方式則為(式 3.4)所示。與傳統的靜態量化方法不同，LSQ+將量化參數 scale 與 zero-point 視為可學習參數，並與模型權重一同透過反向傳播進行更新，能使模型根據實際資料分布自動調整最佳的量化配置，進而降低因量化誤差所導致的精度損失。

$$X_q = \text{clamp} \left(\text{round} \left(\frac{X - Z}{S} \right), q_{min}, q_{max} \right)$$

(式 3.3)

$$X' = SX_q + Z = S \left(\text{clamp} \left(\text{round} \left(\frac{X - Z}{S} \right), q_{min}, q_{max} \right) \right) + Z$$

(式 3.4)



在模型訓練過程中，scale 與 zero point 的調整過程會分成三個不同階段。第一階段程式會截取 activation 的最大值與最小值，並以此計算出初始化的 scale 跟 zero point，運算過程如(式 3.5)。

$$\begin{aligned}
 x_{max} &= \max(activation_tensor) \\
 x_{min} &= \min(activation_tensor) \\
 scale_{init} &= \frac{x_{max} - x_{min}}{q_{max} - q_{min}} \\
 zero_point_{init} &= x_{min} - scale_{init} \times q_{min}
 \end{aligned}
 \tag{式 3.5}$$

到了 LSQ+第二個階段，會採用滾動式更新的方式更新 max 與 min，如(式 3.6)，而 scale 與 zero point 的計算方式與(式 3.5)的運算相同。

$$\begin{aligned}
 x'_{max} &= 0.9 \times x_{max} + 0.1 \times \max(activation_tensor) \\
 x'_{min} &= 0.9 \times x_{min} + 0.1 \times \min(activation_tensor) \\
 scale_{init} &= \frac{x'_{max} - x'_{min}}{q_{max} - q_{min}} \\
 zero_point_{init} &= x'_{min} - scale_{init} \times q_{min}
 \end{aligned}
 \tag{式 3.6}$$

到了 LSQ+的第三個階段，會直接將 scale 與 zero point 視為模型變數，並使用反向傳播的方式進行參數數值更新，與一般的模型參數相同。

值得注意的是，在整數乘法運算中，量化值若超出所支援之整數範圍，可能導致 overflow 的問題。舉例而言，INT4 的完整表示範圍為 $[-8, 7]$ ，若進行 $-8 \times$

-1 的乘法，結果為+8，已超出 INT4 表示範圍。為避免此類情況，本研究對 activation 之有效值域進行約束，將其限制為 $[-7, 7]$ ，在不顯著影響模型表現的前提下，確保所有後續運算皆可安全地在 INT4 資料精度中執行，提升整體系統的數值穩定性與硬體可實現性。下文中提及 INT4 精度時，接代指區間約束後的 INT4。

3.5.2 三元權重神經網路訓練

由於我們使用的 TW4A 低精度運算，若直接進行 Post-Training Quantization(PTQ)會產生明顯的 accuracy loss，因此我們使用 Quantization Aware Training(QAT)的技巧進行訓練，使量化模型可以在訓練過程中適應數值量化造成的資訊損失與量化雜訊。QAT 雖然會顯著增加訓練模型的時間，但經實驗在經過適當的訓練後，量化模型往往能夠將 accuracy loss 控制在可接受的範圍之中。

不過由於量化後的數值並不連續，因此無法直接以該數值進行梯度的計算，因此在訓練過程中我們仍會保存模型的 FP32 權重，並使用該 master copy 進行梯度計算以更好的進行訓練，僅會在 forward pass 的時候對數值進行量化。此外在量化的過程中，部分函數也是不可微的，例如(式 3.3)的 rounding 跟 clamping，而此部分的反向運算我們參考[28]的作法，透過 Straight Through Estimator(STE)的技巧避開不可微的環節，也就是將這些函數的微分視為 1。

在訓練過程中，若直接對 FP32 模型施加 TW4A 的量化精度，即使有 QAT 的幫助仍會在訓練初期出現顯著的 accuracy loss，不僅拉長整體訓練時間，也可能因誤差平面改變太過劇烈導致模型無法收斂至較好的結果。

為此我們使用了兩個技巧以減輕此處的影響，首先我們會先將 FP32 模型訓練至 TW32A，也就是僅對權重部分進行 QAT 訓練，輸入部分則維持 FP32 的高精度表示。待 TW32A 的訓練結果穩定後，接著訓練 TW4A 的量化模型，以此讓模型過渡到目標精度。

此外，在每一個訓練環節中，我們也使用 progressive quantization(PQ)的技巧以降低量化造成的誤差。首先設定一個 progressive ratio 做為控制項，並隨機產生一組 bitmap，該 bitmap 中會有與 progressive ratio 相同比例的數值為 1，其餘數值為 0。最後我們依照 bitmap 的選擇將量化數值與原始數值結合成新的 tensor。隨著訓練進行，progressive ratio 會由初始的 0 逐漸過渡至 1，實作方式如圖 3.14 所示。

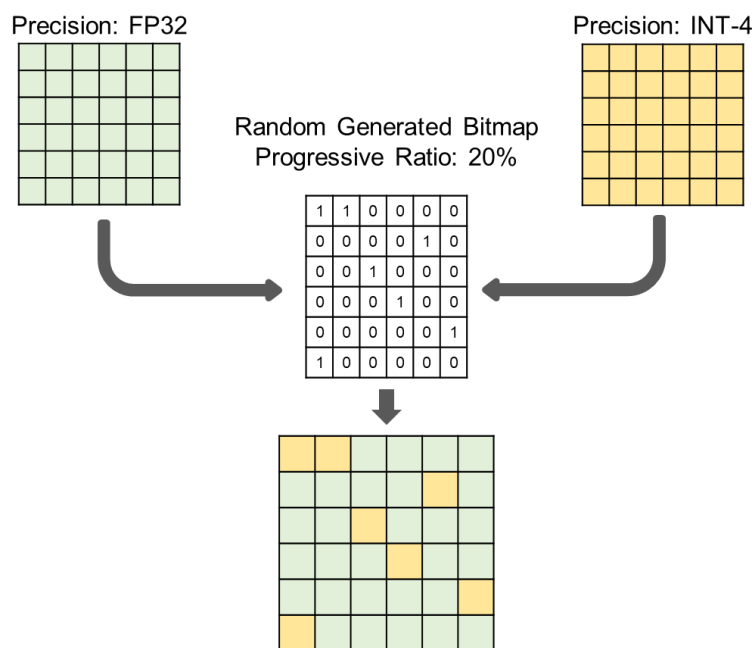


圖 3.14 Progressive Quantization

而在實際硬體中運算時，會盡可能使數值保持量化後的精度，以避免過多的定點數運算使複雜度上升。因此，我們會將前一層輸出的量化還原運算與下

一層的輸入量化運算結合。如此一來便可以避免掉一組定點數的乘加運算，並使前一層的輸出數值可以直接被用於下一層的運算中。



首先 linear layer 的輸出 Y 可以統整如以下(式 3.7)：

$$\begin{aligned}
 X' &= S_x X_q + Z_x \\
 W' &= S_w W_q \\
 Y &= X'W' + B = (S_x X_q + Z_x)S_w W_q + B \\
 &= S_x S_w S_x X_q W_q + S_w W_q Z_x + B \\
 &= S' X_q W_q + B'
 \end{aligned}
 \tag{式 3.7}$$

而輸出 Y 也可以表示如以下(式 3.8)，因此 Y_q 可以整理為(式 3.9)的表示， S_{merge} 與 Z_{merge} 皆為定點數形式，可以由外部存入晶片內進行運算。

$$Y = S_y Y_q + Z_y \tag{式 3.8}$$

$$\begin{aligned}
 Y_q &= \frac{S' X_q W_q + B' - Z_y}{S_y} \\
 &= \frac{S'}{S_y} X_q W_q + \frac{B' - Z_y}{S_y} \\
 &= S_{merge} X_q W_q + Z_{merge}
 \end{aligned}
 \tag{式 3.9}$$

3.6 複雜函數近似

此章節針對兩個 transformer 架構常見之非線性函數，Softmax 與 GELU，進行硬體友善的近似，並介紹現有研究之硬體實作方式。

而在進行 QAT 訓練時，不同於前一段落 TW4A 訓練使用的 PQ，我們使用 Progressive Mixed Precision(PMP)的方式來降低函數近似造成的影響。實作的方式如圖 3.15 所示。與 PQ 相同，progressive ratio 會從 0 漸漸增長為 1，並維持一段時間直到訓練結果穩定。但差別在於，我們會將原始與近似後的結果進行線性組合，使模型逐步適應函數近似造成的誤差。

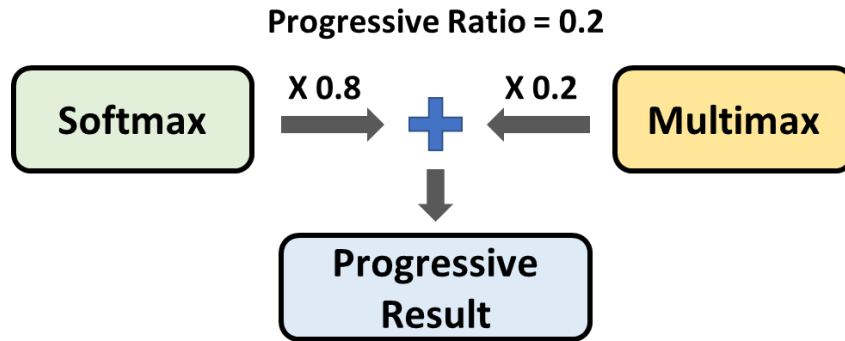


圖 3.15 Progressive Mixed Precision

3.6.1 Multimax

3.6.1.1 Softmax 硬體實作技巧比較

Softmax 在自注意力機制中扮演重要角色，然而，如(式 2.1)中可以觀察到其中包含 exponential、除法等高複雜度運算，導致硬體實現變得困難。為了克服此問題，已有許多先前研究提出各種針對硬體實作友善的方案。其中一類方法 [29, 30] 採取直接實現原始函數的策略，以維持數值準確度，避免對模型效能造成影響。然而，這類設計往往伴隨高電路面積、功耗等代價，對資源受限的應用場預而言並不理想。



此外，部分研究使用包括 LUT[31, 32]以及泰勒級數展開[33]的技巧。這些方法在硬體成本上確實具備優勢，但其設計仍是從「近似原始數學公式」的角度出發，並未針對硬體效率進行函數的重新設計與優化。仍難以符合資源高度受限場域的需求。

3.6.1.2 構想與實現

透過實驗觀察，我們發現 Softmax 函數在多數情況中，其輸出往往被一個或少數幾個接近 1 的數值所主導，而其他輸出則極為接近 0，如圖 3.16 中的範例。這些接近 0 的輸出對最終結果的貢獻十分細微，幾乎可忽略不計。基於此特性，本研究進一步提出一種 Softmax 近似計算方法，稱為 Multimax。與傳統 Softmax 需要計算所有輸入值的指數比值不同，Multimax 僅針對輸入中的前 N 個最大值進行計算與比值分配，以此有效降低運算複雜度與硬體實現成本。

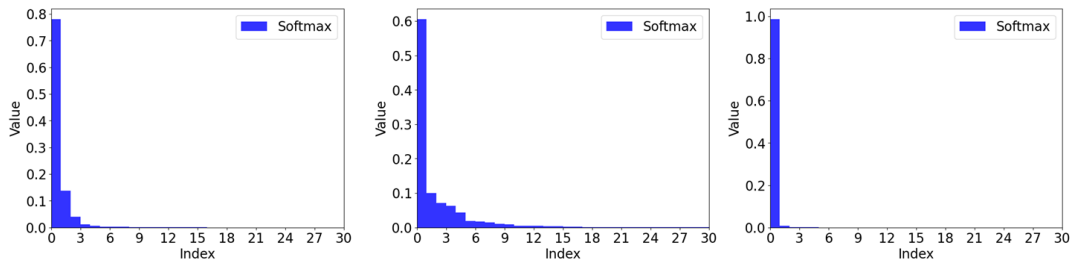


圖 3.16 Softmax 輸出範例

在實際硬體架構中，我們針對不同應用場景，提供了 $N=1$ 、3、5 等多種模式，以兼顧運算準確度與硬體效能。具體而言，Multimax 會先選出前 N 個最大值，並以其中最大者作為基準，逐次計算其與其他候選值之間的兩兩差值，進一步近似指數比值。為了保留總和為 1 的性質，我們採用遞迴式比值分配，將剩餘權重依序分配給後續值，最後一個值則分配所有剩餘比重，確保輸出符合

Softmax 的機率分佈特性。相較於直接計算前 N 個值的指數和，Multimax 的設計可避免執行完整的多項指數計算與累加，大幅降低硬體中實現指數與除法所需的邏輯複雜度，並可根據需求彈性切換 N 值，而不需額外增加計算資源，具有高度的可擴充性與硬體友善性。

(式 3.10)將以 $N=3$ 為例說明 Multimax 的運算方式。

$$\begin{aligned}
 &\text{Assume } x_{int}, y_{int}, z_{int} \\
 &x = S \times x_{int} + Z \\
 &y = S \times y_{int} + Z \\
 &z = S \times z_{int} + Z \\
 &\text{Assume } x \geq y \geq z \\
 &diff_{xy} = (S \cdot x_{int} + Z) - (S \cdot y_{int} + Z) = S \cdot (x_{int} - y_{int}) \\
 &ratio_x \cong \frac{e^x}{e^x + e^y} = \frac{e^{S \cdot (x_{int} - y_{int})}}{e^{S \cdot (x_{int} - y_{int})} + 1} = \frac{e^{S \cdot diff_{xy}}}{e^{S \cdot diff_{xy}} + 1} \\
 &ratio_y \cong \frac{e^y}{e^y + e^z} \times (1 - ratio_x) = \frac{e^{S \cdot diff_{yz}}}{e^{S \cdot diff_{yz}} + 1} \times (1 - ratio_x) \\
 &ratio_z \cong \frac{e^z}{e^y + e^z} \times (1 - ratio_x) = \frac{1}{e^{S \cdot diff_{yz}} + 1} \times (1 - ratio_x)
 \end{aligned}
 \tag{式 3.10}$$

本研究所提出的 Multimax 近似方法具備多項優勢：

1. 降低硬體成本：傳統的 Softmax 計算過程中，包含了指數運算與除法運算等複雜操作，對硬體資源需求極高。Multimax 則利用 LUT 實現 sigmoid 函數，輸入為兩個主要值之間的差值，透過 LUT 近似指數比值，免除了實作昂貴的指數與除法電路，顯著簡化硬體設計。



2. 簡化矩陣運算：由於 Multimax 輸出中僅有 N 個位置為非 0 數值，其餘皆為 0，因此後續進行矩陣乘法(將 Multimax 結果與 V 矩陣相乘)時，僅需針對這 N 個位置進行乘加運算。特別是在當 $N=1$ 的情況下，此操作簡化為從 V 矩陣中直接選取對應的 row，完全不需要乘法與量化運算，能大幅降低計算延遲與能耗。

圖 3.17 比較了傳統 Softmax 與 Multimax 的輸出分佈情形。由圖中可見，Softmax 輸出包含許多接近 0 的值，而 Multimax 則省略了這些對結果影響甚微的計算，同時仍能近似 Softmax 所產生的前 N 大值得數值分布，達到計算效率與推論準確度間的平衡。

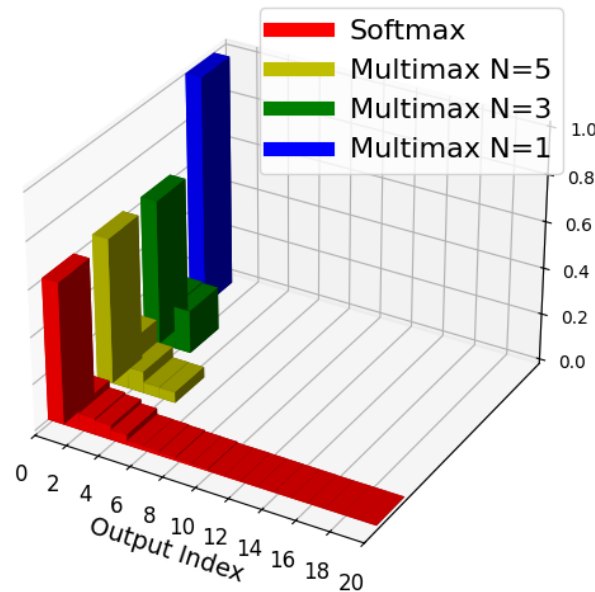


圖 3.17 Softmax 與 Multimax $N=1, 3, 5$ 結果比較

3.6.2 QGELU



3.6.2.1 GELU 硬體實作技巧比較

在 Transformer 模型架構中，激活函數 GELU 占了舉足輕重的地位，然而 GELU 複雜的運算過程(式 2.6)使得硬體的實作成本居高不下，部分研究[34]採用直接實作的方式設計 GELU 的硬體，可以最大程度維持模型的效能，卻需要付出龐大的硬體成本。其他研究則提出簡易的近似方式以降低硬體成本，例如[35]提出的 FGELU，以及[36, 37]的技巧，便是使用 piece-wise linear 的技巧近似 GELU，然而激進的近似方式雖能大幅降低硬體面積，卻使得模型出現明顯 accuracy loss。

3.6.2.2 構想與實現

為了在模型效能與硬體成本間取得平衡，本研究提出使用 piece-wise quadratic 的方式近似 GELU(簡稱為 QGELU)，透過將 GELU 非線性區間分割為多個區段，並使用二次函數近似每一個區間中的非線性曲線，可以盡可能貼近原始的 GELU 函數。為了降低硬體實作時，二次函數係數乘以輸入運算乘法器的硬體成本，我們將二次函數係數皆調整為以簡易 shift & add 運算便可完成的係數，在保有二次函數非線性優勢的同時盡可能降低硬體成本。由表 3.4 可以發現，由於 GELU 的非線性區間主要為 $[-3, 3]$ ，因此我們僅針對此部分設計分段式二次函數，而在小於-3 以及大於 3 的區間，則直接依序賦予 0 或輸入的數值。

表 3.4 QGELU 區間分割與二次函數係數

	Interval	Binomial
QGELU	$(-\infty, -3]$	$y = 0$
	$(-3, -2]$	$y = -\left(\frac{1}{32} + \frac{1}{256}\right)x^2 - \left(\frac{1}{4} - \frac{1}{32}\right)x - \frac{85}{256}$
	$(-2, -1.5]$	$y = -\left(\frac{1}{32} + \frac{1}{64}\right)x^2 - \left(\frac{1}{4} + \frac{1}{64}\right)x - \frac{100}{256}$
	$(-1.5, -1]$	$y = \left(\frac{1}{32} + \frac{1}{128}\right)x^2 - \left(\frac{1}{64} + \frac{1}{128}\right)x - \frac{57}{256}$
	$(-1, 0]$	$y = \left(\frac{1}{4} + \frac{1}{32}\right)x^2 + \left(\frac{1}{2} - \frac{1}{16}\right)x$
	$(0, 1]$	$y = \left(\frac{1}{4} + \frac{1}{32}\right)x^2 + \left(\frac{1}{2} + \frac{1}{16}\right)x - \frac{1}{256}$
	$(1, 2]$	$y = -\left(\frac{1}{256}\right)x^2 + \left(1 + \frac{1}{8}\right)x - \frac{72}{256}$
	$(2, 3]$	$y = -\left(\frac{1}{32} + \frac{1}{128}\right)x^2 + \left(1 + \frac{1}{4}\right)x - \frac{100}{256}$
	$(3, \infty)$	$y = x$

圖 3.18 為 GELU、FGELU、QGELU 的曲線繪圖，可以發現 FGELU 由於使用線性的近似方式，因此與 GELU 間有很大的誤差，而 QGELU 由於使用二次函數近似，因此可以達到肉眼幾乎無法分辨的近似結果。

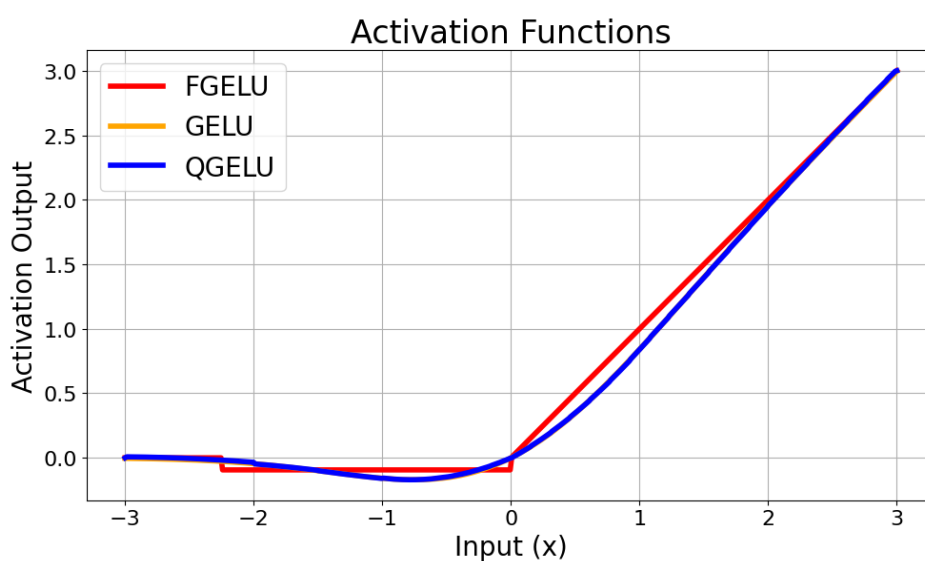


圖 3.18 GELU vs. FGELU vs. QGELU 繪圖



圖 3.19 進一步呈現了 QGELU 近似曲線的放大細節圖，儘管在微觀層面上，QGELU 與 GELU 之間仍存在些微誤差，但整體偏差幅度相當有限。

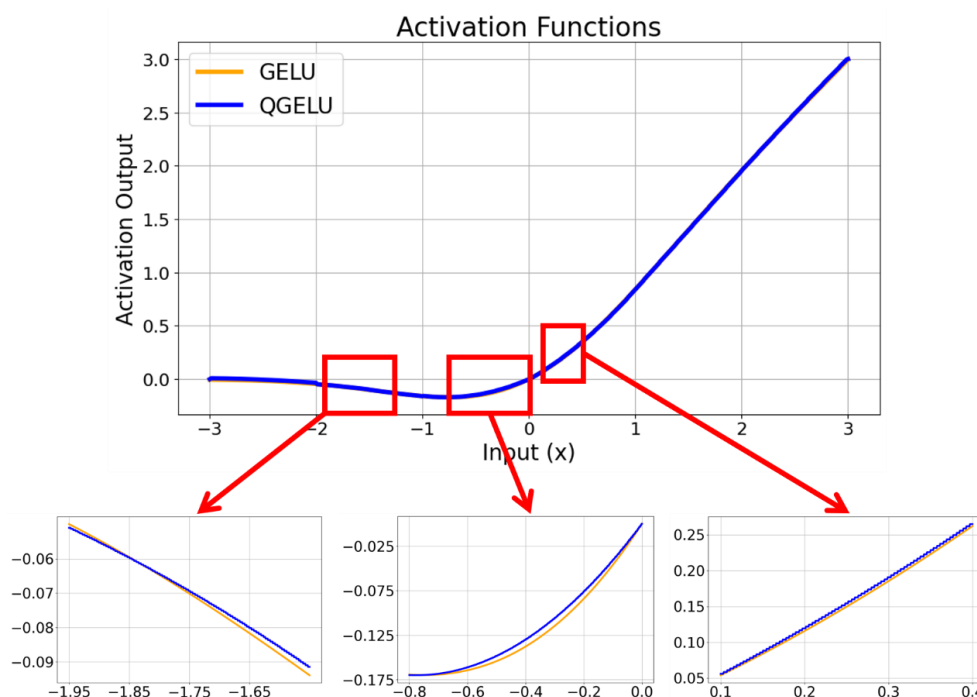


圖 3.19 QGELU 局部曲線放大圖

圖 3.20 為擷取 ViT 模型中 GELU 的輸出分布，可以發現實際模型中 GELU 的輸入主要分布於-4~4 的區間。為了定量評估 QGELU 的近似精度，我們根據訓練模型中 GELU 的實際輸入分佈，隨機生成了 100,000 筆測試樣本，並計算其輸出與 GELU 函數之間的 MSE，如表 3.5 所示。此結果顯示，QGELU 與 GELU 的 MSE 僅有 1.87×10^{-5} ，在所有區間中均能有效近似原始 GELU 函數，具備高度的精確性。

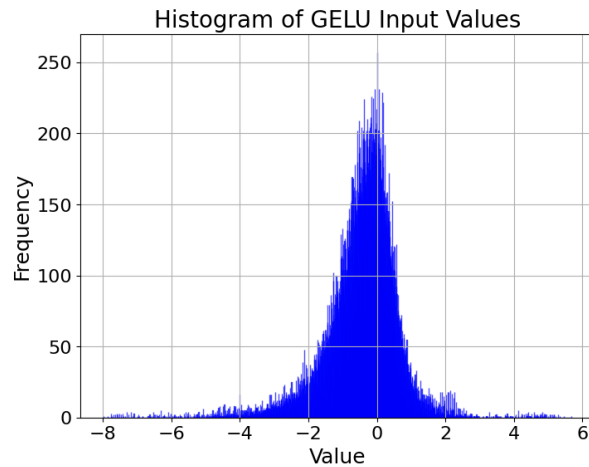


圖 3.20 ViT 模型 GELU 輸入分布

表 3.5 GELU vs. QGELU vs. FGELU 誤差分析

Function	MSE
GELU	1.34×10^{-8}
QGELU	1.87×10^{-5}
FGELU	6.40×10^{-3}

模型訓練部分，我們於 forward 時使用 QGELU 進行近似，而進行 backward 時，為避免不同區間之間不連續的情形，則以原始的 GELU 計算梯度以避免數值異常，並在訓練程式中使用 `.detach` 的方式消除 QGELU 的梯度。模型訓練時，為了模擬硬體運算時的精度，我們在 QGELU 的輸入以及輸出皆會進行數值 truncate 以貼合硬體加速運算的結果。

3.7 加法近似運算

由於本研究採用 Multimax 作為 Softmax 函數的近似方法，相較於原始模型，其部分運算對精度的要求得以放寬。且由晶片整體面積與功耗分析可以發現，

加法電路在所有運算模組中貢獻了最主要的面積與功耗佔比。因此，為了進一步降低整體晶片的功耗，本研究將優化重點放在近似加法器電路的設計上。

在介紹本論文所提出的加法器架構之前，本節將先回顧與介紹現有文獻中所提出的各種加法器設計方法。接下來的段落將針對這些架構進行誤差、面積與功耗等方面的比較分析，並說明本研究所提出之近似加法器架構的設計理念與優勢。

3.7.1 加法近似技巧文獻回顧

在神經網路硬體加速器的設計中，加法器通常佔據大量邏輯閘面積與功耗，特別是在大幅優化乘法運算的低精度運算單元中更為顯著。為了降低加法運算的硬體複雜度與能耗，近年來已有許多文獻提出各種加法近似技巧，透過犧牲部分計算精度以換取電路面積、功耗或運算速度的提升。

論文[38] 提出以 NAND gate + NOR gate 組合實現近似加法器的設計方法，並發展出單層與雙層的近似架構。此方法特別適用於對誤差容忍度要求較低的應用場景，能有效提供低面積、低功耗的硬體實現方案。圖 3.21 顯示了該研究中所提出之單層與雙層近似加法器的電路架構。

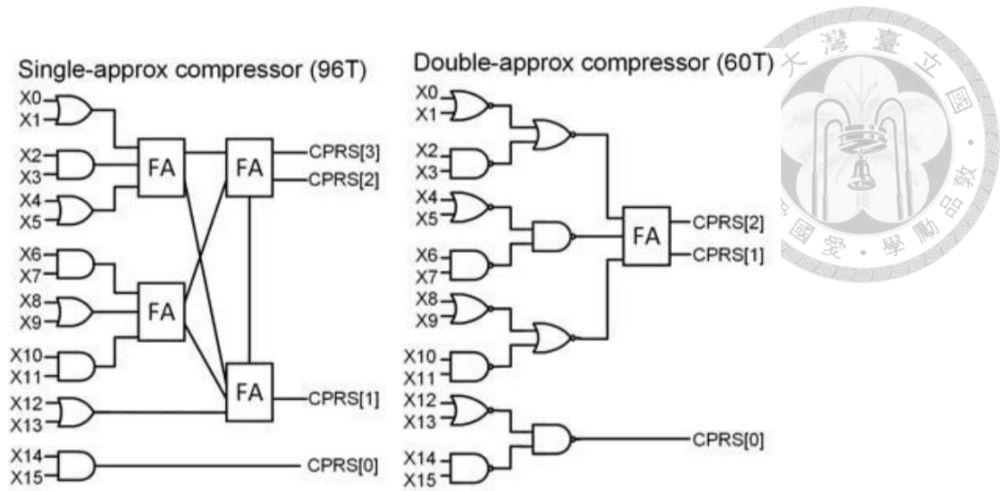


圖 3.21 NAND gate + NOR gate 單層近似加法器與雙層近似加法器架構[38]

論文[39]提出以 OR gate 為主的近似加法器，並以單層近似的方式進行實作。

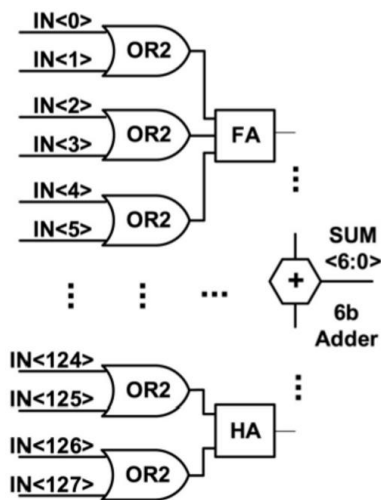


圖 3.22 OR gate 近似加法器架構[39]

論文[40]進一步針對論文[39]提出的方法進行改進，指出由於 OR gate 在兩個輸入皆為 1 時，會造成較大的運算誤差，當輸入訊號分布不均時，此誤差現象會更加明顯。具體而言，作者在 OR gate 層之前加入用於輸入交換(input swapping)的 MUX，透過在有限範圍內重新分配 logic-1 輸入，將其平均分散至

各個邏輯閘，藉此有效降低總體誤差，提升加法器在實際應用上的適用性與可靠度。

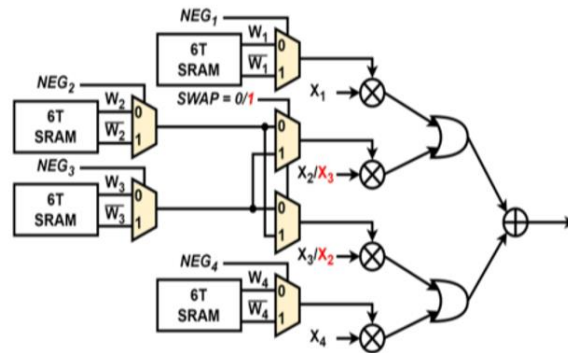


圖 3.23 OR gate + Input Swapping 近似加法器架構[40]

最後，論文[35]，即由本實驗室陳俊瑋學長所提出的方法，採用了 OAI21 3-input logic gate 來取代 LSB 運算單元中第一層加法器的運算，藉此大幅降低電路面積與功耗。該研究亦透過 QAT 進行模型訓練與驗證，證明此方法不會造成過大的 accuracy loss。此外，論文[35]中亦提出 dual-mode 控制機制，在誤差容忍度較高的運算部分採用低精度近似計算，而在需要較高準確度的部分則維持高精度運算，進一步提升整體運算的近似彈性與效能表現。

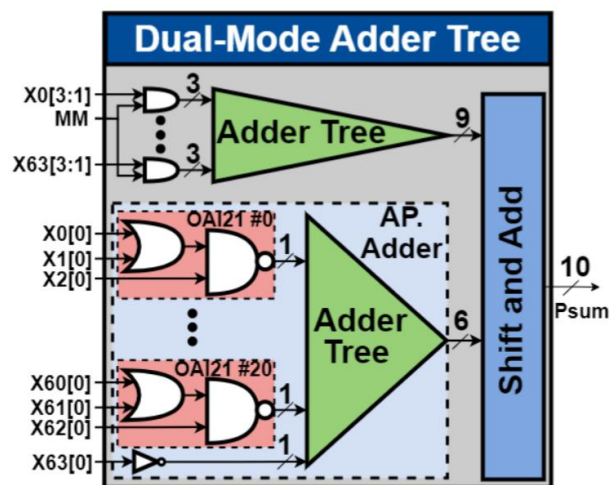
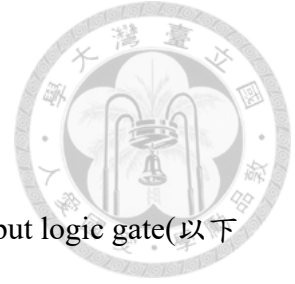


圖 3.24 OAI21 Dual-Mode Adder 近似加法器架構[35]



3.7.2 OAI21+AAI21 雙層近似加法器

此段落介紹本研究提出之近似加法器，我們由 OAI21 3-input logic gate(以下簡稱 OAI21)進行延伸，每個 OAI21 由一個 OR gate、AND gate 與 inverter(可合併為 NAND gate)構成，如圖 3.25 的架構。

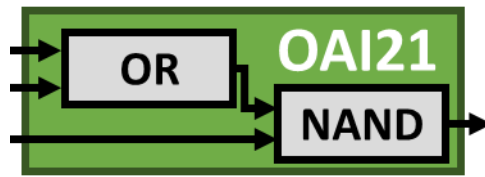
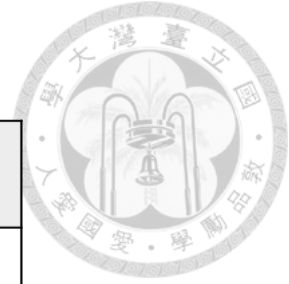


圖 3.25 OAI21 邏輯閘架構

接下來我們分析以 OAI21 替代 full adder 的可行性，表 3.6 中為 OAI21 的真值表，與 OAI21 的輸出結果以及經過 full adder 的結果比較。所有傳入 OAI21 的輸入都經過反轉，且由於 OAI21 將三個輸入壓縮為 1-bit 輸出，因此我們將輸出乘以 2 以降低近似誤差。透過觀察真值表可以發現 OAI21 的三個輸入間並不對等，相同輸入下若將輸入位置交換可能獲得相異的結果。

表 3.6 OAI21 真值表與近似加法結果比較

Inputs	Sum	OAI21 (Inputs Inverted)
(0, 0, 0)	0	0
(0, 0, 1)	1	2
(0, 1, 0)	1	0
(0, 1, 1)	2	2
(1, 0, 0)	1	0
(1, 0, 1)	2	2
(1, 1, 0)	2	2
(1, 1, 1)	3	2
Mean Error		-0.25



由表 3.6 可以發現 OAI21 的近似結果的期望值為負，因此本研究使用 AAI21 的 3-input logic gate 來做為相對應的補償，AAI21 由兩個 AND gate 與 inverter 組成，可視為一個 3-input NAND gate，不過為了與 OAI21 相呼應，此篇論文會以 AAI21 稱呼之。由於 AAI21 的近似期望值為正，因此若將 OAI21 與 AAI21 交替擺放，可以使整體近似結果之期望值為 0。

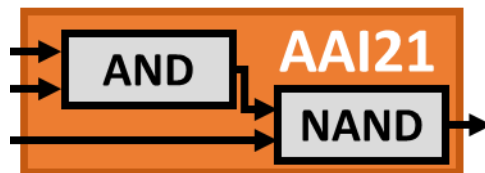
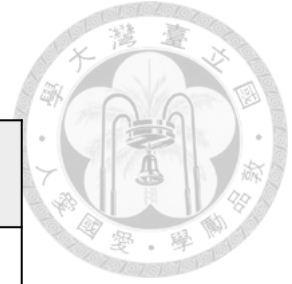


圖 3.26 AAI21 邏輯閘架構

表 3.7 AAI21 真值表與近似加法結果比較

Inputs	Sum	AAI21 (Inputs Inverted)
(0, 0, 0)	0	0
(0, 0, 1)	1	2
(0, 1, 0)	1	2
(0, 1, 1)	2	2
(1, 0, 0)	1	2
(1, 0, 1)	2	2
(1, 1, 0)	2	2
(1, 1, 1)	3	2
Mean Error		+0.25



此研究提出的近似加法器架構使用 OAI21 與 AAI21 混和排列的方式構築，並使用雙層近似以盡可能降低近似加法器的面積與功耗，不過由於加法器的輸入是 64 個 1-bit 輸入，而每個 OAI21 或 AAI21 會將 3 個輸入壓縮為 1 個輸出，因此會剩餘一個輸入，此輸入會留待最終近似加法完成後使用一般的加法器進行加總。且由於我們使用雙層的近似架構，因此第二層的每個輸出會由 9 個輸入壓縮而成，因此我們賦予其 4 倍的權重，以降低輸入壓縮造成的誤差。

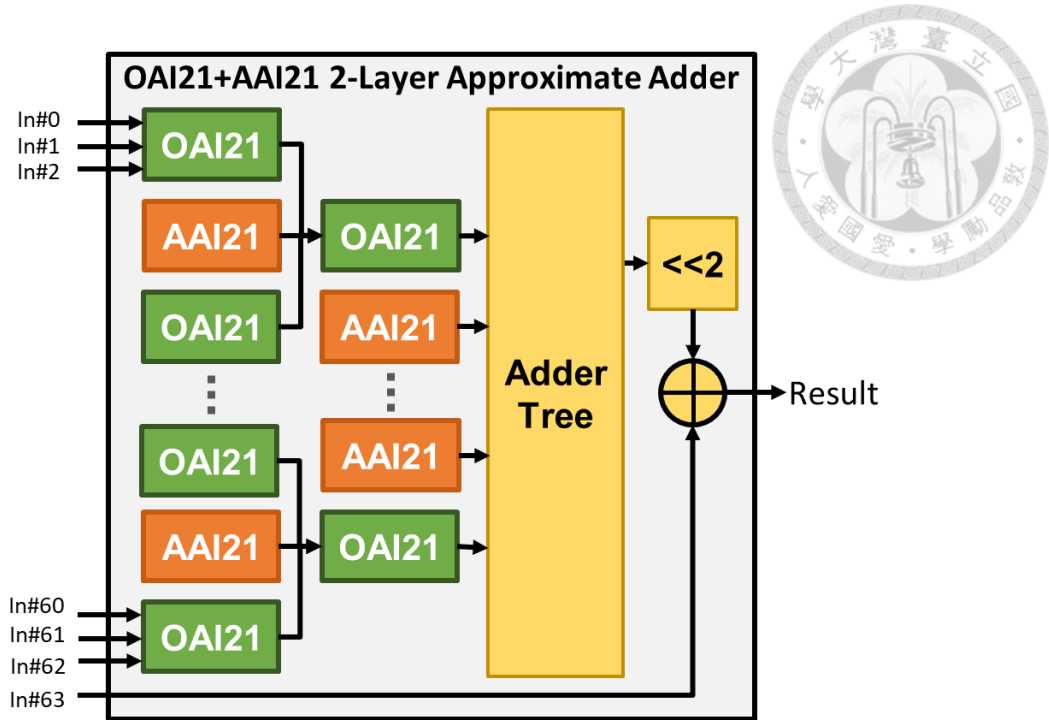


圖 3.27 OAI21 + AAI21 雙層近似加法器架構

3.7.3 近似加法硬體實作結果分析

此段落進行各架構近似加法器之比較與分析，主要針對各近似加法器架構在相同輸入下的誤差，以及實作的面積、功耗方面進行比較。比較對象包括 Vannila 加法器，以及段落 3.7.1 介紹的 NAND gate + NOR gate 雙層近似加法器、OR gate 近似加法器、OR gate + Input Swapping 近似加法器、OAI21 近似加法器等，所有比較對象皆為 64 個 1-bit 輸入進行加總以進行公平比較。

所有架構皆使用 40 奈米製程進行合成，頻率為 200MHz。輸入則是自訓練完成的 TW4A ViT 模型中提取 Ternary 權重與 INT-4 輸入的分布，並依照其實際分布隨機生成符合其分佈比例的測試資料，並使用 Weight * Activation 後的 LSB 部分當作輸入。圖 3.28 是從訓練完成的 TW4A Swin-tiny 中 Linear Layer 提取的權重與輸入分布圖，可以發現 Ternary Weight 的特性會使權重為 0 的比例自然提高，因此相乘後 LSB 為 0 的比例也會相應提升，這使得圖 3.29 中實際加總結果

的分布會往 0 的方向偏移，峰值約落在 15~20 的區間。這與使用隨機生成 64 個 0 或 1，加總的最大峰值會落在約 32 處有所不同。這個做法可以讓近似誤差的比較更貼近 TW4A 加速電路的實際情境，篩選出最適合本研究的近似架構。

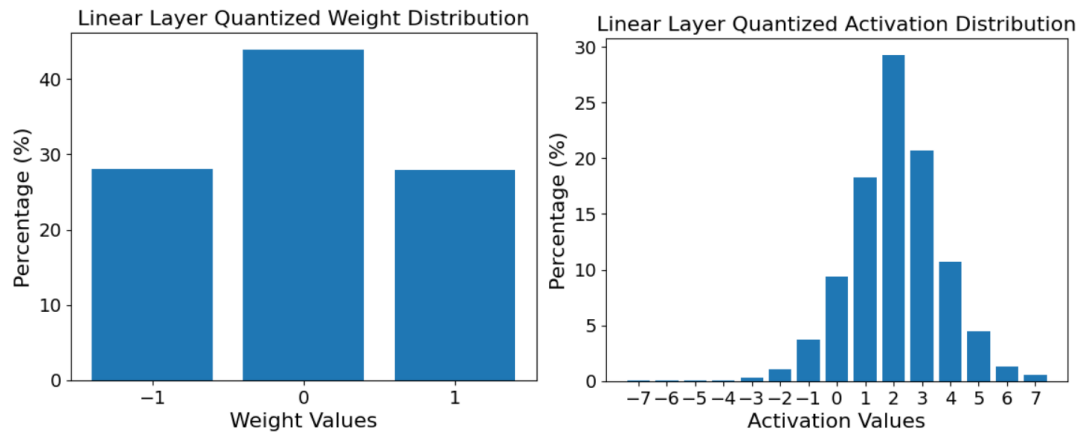


圖 3.28 TW4A Swin-tiny Linear Layer 之權重與輸入分布

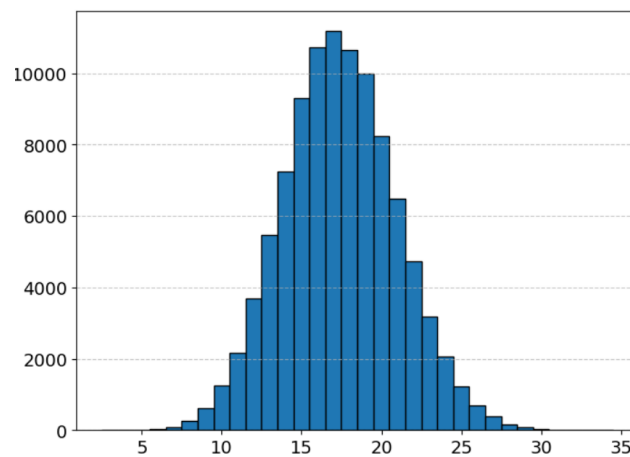


圖 3.29 近似加法測試資料實際加總結果分布

表 3.8 近似加法器面積與功耗比較

Approximate Adder	Area	Power (*10e-5)
Vanilla (baseline)	3711	22.06
NAND + NOR Gate	822	3.727
OR Gate	1760	9.075
OR Gate + Input Swap	2371	15.04
OAI21	1046	4.807
OAI21 + AAI21 2-Layer	594	2.608

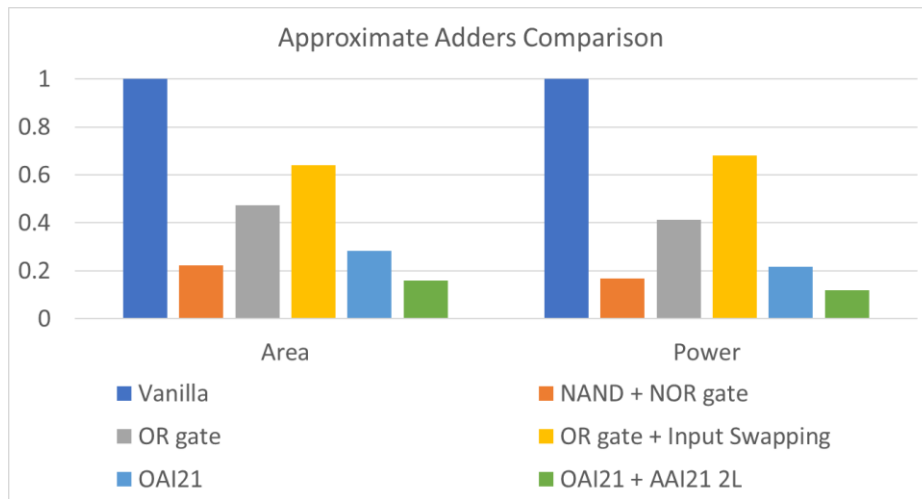


圖 3.30 近似加法器面積與功耗比較

圖 3.31 為近似架構實際模擬的近似結果分布圖，圖中縱軸為近似加總結果，橫軸為實際加總結果，紅色斜直線對應 Vanilla 加法器的輸出，理想的近似加法器之輸出分布應盡可能貼近該直線。由於比較表中部分架構(例如 OR gate + Input Swapping)合成後面積較大，不符合我們對於近似加法器的預期，因此下方的誤差分布圖僅列出與本研究提出的 OAI21 + AAI21 雙層近似加法器之面積與功耗表現較接近的 NAND gate + NOR gate 雙層近似加法器的近似結果以進行比較。可以發現本研究的技巧由於皆使用 3-input gate，因此與論文[38]的方式相比能達到更高的壓縮比，使得近似加法器的面積大幅降低，同時也在誤差方面勝過[38]提出之近似加法器。

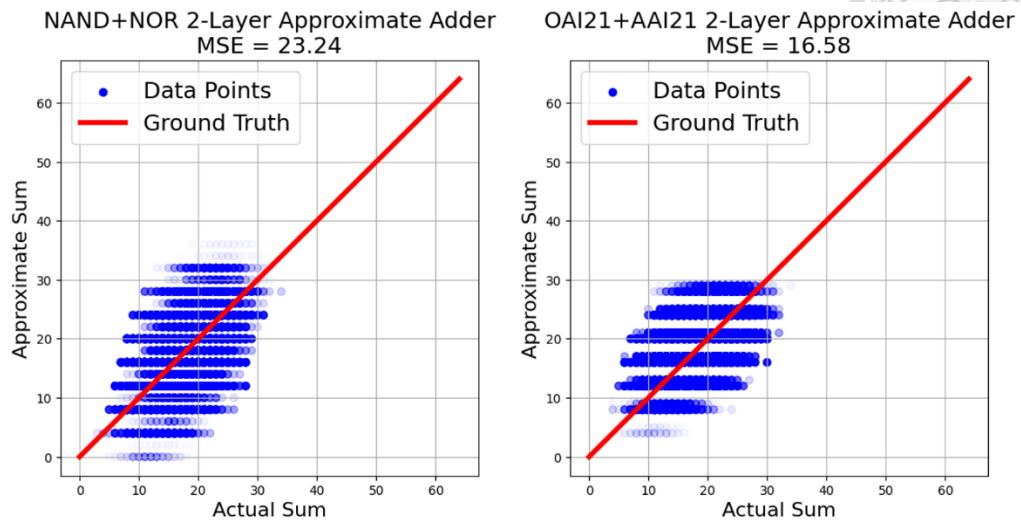


圖 3.31 近似加法器誤差分布圖

圖 3.32 以圖像化的方式比較 OAI21 + AAI21 雙層近似加法器與 Vanilla 加法器的面積與功耗，分別有著 86.2%與 90.8%的下降。

表 3.9 近似加法器 MSE 比較

Approximate Adder	MSE
Vanilla (baseline)	0
NAND + NOR Gate	23.2
OAI + AAI 2-Layer	16.6

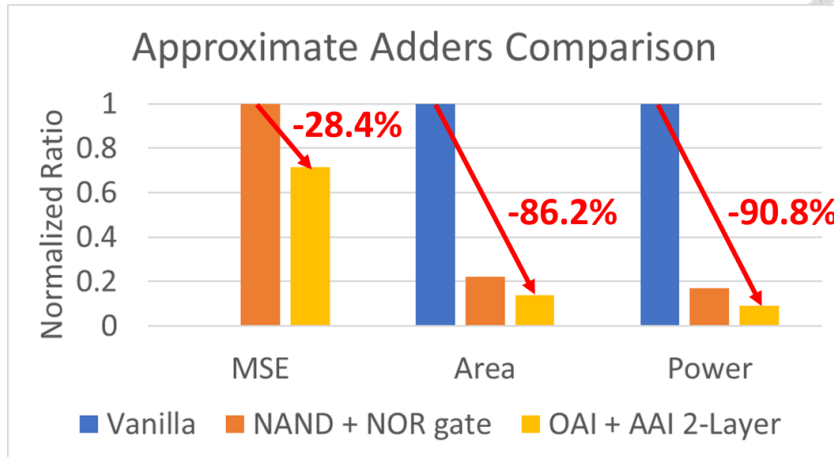


圖 3.32 近似加法器 MSE、Area、Power 比較繪圖

3.8 模型訓練結果彙整

本章節將針對本研究所提出之量化與函數近似方法在不同應用領域中的實驗結果進行彙整與分析。具體涵蓋的應用情境包含影像辨識、文字生成、以及語音轉文字三大類任務，分別對應於常見的 Transformer 模型應用場域，驗證本研究提出方法的通用性與穩定性。

此外，我們也針對所設計之近似加法器架構進行模擬訓練與效能評估，探討不同加法近似模式(如 LSB 與末兩位近似)對模型準確率與推論能耗之影響。實驗中，我們分別以精確加法與不同程度的近似加法進行比較，並分析各近似策略在不同應用場景中的適用性與限制。

本章節的分析將作為後續硬體實現與整體系統設計的重要依據，並驗證所提出方法於實際應用中具備良好的可行性與效能表現。

3.8.1 影像辨識任務訓練結果

在影像辨識任務中，我們使用原始 ViT 架構、DaViT、Swin 三個 ViT 系列模型進行量化近似技巧的驗證。表 3.10 中整理各階段之訓練結果，並比較與全精度模型之差異，以方便觀察不同技巧對於模型 accuracy 的影響。可以觀察到雖然 TW4A 近似會對 accuracy 造成一定程度的下降，但透過 QAT 後，皆能把模型效能的損失控制於合理範圍內。此外，我們所提出之 Multimax 與 QGELU 函數近似法，即便在大幅簡化運算邏輯與降低硬體實作成本的前提下，仍能在三種模型上維持穩定的分類效能，顯示其良好的近似能力與模型兼容性。

表 3.10 影像辨識任務訓練結果

Dataset	Model	Precision	Softmax	GELU	Acc(%)	Acc Loss
ImageNet 100	ViT-base	FP32	Softmax	GELU	87.20	--
		INT8	Softmax	GELU	87.38	+0.16%
		TW4A	Softmax	GELU	86.00	-1.20%
		TW4A	Multimax N=3	GELU	85.66	-1.54%
		TW4A	Multimax N=3	QGELU	85.60	-1.60%
	DaViT-tiny	FP32	Softmax	GELU	85.10	--
		INT8	Softmax	GELU	85.28	+0.18%
		TW4A	Softmax	GELU	84.50	-0.60%
		TW4A	Multimax N=3	GELU	84.68	-0.42%
		TW4A	Multimax N=3	QGELU	84.44	-0.66%
	Swin-tiny	FP32	Softmax	GELU	85.68	--
		INT8	Softmax	GELU	85.92	+0.24%
		TW4A	Softmax	GELU	85.90	+0.22%
		TW4A	Multimax N=3	GELU	86.00	+0.32%
		TW4A	Multimax N=3	QGELU	85.60	-0.08%

我們也將 TW4A + Multimax + QGELU 的訓練結果與[35]使用的 1W4A(Binary Weights + INT-4 Activation)精度結果相比較，以評估不同量化策略在實際任務中的效能表現差異。由表 3.11 的結果可以觀察到，Ternary 權重的量化方式即使加上 Multimax 以及 QGELU 的函數近似技巧，模型的表現與比單純 1W4A 不加上函數近似的結果表現接近或更加優異。

表 3.11 TW4A vs. 1W4A 影像辨識任務訓練結果比較

Dataset	Model	Precision	Softmax	GELU	Acc(%)	Acc Loss
ImageNet 100	ViT-base	FP32	Softmax	GELU	87.20	--
		TW4A	Multimax N=3	QGELU	85.60	-1.60%
		1W4A	Softmax	GELU	85.74	-1.46%
	DaViT-tiny	FP32	Softmax	GELU	85.10	--
		TW4A	Multimax N=3	QGELU	84.44	-0.66%
		1W4A	Softmax	GELU	83.84	-1.26%
	Swin-tiny	FP32	Softmax	GELU	85.68	--
		TW4A	Multimax N=3	QGELU	85.60	-0.08%
		1W4A	Softmax	GELU	83.50	-2.18%

3.8.2 文字生成任務訓練結果

文字生成任務中，我們使用 GPT-2 作為基礎模型進行訓練與評估，與影像辨識的結果分析相同，我們列出各階段之訓練結果，並與 1W4A 結果比較。值得注意的是，文字生成任務的注意力矩陣維度通常相比 ViT 而言較大，這使得 Softmax 函數的近似誤差更容易累積，進而影響生成結果的語意一致性與流暢性。為了降低 Softmax 近似誤差，此處採用 Multimax N=5 進行近似，相較於 Multimax N=3 可提供更細緻的分段精度，兼顧數值穩定性與硬體實作效率。與 1W4A 相比，TW4A + Multimax + QGELU 的結果在降低 performance loss 上具有明顯優勢。

表 3.12 文字生成任務訓練結果

Dataset Model	Precision	Softmax	GELU	Perplexity↓	Perf. Loss
Wikitext-2 GPT-2	FP32	Softmax	GELU	11.10	--
	TW4A	Softmax	GELU	14.85	+3.75
	TW4A	Multimax N=5	GELU	16.06	+4.96
	TW4A	Multimax N=5	QGELU	16.09	+4.99
	1W4A	Softmax	GELU	18.80	+7.70



3.8.3 語音轉文字任務訓練結果

語音轉文字任務則是使用 Whisper 模型進行訓練，與 GPT-2 相同也使用 Multimax N=5 的策略進行近似，由表 3.13 的結果可以觀察到本研究使用的作法相比 1W4A 的 performance loss 低了不少，證明 TW4A 精度與兩種函數近似技巧維持模型效能的有效性。

表 3.13 語音轉文字任務訓練結果

Dataset Model	Precision	Softmax	GELU	WER(%) ↓	Perf. Loss
LibriSpeech Whisper	FP32	Softmax	GELU	4.30	--
	TW4A	Softmax	GELU	7.66	+3.36%
	TW4A	Multimax N=5	GELU	7.98	+3.68%
	TW4A	Multimax N=5	QGELU	8.04	+3.74%
	1W4A	Softmax	GELU	8.80	+4.50%

3.8.4 稀疏性分析

針對三個主要模型 ViT-base、GPT-2 與 Whisper，我們分析訓練完成之 TW4A 模型的權重稀疏性，各模型的稀疏程度如表 3.14 所示。由此結果可以佐證 ternary 權重會對量化模型產生自然稀疏性的特性。

表 3.14 各模型稀疏性分析

TW4A MODEL	Overall Sparsity
ViT	44.59%
GPT-2	50.56%
Whisper	46.89%

3.8.5 加法近似訓練結果

由於採用加法近似不可避免會造成 accuracy loss，且在模型中的關鍵運算部分影響尤為顯著，因此需要針對近似加法造成的影響進行 QAT。在軟體訓練階段若要模擬加法近似，需對矩陣乘法進行元素級別的運算修改，因此本段落使用 Swin-tiny 模型做為訓練對象，訓練過程與量化訓練無異，僅在加法運算單元中替換為近似加法器之模擬，且皆使用 ImageNet100 資料集進行訓練。另外也使用 Whisper 模型進行加法近似的訓練。本段落之近似訓練接續 3.8.1 章節的訓練結果，由 TW4A + Multimax + QGELU 的量化近似結果當作 baseline，並在此之上進行加法近似訓練。

在 Swin-tiny 的訓練實驗中發現，若直接對整個模型進行加法近似，會產生嚴重的 accuracy loss，因此我們採用逐層近似的方式，針對 Transformer 架構中不同的運算，包括注意力機制中的 Q、K、V 矩陣的矩陣乘法、 QK^T 矩陣乘法，以及最末尾的 Projection Layer，以及 MLP 中的第一層與第二層 Linear Layer。然而，經過實驗亦發現，若對 MLP 部分進行加法近似，不論僅近似 LSB 或使用 2-bit 加法近似，都會造成超過 1% 的準確率損失，如表 3.15，對我們的應用目標而言，屬於無法接受的結果。因此，在後續的實驗中，我們僅對注意力機制中的運算採用加法近似，而 MLP 部分則維持高精度運算，以兼顧運算效能與模型準確率，如表 3.16。

此外，attention 機制中不同運算對於加法近似的容忍程度也有所不同，例如計算 Q 矩陣與 K 矩陣的 Linear Layer，以及 QK 矩陣相乘的運算皆能容忍較大的近似誤差而不會造成明顯的 accuracy loss。然而 MHA 中最末一層 Linear Layer 以及與 V 矩陣相乘的運算卻對加法近似相當敏感。此現象推測原因為前二者的運算發生於 Softmax 運算前，由於本研究已將 Softmax 近似為 Multimax，因此對於結果數值的精度需求下降，使得加法近似造成的誤差不會對最終結果造成影

響。也因此，我們對於這兩類運算使用不同程度的加法近似策略，在誤差容忍度較高的運算中，採用末 2-bit 近似。反之則僅採用 LSB 近似以保持模型的效能。

為了進一步彌補近似造成的誤差，我們參考[41]的做法於訓練時加入了誤差補償項。作法為計算近似加法後的結果與原始結果誤差，並滾動式計算平均誤差值已獲得更好的補償效果。由於 inference 時的 batch size 與 sequence length 與訓練時未必相同，因此我們對 batch 與 sequence 維度進行平均，最終得到的補償項為對每個 feature channel 的偏差修正值，並在推論時套用於所有位置以減少近似誤差造成的精度損失。

(式 3.11)以 ViT 模型進行誤差補償項的公式說明。

$$\begin{aligned}
 result_{golden} &\in \mathbb{R}^{B \times S \times D} \\
 result_{approximate} &\in \mathbb{R}^{B \times S \times D} \\
 mean_error &= result_{golden} - result_{approximate} \\
 avg_error &\in \mathbb{R}^D \\
 avg_error &= \frac{1}{B \cdot S} \sum_{b=1}^B \sum_{s=1}^S mean_error_{b,s} \\
 MA_error_i &= 0.8 \times MA_error_{i-1} + 0.2 \times avg_error \\
 &\text{broadcast } MA_error \text{ to shape } [B, S, D] \\
 result_{compensated} &= result_{approximate} + MA_error
 \end{aligned}
 \tag{式 3.11}$$

表 3.15 Swin-tiny 加法近似訓練結果(baseline=TW4A + Multimax + QGELU)

Dataset Model	Window Attention				MLP		Acc.(%)	Acc. Diff.
	QK LL.*	QK matmul	Concat LL.	V LL.	LL.#0	LL.#1		
ImageNet 100 Swin-tiny							85.60	--
							85.46	-0.14%
							85.14	-0.46%
							85.00	-0.60%
							84.88	-0.72%
							84.24	-1.36%

□ : No approximation

* : LL. Stands for Linear Layer

■ : Approximated matrix multiplication with 2-bit OAI21 + AAI21 2-layer adder w/ error compensation

■ : Approximated matrix multiplication with LSB OAI21 + AAI21 2-layer adder w/ error compensation

表 3.16 Swin-tiny 加法近似訓練設定(baseline=TW4A + Multimax + QGELU)

Dataset Model	Window Attention				MLP		Acc.(%)	Acc. Diff.
	QK LL.*	QK matmul	Concat LL.	V LL.	LL.#0	LL.#1		
ImageNet 100 Swin-tiny							85.60	--
							85.46	-0.14%
							85.14	-0.46%
							85.00	-0.60%
							84.88	-0.72%

□ : No approximation

* : LL. Stands for Linear Layer

■ : Approximated matrix multiplication with 2-bit OAI21 + AAI21 2-layer adder w/ error compensation

■ : Approximated matrix multiplication with LSB OAI21 + AAI21 2-layer adder w/ error compensation

在 Whisper 的訓練結果表 3.17 中，同樣可以發現模型對於 attention 部分運算的容忍程度較高，到了 MLP 的近似則會造成>1%的 WER loss。與 Swin-tiny 的訓練結果相似。其中差異在於，我們發現若對 Whisper 施加 2-bit 的近似，會造成較明顯的 performance loss，因此最後決定全部採用 LSB 近似。這現象或許是因為相比於影像辨識有明確的分類類別，語音轉換只要在輸出數值部分產生些許誤差便會對應到不同的 token，進而使 WER 上升，使得 Whisper 對於加法近似的敏感度相較 Swin-tiny 來的高。

表 3.17 Whisper 加法近似訓練結果(baseline=TW4A + Multimax + QGELU)

Dataset Model	Attention				MLP		WER(%)	WER Diff.
	Q LL.*	K LL.	Concat LL.	V LL.	LL.#0	LL.#1		
LibreSpeech Whisper							8.04	--
							8.36	-0.32%
							8.79	-0.75%
							8.69	-0.65%
							8.88	-0.84%
							9.34	-1.30%
							9.92	-1.88%

□ : No approximation

■ : Approximated matrix multiplication with 2-bit OAI21 + AAI21 2-layer adder w/ error compensation

■ : Approximated matrix multiplication with LSB OAI21 + AAI21 2-layer adder w/ error compensation

* : LL. Stands for Linear Layer

3.8.6 模型輸出結果比較

本段落展示各應用之模型訓練結果的比較。首先是 ViTs 的訓練結果，圖 3.33 以 ViT-Base 為範例，展示 fp32 模型與完成量化近似結果的 ViT-Base 模型的分類結果差異，圖中紅框的影像為模型分類錯誤之影像。

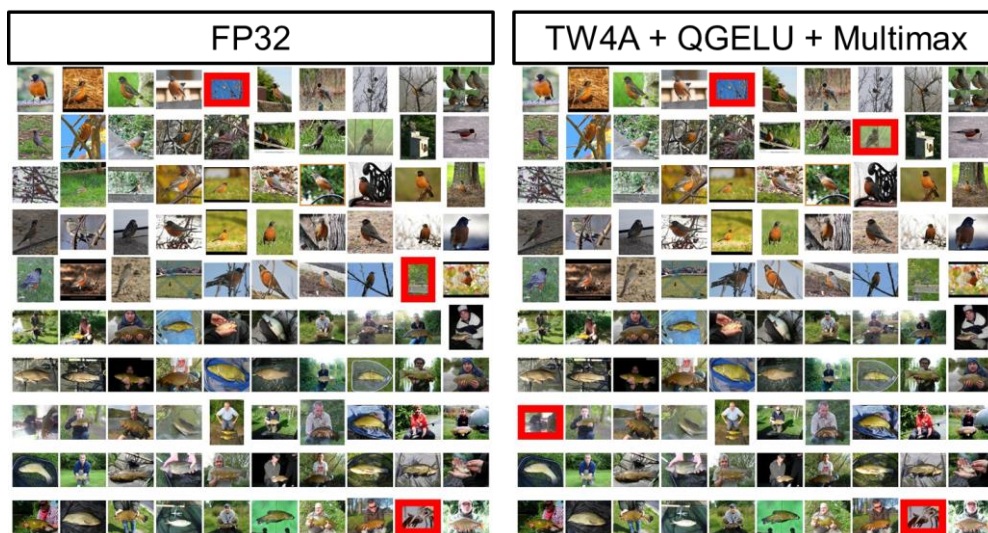


圖 3.33 ViT-Base 模型輸出比較

表 3.18 展示 GPT-2 的文字生成任務訓練結果。

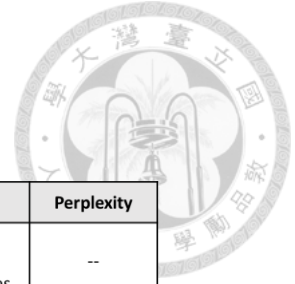


表 3.18 GPT-2 模型輸出比較

ID	MODEL	Result	Perplexity
1	Golden	In the 2010 Census, the racial makeup of the city was 61.55 % African American, 35.71 % White, 0.9 % Asian, 0.2 % Native American, < 0.02 % Pacific Islander, 0.59 % from other races, and 0.89 % from two or more races.	--
	FP32	the 1970 Census, the population makeup of the city was 94.5 % White American, 0.45 % Native, 0.39 % Native, 0.8 % Pacific American, 0 0.1 % Pacific Islander, and .06 % from other races, and 1.06 % from two or more races.	4.89
	TW4A + QGELU + Multimax	by the 1970 census, the city makeup of the city was 49.2 % White American, and .5 % African, and .0 % Asian, and .9 % Asian American, 0 0.0 % Asian Islander, 0.0 % from other races, 0 0.0 % from other or more races.	6.87
2	Golden	Eight of these guns were positioned on the main deck and the other six guns were placed above them in the superstructure.	--
	FP32	The were these were were mounted in the main deck and werethe other four were were placed in . the forwardstructure.	5.54
	TW4A + QGELU + Multimax	The of were were mounted in the main deck of werethe two two were were mounted on . the casstructure.	8.24

表 3.19 展示 Whisper 的語音轉文字任務訓練結果，表中紅自為辨識錯誤之字元。

表 3.19 Whisper 模型輸出比較

ID	MODEL	Result
1	Golden	i tolled the bell again
	FP32	i told the bell again
	TW4A + QGELU + Multimax N=5	i told the bell again
2	Golden	no doubt many predictions have been colored to suit the new age and unquestionably false prophets fakirs and conjurers have become the pest of the tribes during the transition period
	FP32	no doubt many predictions have been coloured to suit a new age and unquestionably false profits fakers and conjurers have become the pest of the tribes during the transition period
	TW4A + QGELU + Multimax N=5	no doubt many predictions had been coloured to suit the new age and unquestionably false prophets facres and congress had become the pest of the tribes during the transition period

3.9 第三章總結

本章節首先介紹了三個用於驗證本研究低精度量化與非線性函數近似技巧的主流 transformer 任務，包括影像辨識、文字生成與語音轉文字，以及對應使用的模型架構與訓練資料集解析。接著介紹本研究使用的量化精度，TW4A，

並分別介紹 Ternary Weight 以及 INT-4 Activation 的實作及訓練方式。針對 Transformer 模型中的非線性函數，Softmax 與 GELU，於此章節中介紹本研究提出的函數近似技巧，Multimax 與 QGELU，並詳細說明其設計理念與實作方式。

除了參數量化與函數近似外，本章節亦針對加法單元之近似運算進行設計與實驗驗證。除了介紹現有研究提出的近似加法器，我們也提出創新的 OAI21 與 AAI21 混合式雙層近似加法器，在誤差、面積、功耗分析上皆證明擁有超越現有近似加法器的各項指標，進一步強化推論階段的硬體能效。

最後，本章節彙整各應用場景之訓練結果，證明本研究提出之 TW4A + Multimax + QGELU 量化近似策略，即使在低精度與近似運算的條件下，經過適當的訓練策略後，模型仍能保有良好的效能，並在各任務與模型皆擁有超越 1W4A 的效能。顯示本研究提出之技術具備跨任務、跨架構的適應能力與實務部署潛力。上述成果為下一章晶片加速器的設計提供了明確的依據與參數基礎。

第四章 晶片架構設計



前段針對 Transformer 架構、量化精度設定、非線性函數近似技巧，以及對應的軟體模擬與訓練結果進行了完整介紹，驗證了本研究提出的方法在多種應用情境下的效能表現與可行性。本章節將進一步說明這些技術在硬體層面的具體實作方式，並介紹整體加速晶片的架構設計與資料運算流程。

首先，我們將深入介紹各個功能模組的硬體實作，包括負責主體 MAC 運算的 PE 電路、負責其他非乘加運算的 PPU 單元，以及專為 Softmax 函數近似設計的 Multimax 運算模組。這些模組共同構成 Transformer 模型中主要運算流程的硬體對應，透過設計實現高效能與低功耗的目標。最後，我們將針對完整的推論運算流程進行說明。

4.1 晶片架構總覽

4.1.1 運算電路概覽

在 Transformer 的運算架構中，不論是 Linear Layer 抑或是 QK^T 矩陣乘法，其核心皆由乘加運算構成的矩陣運算，因此在加速電路中便需要大量負責乘加的硬體以支援大維度的矩陣運算。在本論文的晶片架構中，我們將以 Processing Element(PE)稱呼此類電路。

而 transformer 架構中仍有許多除矩陣運算外的加速需求，包括先前介紹的 activation 函數、Softmax、將 partial sum 加總的累加運算，以及最終輸出的量化運算，皆需要特殊設計的對應硬體以進行運算加速。在本研究中，我們參考[35]

的命名，將這些功能整合為後處理運算單元(Post Processing Unit, PPU)。PPU 會與 PE 進行搭配，以進行 Transformer 中主要運算的加速。



4.1.2 記憶體配置

為了達成輸入資料的再利用，以及避免待運算資料於晶片內外往返造成 latency 增加，我們於晶片上配置 on-chip memory。本晶片上有個四種 memory，分別負責儲存 Ternary Weight、INT-4 Activation、Scale/Zero Point，以及晶片運算過程中的 Partial Sum。

我們參考[35]的做法，使用 register file 實現 memory。由於 Partial Sum memory 具有同時寫入與讀取的需求，因此採用的是 dual-port 的 memory，其餘三種 memory 則是 single-port 的配置。

各種類 memory 的儲存空間容量如表 4.1 所示。

表 4.1 On-chip memory 配置

Memory	Size (KB)
Partial Sum	32
Activation	8
Weight	16
Scale/Zero Point	4.5
Total	60.5

4.2 PE(Processing Element) Cube

PE 為本設計中負責執行矩陣乘加運算的核心電路單元。為了提升每個運算週期可處理的資料量，進而增強整體運算吞吐量，我們以 PE Unit 作為基本組成單元，進一步建構出規模更大的 PE Cube，以滿足 Transformer 模型在執行時對高效運算能力的需求。

本節將首先介紹 PE Unit 的架構設計，說明其如何實現 ternary weight 與 INT4 activation 的高效乘加運算，接著介紹 PE Cube 的整合架構，說明其在晶片中的配置方式。

4.2.1 PE Unit

PE Unit 的架構如圖 4.1，主要分為兩個部分，一是負責乘法運算的部分，二是負責相乘結果加總的加法器電路。首先是乘法器的部分，由於我們的被乘數之一為 ternary 精度，因此乘法器的結果相對簡單。

不過需要注意的是，為了將加法器以 Multi-bin Adder 的架構實現，乘法器的輸出需要 shift 一個+8 的 offset，以將乘法器結果轉為正數，由-7~+7 轉換為+1~+15。若不加入這個 offset，負值的 sign bit 會被 counter 當成輸入被加總，導致結果出現異常。因此，為了讓 multi-bin adder 的每一個 counter 皆是如同圖 4.1 中的 1、2、4、8 Counter，我們賦予一個 offset 來解決此問題。這個 offset 會在加法器完成運算(圖 4.1 中最後一個 Adder)後減回，且由於 offset 的計算為固定數值加減，因此不會造成明顯的硬體成本上升。

加法器部分，我們將乘法器輸出的 4 個 bit 分別加總，由於每個 PE Unit 可以處理 64 組的 TW4A 乘加運算，因此每一個 Counter 皆是負責 64 個 1 bit 的加總。最後會依照每個 bit 的位置做 shift & add 運算得到結果。在圖 4.1 中可以觀

察到，最末兩個 bit 的 Counter 中有兩個 adder，分別是高精度的 adder 以及章節 3.7 介紹的 OAI21 + AAI21 雙層近似加法器。

之所以要採用此配置是因為在近似加法的訓練結果觀察到，雖然近似加法可以有效降低功耗，但是並非所有運算皆可以承受近似加法造成的影響。為了讓我們的加速晶片在對精度有高度要求的場域中仍可正常執行運算，因此保留兩套加法電路，並透過 input gating 的方式選定將使用的加法電路，並避免另一套電路造成無謂的功耗。

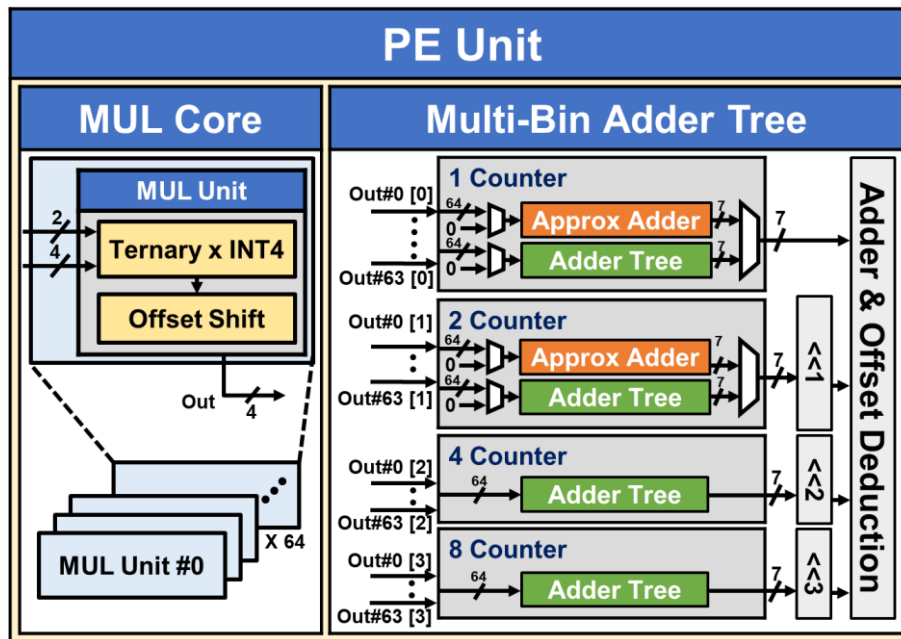


圖 4.1 PE Unit 電路架構

運算時，會將矩陣資料以 64 組為一單位的方式送入 PE Cube，如圖 4.2 所示。

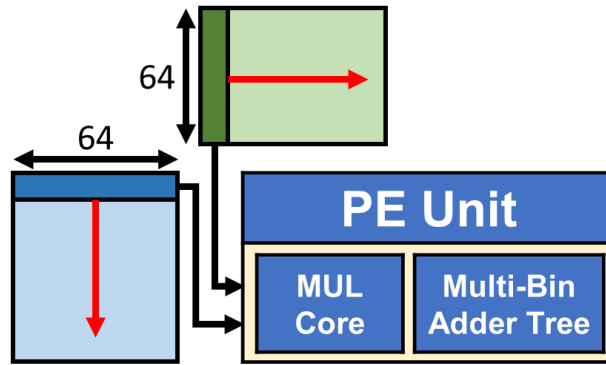


圖 4.2 PE Unit 資料流

4.2.2 PE Cube

為了擴增 PE 的運算能力，我們將 PE Unit 組合為更大的 PE Cube，堆疊方式如圖 4.3 所示。首先，我們將 16 個 PE Unit 組合為一個 PE Plane，接著將 PE Plane 重複四套，便可以組成一個 PE Cube。

由於一個 PE Unit 可以提供 64 組 TW4A 乘加運算，每個 PE Cube 有 64 個 PE Unit，因此可以提供 4096 組 TW4A 乘加運算算力。

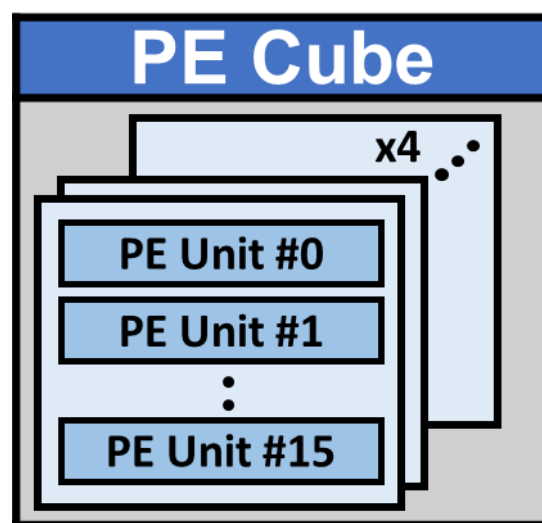


圖 4.3 PE Cube 架構

圖 4.4 以一個 PE Plane 為例圖解資料的讀取方式。在運算過程中，我們會先將 activation 的資料 preload 到 buffer 中，並傳入 PE 中，達成 temporal reuse。而 weight 的資料會於每個 cycle 從 memory 中讀取，並 broadcast 到各個 PE Unit 中，達到 spacial reuse，並依照圖中灰色箭頭方向依序讀取。

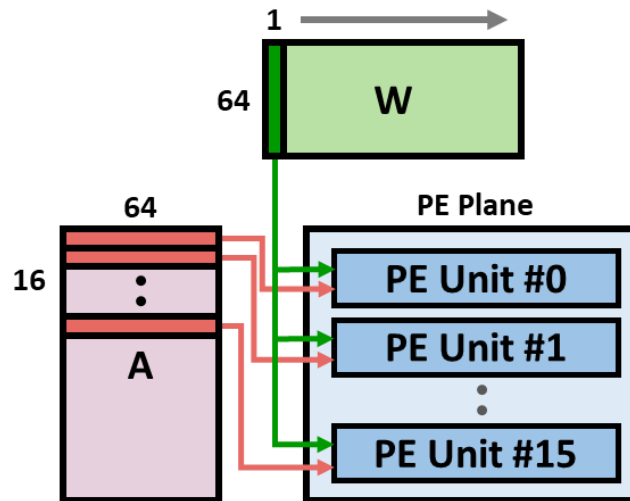


圖 4.4 PE Plane 運算流程

PE Cube 的運算流程如圖 4.5，由於一個 PE Cube 中有 4 套 PE Plane，因此運算維度得以擴展四倍。圖 4.5 的運算過程中，會接著 fetch 下一個 Activation tile 的資料，依照 Activation Memory 的大小，本晶片可以儲存兩個 tile 的 Activationy 資料。而 Weight 則依序讀取 128 筆的資料，因此一個 PE Cube 可以負責[32, 256]的 Activation 與[256, 128]的 Weight 的相乘運算。

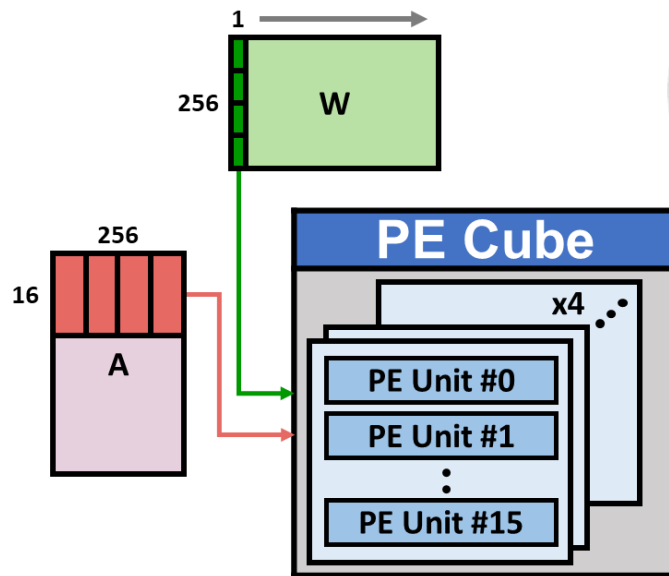


圖 4.5 PE Cube 運算流程

在實際的矩陣運算中，需要將大型矩陣拆分為多個 block 以利運算，以一個 PE Cube 而言，會需要如圖 4.6 一般進行拆解。圖中深色部分代表一個 PE Cube 的 memory 一次能夠容納的資料，在維度超過 memory 空間時，需要在一個 block 完成運算後，沿圖 4.6 中灰色箭頭的方向繼續讀取下一個 block。

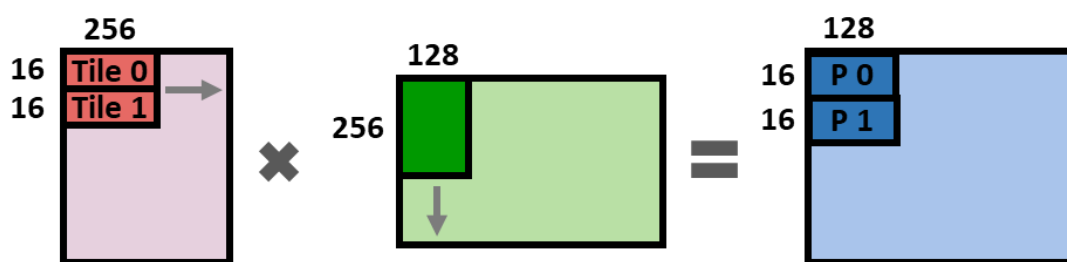


圖 4.6 矩陣乘法 block 讀取示意圖

最後我們也進行 PE Cube 中採用近似加法電路後的硬體參數進行分析，表 4.2 中列出了三種構型，包括 vanilla 的版本，也就是 PE Unit 中四個 adder 都使用高精度加法器。而 LSB approximate 則是將 LSB 的 adder 替換為近似加法器。再來是將 LSB 改為高精度與近似加法器並行的架構。最後則是我們最終使用的

版本，將末 2-bit 的 adder 採用高精度與近似加法並行的架構。由表 4.2 可以發現我們的做法會增加整體 PE 的面積，但在開啟近似運算的情況下，可以有效降低整體功耗。



表 4.2 PE Cube 架構近似加法架構比較表

PE Cube		Area	Area Improvements	Power	Power Improvements
Vanilla (baseline)		151429	--	18.7	--
LSB Approximated		140211	-7.4%	17.3	-7.5%
Dual Mode	Exact	158101	+4.4%	20.5	+9.6%
	Approx.			17.5	-6.4%
Dual Mode 2-Bit	Exact	163497	+8.0%	20.0	+7.0%
	LSB			17.9	-4.3%
	2-Bit Approx.			14.7	-21.4%

4.3 PPU(Post-Processing Unit)

4.3.1 架構介紹

介紹完負責主要 TW4A 乘加運算的 PE 電路後，我們接著介紹負責剩餘運算的 PPU 電路。PPU 主要負責以下幾種運算的實作，包括 PE 結果的加總、結果量化運算、QGELU、Multimax 等，而 PPU 與 PE 連接的方式如圖 4.7 所示。PE Cube 中 4 個 PE Plane 的 PE Unit 的結果會匯集至同一個 PPU 中，進行結果的加總與後處理運算，因此每一個 PE Cube 都會搭配一個 PPU 核心，每個 PPU 核心會包括 16 組 PPU Unit，對應到 PE plane 中的 16 個 PE Unit。

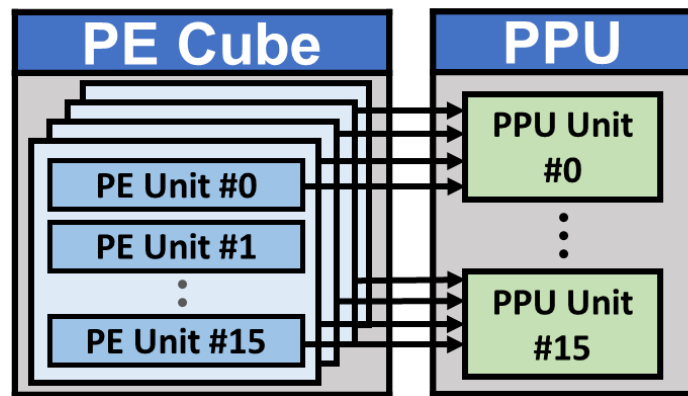


圖 4.7 PE Cube 與 PPU 連接關係

圖 4.8 是一個 PPU Unit 的架構，其中包括 Accumulator、Requantizer、QGELU、Pre-Sort 等電路，下方會對各個子模組進行詳細的介紹，透過模組間的整合，得以讓我們實現 Transformer 模型中主要運算的加速。PPU Unit 的輸入為各 PE Plane 的運算結果、儲存於 memory 中的 Partial-Sum、用於量化運算的 Scale 與 Zero Point，以及控制訊號等。

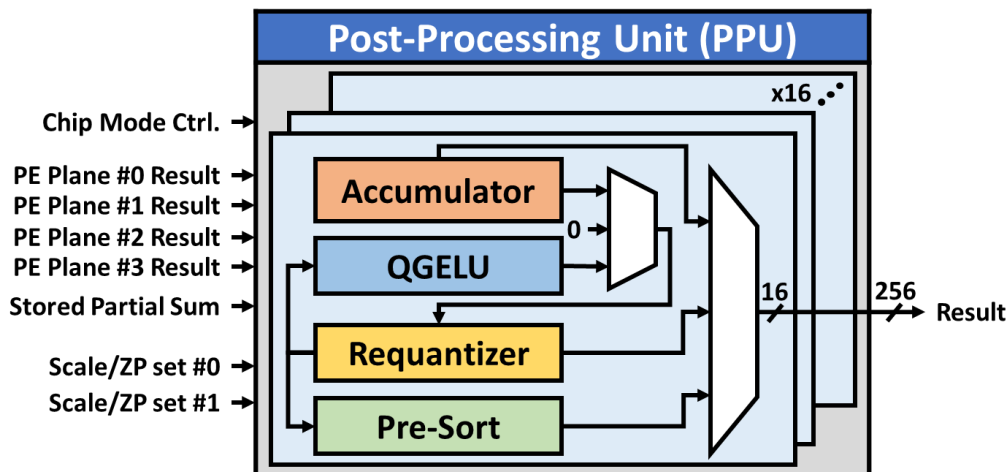


圖 4.8 PPU Unit 電路架構

4.3.2 Accumulator

在 PE Cube 內的各 PE Plane 完成乘加運算後，會將結果傳輸至 PPU 中的 Accumulator 進行加總，當矩陣維度較大時，我們需要將矩陣拆分以利運算，因此運算過程中產生的 partial sum 會需要儲存回 Partial Sum Memory，而在下個運算階段時，會從 memory 中讀取 Partial Sum，並與當下的 PE 結果相加，如圖 4.9 的架構圖中所示。若當下運算為第一個 tile，則不需要與 memory 的資料相加，因此會透過 MUX 控制加法器的輸入。

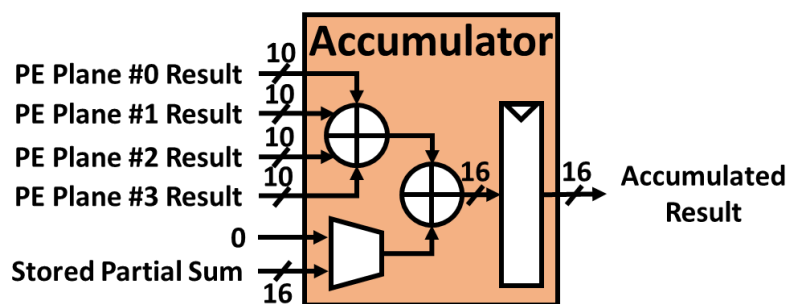


圖 4.9 Accumulator 電路架構

在實際運算中，除了矩陣的最後一個 tile 外，大部分的運算皆是普通的 MatMul 運算，此類型運算不涉及量化、QGELU 等後處理，僅需將 PE 的結果相加後儲存回 memory 便可。運算流程如圖 4.10 所示。

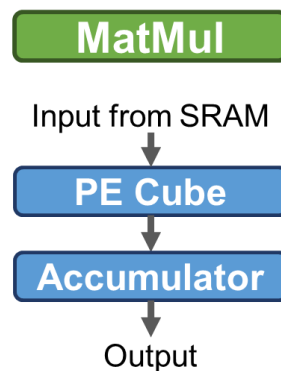


圖 4.10 MatMul 運算流程



4.3.3 Requantizer

Requantizer 電路主要負責數值量化的運算，其電路由一個乘法器與加法器組合而成，負責 Scale 的乘法以及 Zero Point 的加法。

然而，除了 Linear Layer 最後的量化外，Requantizer 還需要協助一些額外的運算。例如在 QGELU 運算中，Accumulator 的結果仍為 INT 精度，然而 QGELU 硬體需要的是實際的數值，因此會透過 Scale 跟 Zero Point 將結果轉為定點數，並傳入 QGELU 模組。在 QGELU 完成計算後，也會將結果傳回 Requantizer，透過另一組 Scale 跟 Zero Point 量化回 INT4 的表示。也因為需要兩組 Scale 跟 Zero Point 才能完成 QGELU 運算，因此從架構圖 4.11 上也可以觀察到，輸入會是兩組的 Scale 跟 Zero Point，而 MUX 會依照控制訊號好選定乘加器所需的參數組合。

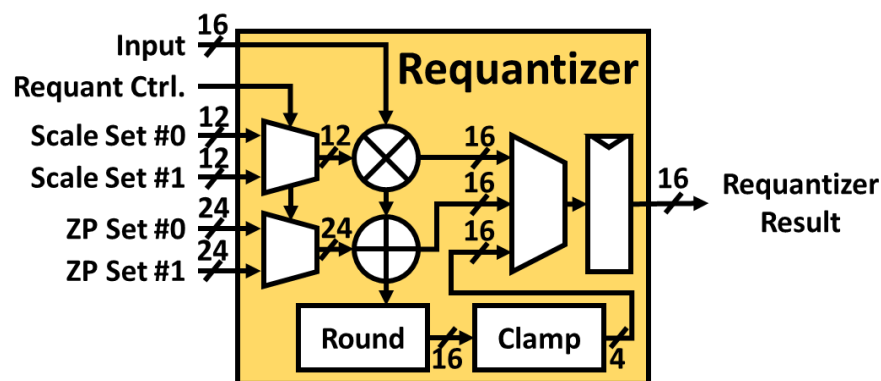


圖 4.11 Requantizer 電路架構

在 Multimax 運算中，如(式 3.10)可以觀察到，需要對兩兩一組數值的 difference 乘上共享的 Scale 才能貼近真實的 ratio 結果，因此，Requantizer 也需要負責此部分的運算，也就是乘上 Scale 的計算。

圖 4.12 是一般 Linear Layer 運算的流程圖，在最後一個 tile 的結果經過 Accumulator 後，便會送入 Requantizer 進行量化運算後才會輸出。

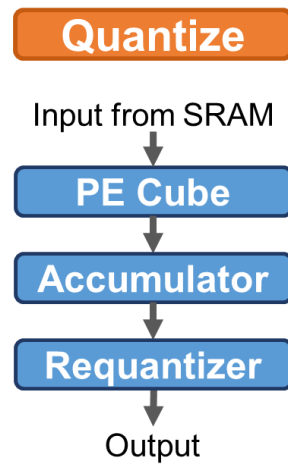


圖 4.12 Linear Layer 運算流程

4.3.4 QGELU

我們依照章節 3.6.2 的做法實作 QGELU 的硬體，如圖 4.13。圖中的 Predetermined QGELU Coefficient 便是表 3.4 展示的 shift & add 二項式係數。電路中，輸入會先經過一個比較器，確定其所屬的區間之後，在選定該使用的 coefficient 組合。而輸入數值同時也會進行平方項的計算，這也是此部分硬體最複雜的運算。

經計算得出的平方項、一次項以及依係數選定的常數項會通過 shift & add 的運算後得出 QGELU 的結果。若輸入數值大小落在我們定義的非線性區間外，則會選擇輸出該數值本身，或是 0。

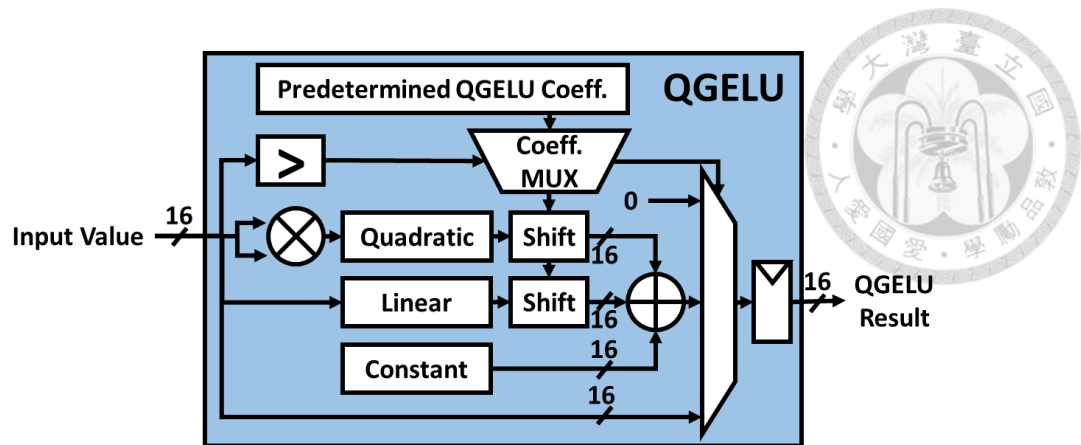


圖 4.13 QGELU 電路架構

QGELU 運算的流程如圖 4.14 所示，經過 PE 與 Accumulator 運算的結果仍是 INT 精度，因此需要先經過 Requantizer 將其轉為定點數，再送入 QGELU 運算。而 QGELU 的輸出仍是定點數形式，因此需要再送入 Requantizer 進行量化才能由晶片輸出。

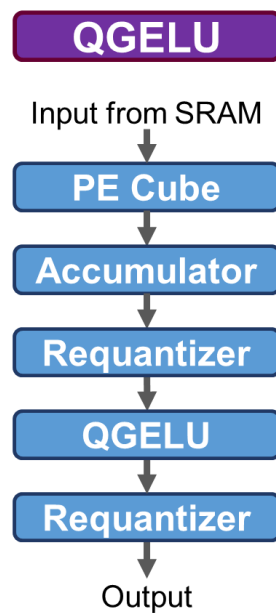


圖 4.14 QGELU 運算流程

為了比較 QGELU 電路與 GELU 電路的面積與功耗，我們使用 designware 建構了 FP32 精度的 GELU 模組，並透過 pipeline 的方式使得 GELU 電路運行於 200MHz，表 4.3 與圖 4.15 為誤差、面積、功耗的比較結果。

表 4.3 GELU vs. QGELU 硬體比較表

Module	MSE	Area (μm^2)	Normalized Area	Power (W)	Normalized Power	Single data Delay(ns)
FP32 GELU	1.34×10^{-8}	1.04×10^5	1	6.15×10^{-2}	1	5
QGELU	1.87×10^{-5}	1.76×10^3	0.0169	6.55×10^{-5}	0.0011	5

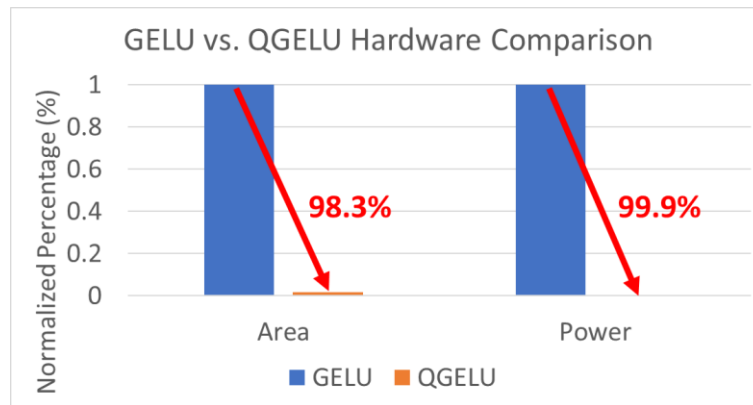


圖 4.15 GELU vs. QGELU 硬體面積功耗比較

4.3.5 Pre-Sort

在 Multimax 的運算中，在實際計算輸出 ratio 之前，必須先從當前運算結果中找出最大的前 N 個數值。此部分會由 PPU 中的 Pre-Sort 電路中進行處理，此部分的電路架構相對簡單，主要功能為接收來自 Accumulator 的輸出資料，進行排序後，將 top-N 結果傳送至下一階段的 Multimax 計算單元進行後續比值計算。

然而，考量到晶片最終輸出僅包含 top-N 個 ratio 數值，若僅輸出結果本身，外部運算裝置將無法得知每個數值對應於原始矩陣中的位置。因此，Pre-Sort 電

路在執行排序的同時，亦會將每筆數值對應的 index 資訊一併儲存，並於傳輸至 Multimax 計算單元時一併輸出，以確保外部系統能正確還原資料位置並完成後續應用。

Pre-Sort 電路架構如圖 4.16 所示。

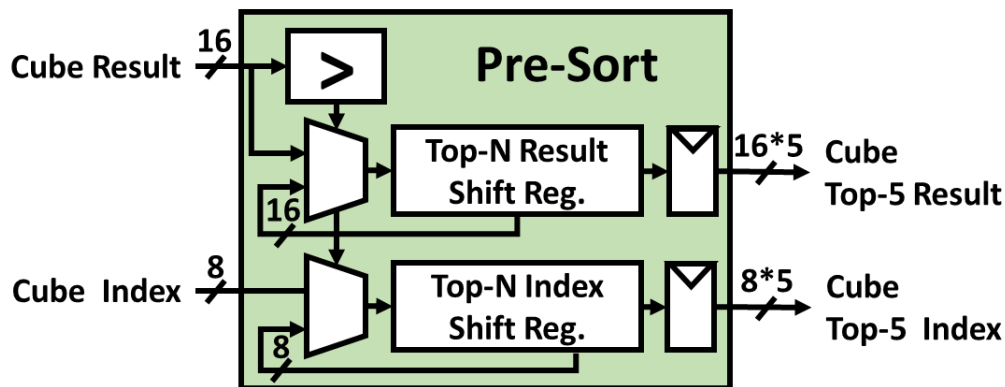


圖 4.16 Pre-Sort 電路架構

4.4 Multimax Calculation Unit

在介紹完 Pre-Sort 電路後，緊接著介紹共同組成 Multimax 加速運算的另一個環節，也就是負責計算 Multimax 結果 ratio 的 Multimax Calculation Unit。需要注意的是，由於 PE Cube#0 與 PE Cube#1 的結果在實際矩陣中同屬一個 row，因此在 Multimax 運算中是需要被一起處理的。在 Pre-Sort 電路將 top-N 運算結果與 index 傳入後，會先將兩組 top-N 數值合併為一組，也就是這個 row 中實際的 top-N 數值，對應到圖 4.17 電路架構中的 Sort & Index Rearrange 環節。

圖 4.17 中以 PE Cube#0 與 PE Cube#1 的結果作為展示。

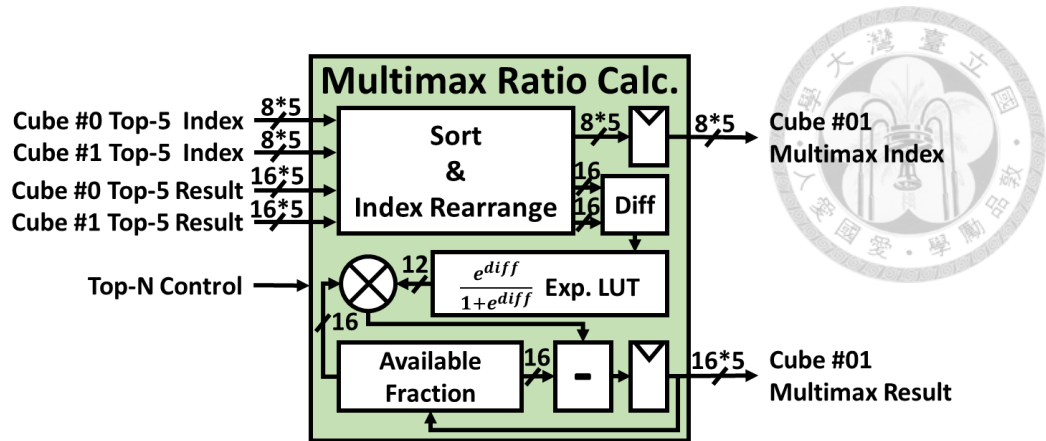


圖 4.17 Multimax Calculation Unit 電路架構

若 Multimax 設定的 N 值為 1，便會直接指派最大輸出比值為 1 並結束運算。但若 N=3 或 5，則需要進行進一步的比值計算。在 Multimax 計算比值的過程中，佔據很大比例硬體成本的是相當於 sigmoid 函數的運算，為了避免實作 exponential 與除法的電路，我們採用 LUT 的方式來實作 sigmoid，其輸入是兩兩一組數值的 difference。由於當一組數據的 difference 相差過大時，sigmoid 的輸出可以直接視為 1，因此我們僅針對部分非線性片段實作 LUT，並不會造成過大的硬體成本。

由(式 3.10)中的介紹可知，除了最大的一組數據外，其餘兩兩一組的數據得到 sigmoid 的結果後皆須乘上前一次運算留下的 fraction，以保持 Multimax 的輸出結果總和為 1。因此，Multimax Calculation Unit 最後的運算環節便是乘上 fraction 以獲取輸出比值，並更新殘餘 fraction 的數值以供下一組數值的比值運算。當計算到最後一個數值時，便會將殘餘的 fraction 當作其輸出，便可以確保 N 個輸出結果的總和固定為 1。

圖 4.18 為 Multimax 的運算流程，在經過 Accumulator 得出結果後，為了計算實際的 difference，需要依照(式 3.10)將結果乘上 Scale，因此需要先經過 Requantizer 的計算。

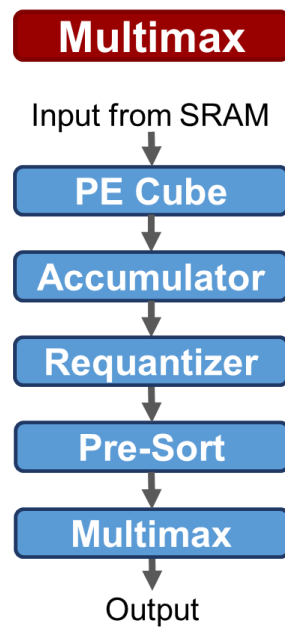


圖 4.18 Multimax 運算流程

與 QGELU 相同，為了比較 Multimax 電路與 Softmax 電路的面積與功耗，我們使用 designware 建構了 FP32 精度的 Softmax 模組，表 4.4 與圖 4.19 為面積與功耗的比較結果。

表 4.4 Softmax vs. Multimax 硬體比較表

Module	Area (μm^2)	Normalized Area	Power (W)	Normalized Power	Clock Cycle (ns)
FP32 Softmax	6.50×10^4	1	6.16×10^{-3}	1	5
Multimax N=5	2.70×10^3	0.042	1.72×10^{-4}	0.028	5

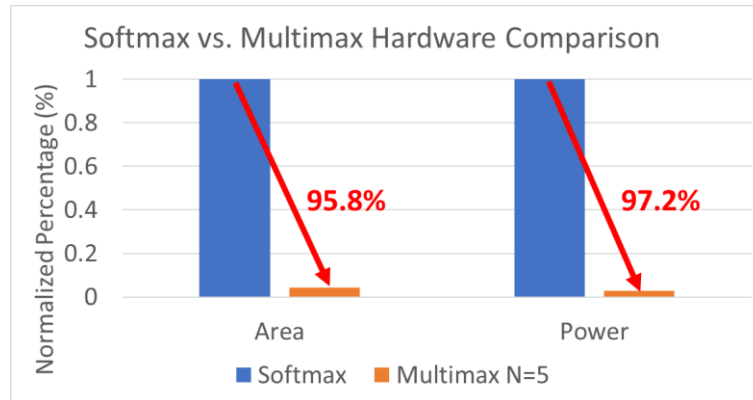


圖 4.19 Softmax vs. Multimax 硬體面積功耗比較

4.5 Self-Test Circuit

由於 IO pad 的對時序的要求較嚴格，以及 V93000 量測設備的輸出負載較大 (約 40pF)，可能使得實際放上機台量測時晶片內部的供電無法確實地將大量輸出訊號於時序要求內傳送至晶片外導致量測結果不理想，或是晶片的最大運作頻率可能會與模擬結果存在差異。

為了解決此問題，本研究在晶片內部設計了自我測試電路(Self-Test Circuit)。當所有運算完成後，晶片會將內部計算結果透過 XOR-based 的方式合併為單一數值，並與外部提供的標準參考值進行比較。晶片僅輸出一個正確訊號(correct signal)與一個有效訊號(valid signal)，且兩者皆會維持 32 個 clock cycle，以確保即使在 IO 腳位驅動能力有限的情況下，也能穩定地被量測設備擷取。Correct 與 Valid 訊號的波型圖如圖 4.20 所示。

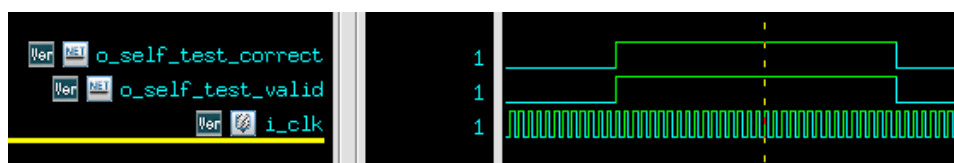


圖 4.20 Self-Test Circuit 輸出訊號波形圖

其硬體架構如圖 4.21 所示。在實際量測過程中，晶片會先以較低頻率進行完整的功能性驗證，以確保功能正確性。在此階段，所有內部運算結果皆會輸出至晶片外部，並與 testbench 的 golden pattern 進行比對。而在頻率提高至 IO 頻寬成為限制因素時，則會啟用自我測試機制，由晶片內部自行驗證運算正確性，以克服 IO 傳輸限制與高負載造成的問題。

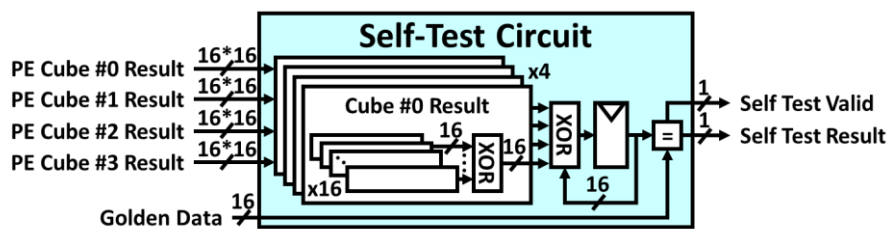


圖 4.21 Self-Test Circuit 硬體架構

4.6 晶片架構

圖 4.22 為整體晶片架構，我們以顏色區分不同功能的模組。圖中黃色部分為 PE Cube，共配置了四組，而每一個 PE Cube 接配置一組 PPU，負責將 PE 運算完成的資料進行後處理。而在 Multimax 運算中，PPU 僅會對結果數值進行排序，排序完成的結果會被送入 Multimax 運算單元中，並依照 Multimax 的設定計算出相對應的 ratio 結果。

圖中上下部分綠色的單元為 memory block，如同前文介紹的，我們使用的四種 memory 類型，分別是 Weight、Activation、Scale/Zero Point、Partial-Sum，並且在 data reuse 的情境下，Cube 之間可以共用 Weight 與 Activation 的資料，因此僅 Scale/Zero Point 與 Partial-Sum 的 memory 是每個 PE Cube 皆須搭配一組，而 Weight 與 Activation 為兩個 PE Cube 共享一組的配置。



最後是藍色部分的控制電路，負責 IO 處理、memory 讀寫控制、晶片運作模式與流程控制等，並包含了章節 4.5 介紹的 Self-Test 電路。

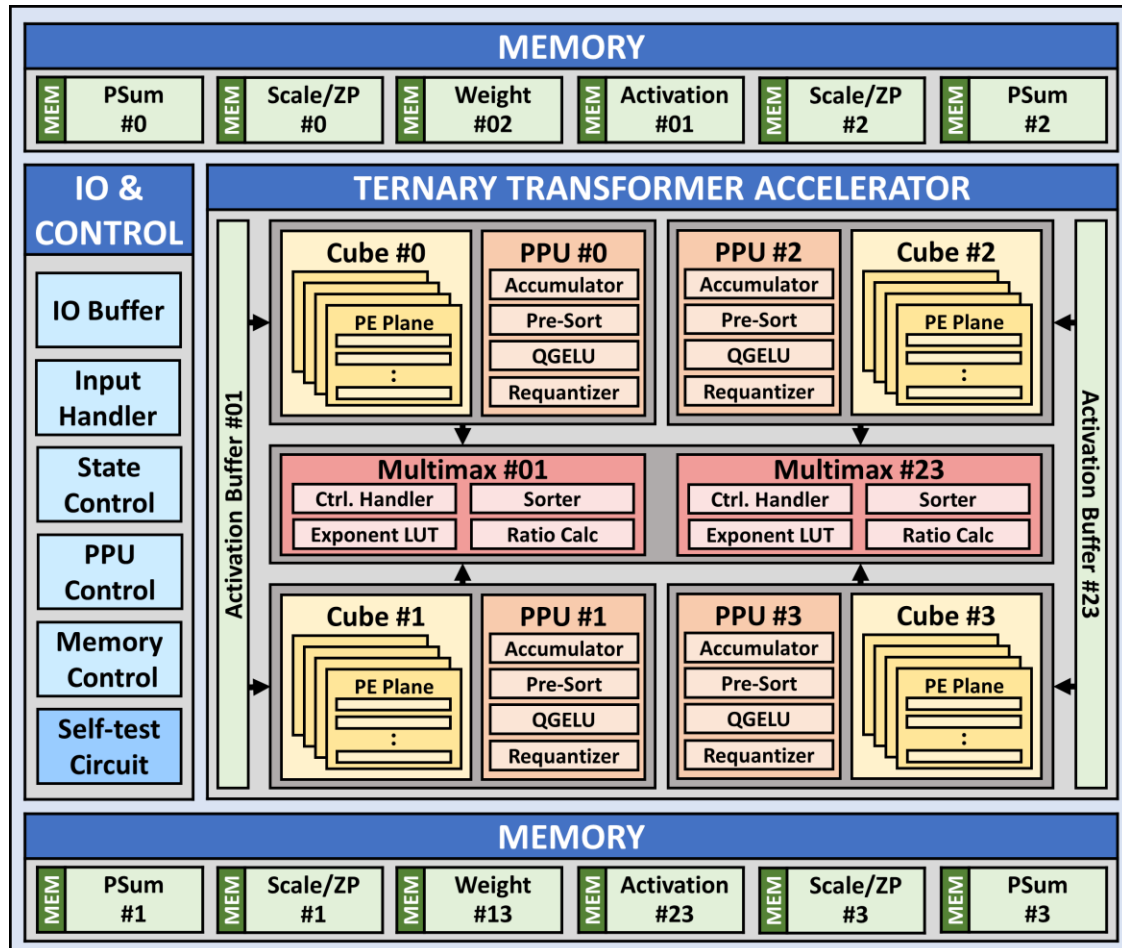


圖 4.22 晶片整體架構圖

4.7 晶片計算流程

章節 4.2.2 中介紹了單一個 PE Cube 的運算流程，不過章節 4.6 也提到在實際的晶片中，配置了 4 個 PE Cube，因此本段落會介紹配置 4 個 PE Cube 下的運算方式。

當只有一個 PE Cube 時，運算過程如圖 4.23。

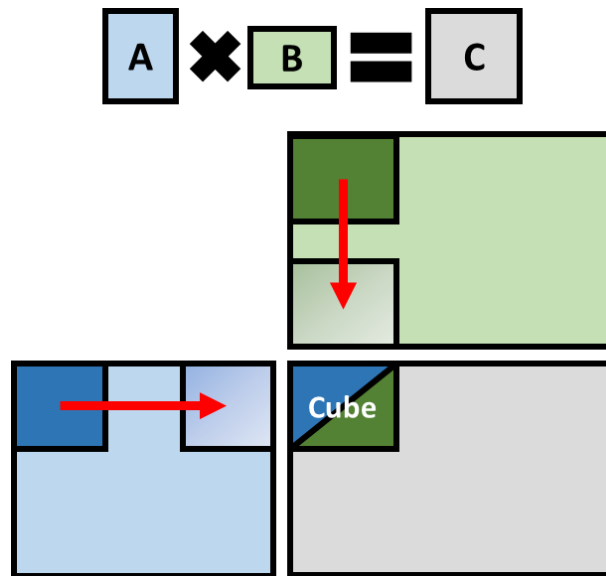


圖 4.23 單一 PE Cube 運算過程

當 PE Cube 擴展為 4 組時，運算流程如圖 4.24 所示，可以注意到整體 throughput 變為 4 倍。

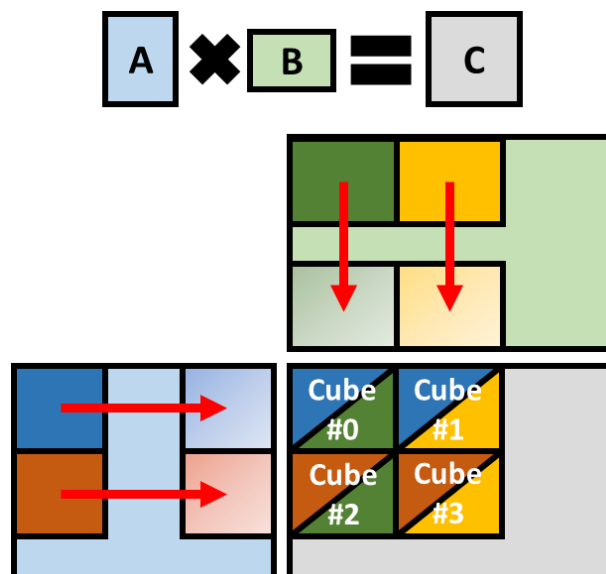


圖 4.24 四組 PE Cube 運算過程

此外，此配置可以在 PE Cube 間達到 data reuse，即 Cube#0 與 Cube#1 共用圖 4.24 中藍色的 Activation block，Cube#2 與 Cube#3 共用紅色的 Activation block。Weight 的部分，Cube#0 與 Cube#2 共用綠色的 Weight block，Cube#1 與 Cube#3 共用黃色的 Weight block。此作法使得即使 PE Cube 的數量由一套變為四套，Activation 與 Weight 的 memory 需求僅會變為兩倍。不過由於結果變成了 4 倍，因此 Partial Sum Memory 需要變為 4 倍。

此外也可以觀察到，Cube#0 與 Cube#1 的結果同屬相同的 row，因此在計算 Multimax 結果時要一併計算，Cube#2 與 Cube#3 也是同理。這解釋了為什麼圖 4.22 的晶片架構圖中 Multimax Calculation 模組僅有兩組的原因。

值得一提的是，雖然本研究將此加速晶片定調為 Transformer 模型加速晶片，但由於其本質上仍屬於矩陣相乘的運算電路，因此若其餘模型中的運算，例如 CNN 模型，可以透過 im2col 或其他轉換形式將輸入變換為本晶片可以支援的資料運算流程，則也可以進行相關運算之加速。

4.8 晶片支援運算總結

此段落總結本研究提出之加速晶片的運算支援範圍，圖 4.25 中將 Transformer block 中本晶片可支援的運算列出，而圖 4.25 中灰色方塊則為不支援之運算。由於 add & norm 運算需要將資料反覆存入晶片中，且本晶片的 on-chip memory 並不算多，造成資料搬運上的 overhead，因此本研究並不支援此部分運算。此外，Single-Head Attention 中的乘上 V 矩陣的運算，在加入 Multimax 近似後，輸出僅會有 N 個 ratio，因此與 V 矩陣的矩陣乘法中每個 row 的運算會被簡化為 N 個數值的乘加運算，為了避免因少量運算還需要將資料送入晶片造成搬運的時間損失，我們選擇讓外部運算單元負責此部分運算。

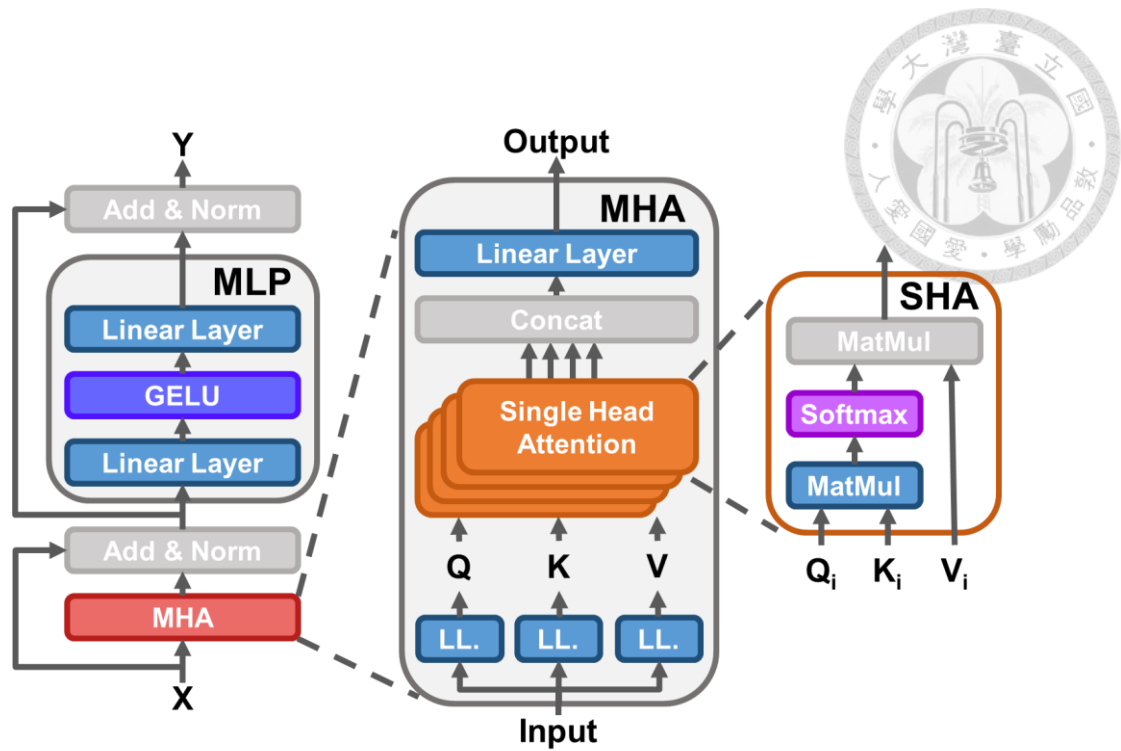


圖 4.25 加速晶片運算支援範圍

為了更詳盡說明各模式下啟動的模組與資料的傳遞過程，圖 4.26 以圖 4.22 的晶片架構圖搭配灰色遮罩將該模式下不使用的模組屏蔽，而其中紅色箭頭做為資料傳遞的展示。

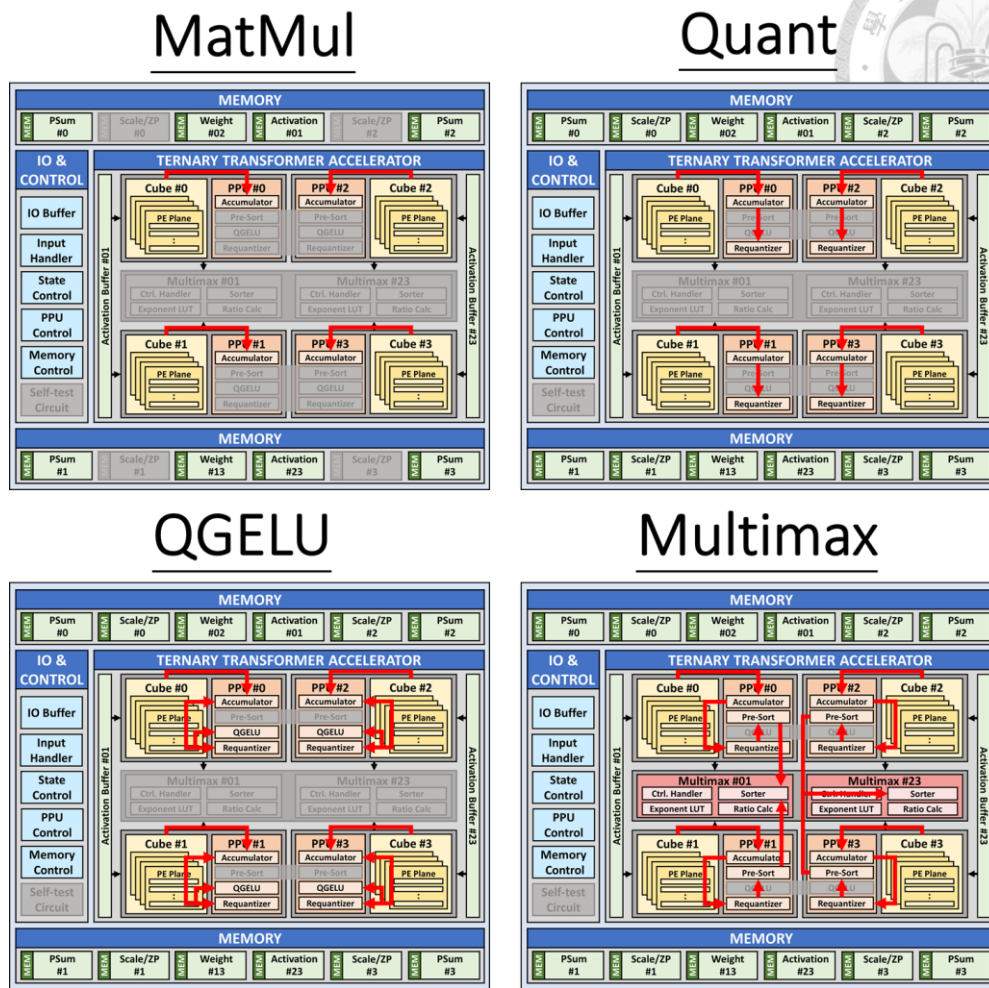


圖 4.26 各模式資料傳遞過程與啟動模組示意圖

4.9 第四章總結

本章節介紹了晶片中主要運算架構的電路，包括負責乘加運算的 PE Cube，以及組成 PE Cube 的 PE Unit 電路，其中也說明如何將近似加法電路融入 PE 的運算中。此外也針對 PE 的資料流，以及運算資料重複利用的方式進行說明。

接著是負責額外運算的 PPU 單元，為了加速除了乘加運算以外的計算，例如數值量化、QGELU、Accumulate、Multimax 前排序等，需要額外的硬體來輔

助，這便是透過 PPU 才得以實現。其中也介紹如何將章節 3.6.2 提出的 QGELU 技巧透過硬體的方式實現。



為了支援 Multimax 的 ratio 運算，我們於 PPU 外額外設計了 Multimax 的電路，透過 LUT 的輔助，我們可以簡化 Multimax 中較複雜的運算，同時保留近似於 Softmax 的運算結果。且為了支援 Multimax $N=1, 3, 5$ 的計算，我們也將此部分的電路設計為可以迭代運算的形式，在不增加硬體成本的情況下支援不同 N 的結果計算與加速。此部分的 Multimax ratio 運算電路與 PPU 中的 Pre-Sort 電路，共同組成 Multimax 的加速電路。

有鑒於下線晶片的最終目標是透過量測儀器獲取實際的功耗與工作頻率數據，而在使用 TSRI 提供之量測儀器時可能遭遇輸出附載過大的情況，導致實際量測出來的晶片參數與模擬的結果不同。為了避免晶片輸出結果時無法確實驅動數十個輸出腳位，我們於晶片中額外設計了一套 Self-Test 電路，透過 XOR 的方式，依照目前的運算結果於晶片內部生成一組混和結果，並與來自 testbench 的結果進行比較，最終僅需輸出兩個 bit 的結果以供外部 testbench 辨識正確性即可。透過在低頻時使用一般輸出方式驗證結果正確性，高頻時使用 Self-Test 電路驗證，希望能降低因儀器附載造成的不理想效應。

最後我們介紹了晶片的實際運算流程以及整體的晶片架構，為了支援日益增長的模型資料維度，避免因 on-chip memory 不足造成晶片無法完成運算，我們將晶片設計為可以分多 pile 運算的方式進行矩陣乘法的實作，此外，透過 activation 與 weight 的資料 reuse，也可以降低讀取 memory 資料造成的 overhead，並提升整體的功耗表現。

圖 4.22 展示了本研究提出的加速晶片整體架構，希望透過圖像化的方式更直觀的介紹晶片中各架構的組成、功能分工，以及彼此之間的連接關係。除了 PE Cube、PPU、Multimax 運算電路外，也納入了 memory 以及 control 模組的配置，以提供更全面的晶片介紹。



第五章 晶片實現



第四章詳細介紹了各運算模組的硬體實現方式與整體晶片之架構，並介紹本研究設計之加速電路的運算流程。本章節則聚焦在晶片的實現上，包括晶片設計與驗證流程、晶片參數與晶片比較表等資訊。

5.1 晶片設計流程

本加速晶片之設計為軟硬體相輔設計，首先會於軟體層面驗證所提出之技巧是否適用於現有 Transformer 模型架構，同時不會造成顯著效能損失。此部分的實作於實驗室內部之 GPU 工作站中完成，包括 python 程式碼的撰寫與各式模型量化感知訓練。在實作細節定案後，會進入 RTL 設計環節，將所構想之量化與近似運算以 verilog 實現，並設計對應之 testbench 以利設計正確性的驗證工作。為確保所設計之硬體與軟體的執行結果相符合，我們採用 python 以軟體的方式生成 golden pattern，透過軟硬體交叉比對的方式驗證我們的電路運算結果。

完成 RTL 設計後，便進入合成與 APR 環節。由於我們使用台灣半導體研究中心(Taiwan Semiconductor Research Institute, TSRI)所提供之 40 奈米(TN40G)製程進行晶片下線，為保護機敏資料不被外洩，任何涉及製程資訊的流程(包括電路合成、APR、power 分析等)皆須於 TSRI 提供之 EDA Cloud 2.0 中完成。我們使用 Design Compiler 進行晶片合成，並進行 gate level 驗證。在此步驟完成後，便使用 Innovus 開始 APR，最終產生完成布局與繞線的 GDS 與 Netlist 檔案。而後會進行 post simulation 以驗證晶片功能性是否正常，以及 area、power 等晶片參數分析。最後才會上傳至 TSRI 進行下線。



整體流程如圖 5.1 所示。

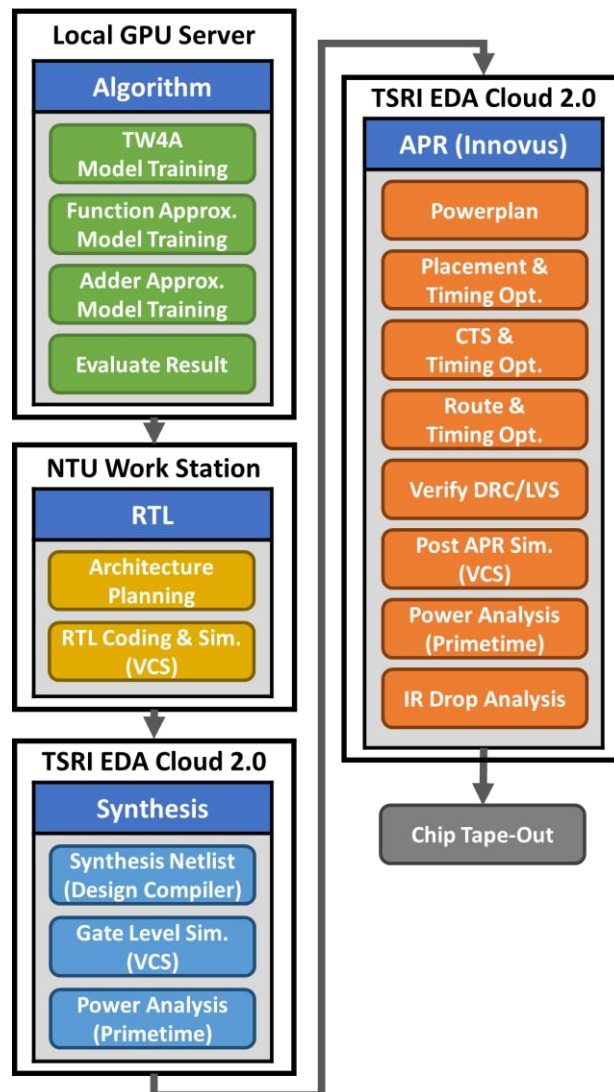


圖 5.1 晶片設計流程

5.2 晶片驗證流程

上個段落提到我們以 Python 生成測試資料以驗證電路運算正確性，此段落以圖像化的方式說明實作細節。首先 Python 會產生多組隨機的 TW4A 輸入(包含 weight 矩陣與 activation 矩陣)，以及用於控制晶片的訊號。輸入會傳入晶片中運算，並取得硬體電路的運算結果，同時，我們也使用 Python 軟體讀取這些輸入，並用純粹的軟體程式計算結果，最後再將兩者進行比對。流程如圖 5.2 所示。

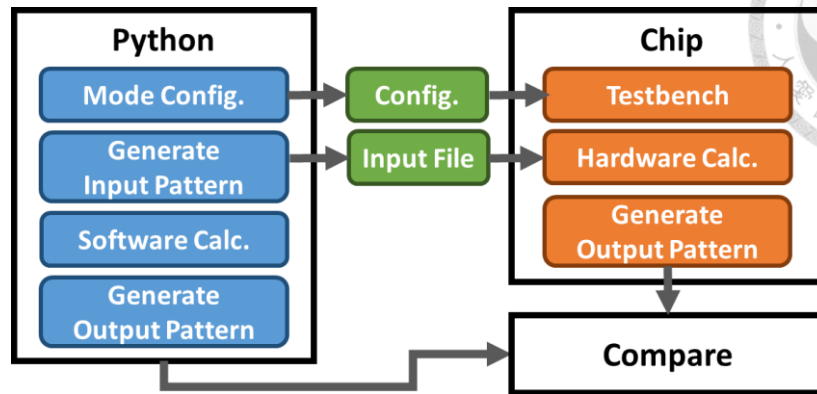


圖 5.2 晶片驗證流程

5.3 晶片參數

5.3.1 晶片布局

本研究之加速晶片採用 TN40G 製程進行實作，完乘 APR 之布局圖如圖 5.3 所示。布局圖中有幾項值得注意的重點，首先是 memory 的擺放，由於晶片中存在 Weight、Activation、Scale/Zero Point、Partial Sum 四種不同用途之 memory，而 memory 的擺放位置則貼近晶片外圍，以讓出空間供邏輯電路擺放與繞線。布局圖中也標註了晶片整體與核心(Core)的尺寸，分別為 $2.08mm \times 2.08mm$ 與 $1.56mm \times 1.56mm$ 。布局圖中的模組劃分僅為示意圖，除 memory macro 位置外並不代表該模組實際分布位置。

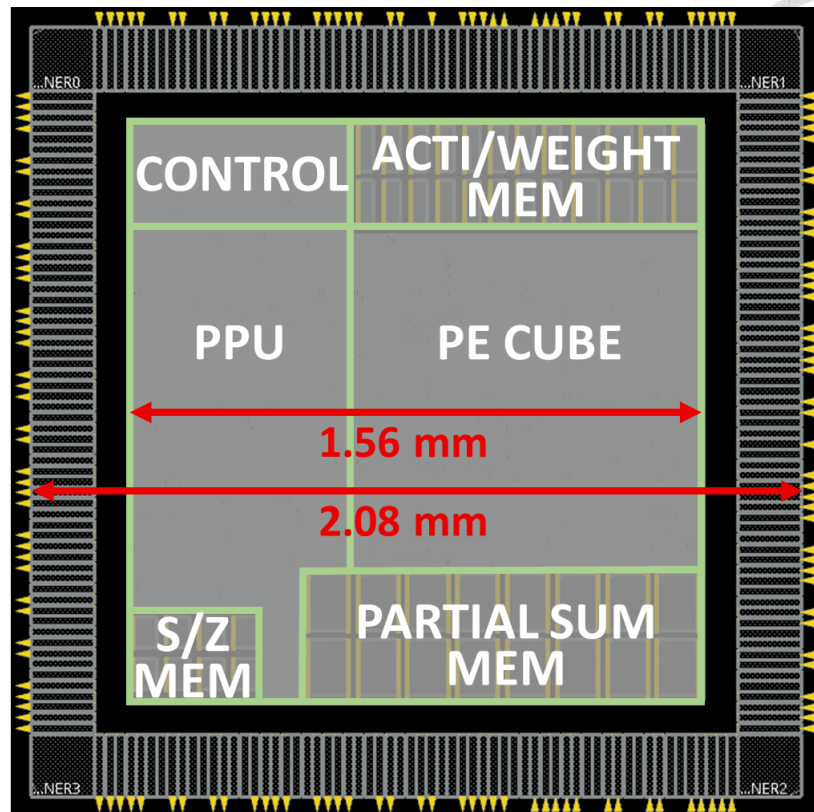


圖 5.3 晶片布局圖

晶片的 Power Plan 部分，我們使用 10 圈的 power ring 以利 IO 輸入的電壓平均分散到晶片外圍各處。此外，每一群組的 memory macro 也會擁有自己的 block ring 以支援 memory 內部的電源供給。用於提供各邏輯閘的 power stripe 也使用雙層配置(橫、豎各兩層)以加強 power plan 的韌性避免過大的 IR drop 出現。

晶片的 static IR drop 分析結果以及晶片 layout 如圖 5.4 所示。

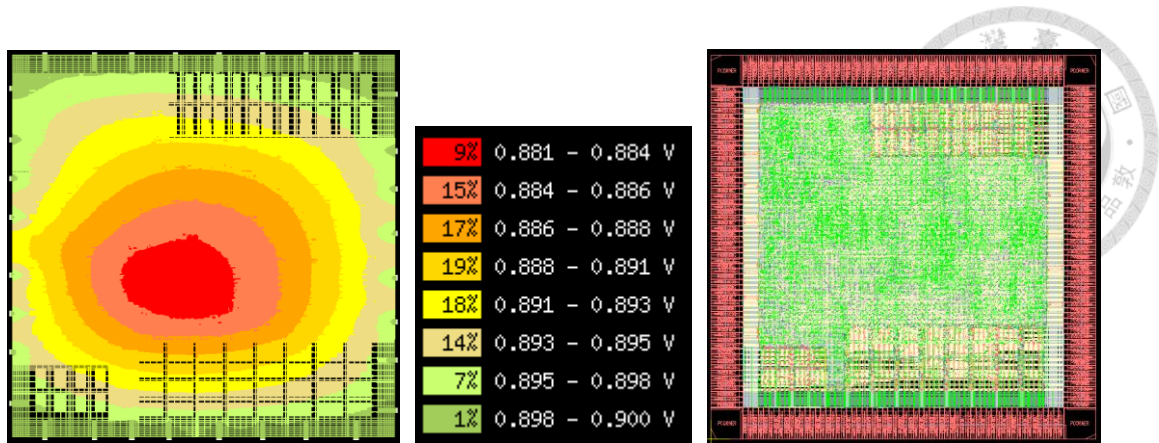


圖 5.4 晶片電壓衰退分布圖與晶片 layout

5.3.2 晶片數據分析

目前已有眾多神經網路加速器相關研究，為了比較不同架構之硬體效能，學者通常採用以下幾個指標進行比較：運算吞吐量、面積效率以及能源效率。此處會就各指標進行介紹。

運算吞吐量(Throughput)是衡量加速電路算力的重要指標，不過由於電路中的運算單元不可能無時無刻處於運作狀態，因此吞吐量一般泛指 PE 的極限吞吐量。吞吐量的單位為 OPS，即 Operations Per Second。雖然在不同精度與目標的運算中，operation 的定義會有些微差異，但此指標仍是目前加速晶片衡量吞吐量的標準指標。計算方式如(式 5.1)所示。

$$Throughput = \frac{\text{Number of Operations Per Cycle(OP)}}{\text{Cycle Time}(S)} \quad (\text{式 5.1})$$

本加速晶片的 operation 定義如下：一個 ternary weight 乘上 INT4 activation 便定義為一個 operation。此外，雖然在 PE Unit 中的 adder 是以 multi-bin 的方式將 INT4 結果拆分為 1 bit 分別加總，但在計算 operation 時，仍是以兩個 INT4 結

果的相加定義為一個 operation。在一個 PE Unit 中，每個 cycle 可以處理 64 個乘法運算與 63 個加法運算，共 127 個 operation。由於本晶片中配置 4 個 PE Cube，每個 cube 中有 64 個 PE Unit，因此在 200MHz 的工作頻率下，算力約是 6.50TOPS。

隨著製程演進與加速電路的發展，晶片的運行頻率與運算量急速增加，因此目前普遍使用的單位為 TOPS，即 Trillions Operations Per Second，會議或期刊論文中的晶片參數也較常以此單位標註。

不過，若無止盡的堆疊運算單元，吞吐量的數值便會不斷增加，然而這個做法是不切實際的，高效能的加速電路應能在盡可能壓低面積與功耗的情況下提高電路的吞吐量。因此，需要有一個標準化的指標，以評估各加速電路設計在同單位面積或功耗下，能發揮多少的運算效能。這便是面積效率(Area Efficiency)與能源效率(Energy Efficiency)的重要性。

面積效率的計算方式如(式 5.2)所示，面積效率的單位為 $TOPS/mm^2$ 。

$$Area\ Efficiency = \frac{Throughput(TOP)}{Chip\ Area(mm^2)} \quad (式\ 5.2)$$

能源效率的計算方式如(式 5.3)所示，能源效率的單位為 $TOPS/W$ ，在能耗要求極高的邊緣運算場域中，能源效率的表現尤為重要。

$$Energy\ Efficiency = \frac{Throughput(TOP)}{Power(W)} \quad (式\ 5.3)$$

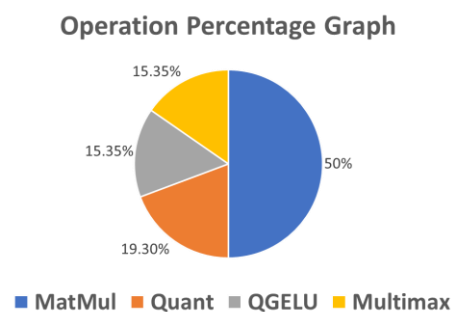


為了提高吞吐量，往往面積與功耗也會隨之上升，因此面積效率與能源效率的用意便是讓各研究者在運算效能與硬體成本間找出平衡點，避免無意義的增加運算能力的情形發生。

表 5.2 中彙整本研究之加速電路晶片的各項參數，包括晶片面積、運行電壓、工作頻率、精度、極限吞吐量、功率、面積與能源效率等。這些參數也將會在下一個章節中與其他現有論文之加速晶片進行比較。由於論文撰寫當下尚無晶片實際測量結果，因此功率部分以 Primetime 進行 Post Simulation 之分析結果為主。且為了避免資料傳輸與 memory 讀取時間運算電路 IDLE 造成能耗估算失準，我們的能耗分析僅取運算模組運算的部分進行 power 分析。此外，為了更加貼合實際運算的功耗，我們分析 ViT-base 架構中本晶片支援的各運算的佔比，並以此計算出 weighted power 做為晶片參數表中的功耗數據。

表 5.1 ViT 架構各加速運算模式佔比

MODE	Percentage(%)
MatMul	50.00
Quant	19.30
QGELU	15.35
Multimax	15.35
total	100



在晶片佈局中，IO pad 以及放置 core power ring 的 margin 佔據了許多面積，以我們的晶片而言，實際的電路本身僅占了晶片面積的 56.3%，因此為了避免這些與運算無關的部件影響面積與能源效率的數據，我們將 Die 與 Core 的數據皆列出。Die 意指整個晶片，包括核心運算電路、IO pad、Core margin 等，而 Core 僅代表核心運算電路。



表 5.2 晶片(Die)參數表

Specification		
Technology		
TN40G		
Die Area		
$2.08 \times 2.08 \text{ mm}^2$		
Core Area		
$1.56 \times 1.56 \text{ mm}^2$		
Memory		
60.5 Kbytes		
Supply Voltage		
0.81 V		
Frequency		
200 MHz		
Precision	Weight Key	Ternary
	Activation Query	INT4
Performance		
6.50 TOPS		
Power		
200.9 mW		
Energy Efficiency	Core	32.8 TOPS/W
	Die	32.3 TOPS/W
Area Efficiency	Core	2.67 TOPS/mm ²
	Die	1.50 TOPS/mm ²

此外，由於 TSRI 所提供的 IO pad 對於 timing 的要求較高，且量測機台之附載很大，導致加上 IO pad 後晶片工作頻率難以提升超越 200MHz，因此我們另外建立了一張晶片參數表，包含的是不含 IO pad 的核心運算電路，並對其進行 power 分析的結果。若不考慮 IO pad 的影響，電路可以運行至 250MHz 或更高，不過由於並未對此設計進行 APR，因此表 5.3 中的 power 數據為 gate simulation 的 power 分析結果。

表 5.3 核心運算電路(Core)參數表

		Specification
Technology		TN40G
Die Area		--
Core Area		$1.56 \times 1.56 \text{ mm}^2$
Memory		60.5 Kbytes
Supply Voltage		0.81 V
Frequency		250 MHz
Precision	Weight Key	Ternary
	Activation Query	INT4
Performance		8.13 TOPS
Power		224.3 mW
Energy Efficiency	Core	36.25 TOPS/W
	Die	--
Area Efficiency	Core	3.34 TOPS/mm^2
	Die	--



為了更直觀地呈現本研究所設計之加速晶片中各模組的硬體資源配置與功耗分佈情形，本段落將透過圓餅圖的方式，分別進行晶片核心模組面積比例與多種運作模式下功耗佔比之分析與觀察。

圖 5.5 顯示的是晶片核心電路之面積分佈情形。需注意的是，實體晶片中仍包含 IO pad 及 margin 等區域，這些部分雖然佔據一定比例的總面積，但因不屬於實際運算核心，因此本分析僅聚焦於純粹運算相關模組(Core)的面積構成。由圖中可觀察到，除了佔據顯著比例的 memory 與 buffer 單元外，運算模組中面積佔比最高者為加法器模組。這也是為什麼我們會積極討論近似加法器的可行性與低硬體成本實現方式。

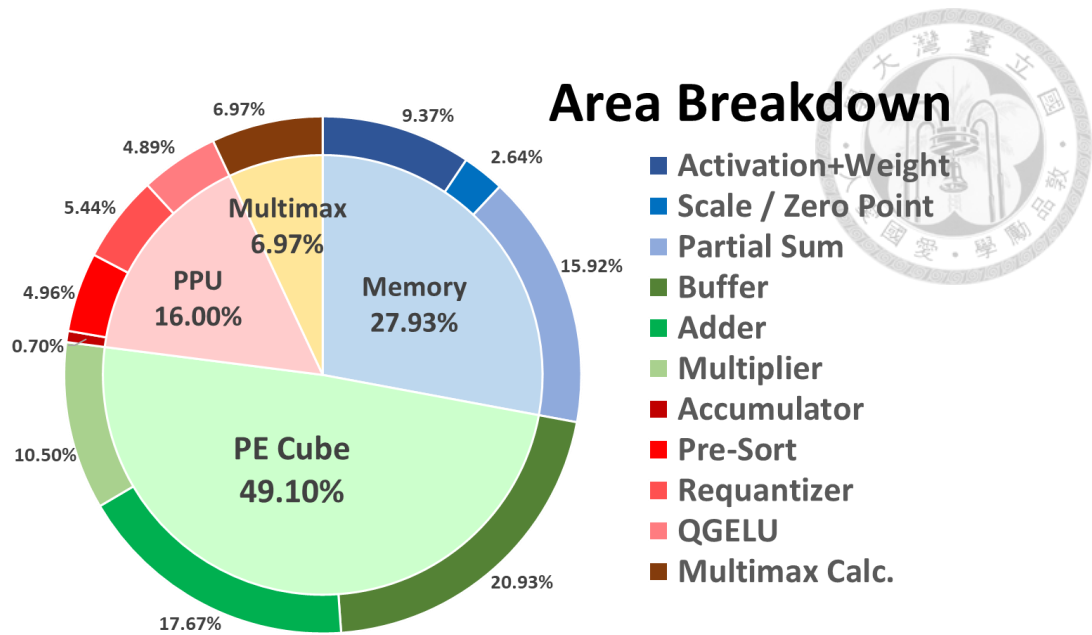


圖 5.5 面積佔比圖

另一方面，晶片在執行不同類型的 Transformer 運算任務時，因啟動模組與資料流動路徑的差異，將導致整體功耗來源有顯著變化。因此，我們亦針對晶片於多種工作模式下的功耗比例進行深入分析，並以圖 5.6~圖 5.9 呈現各模組功耗佔比的變化趨勢。以下將依據不同運作場景逐一討論分析結果，並探討各模組功耗與電路設計策略之間的對應關係。為了更直觀觀察晶片中各模組的運行狀態，我們將該模式下不作動的模組以灰色表示。

首先是單純進行矩陣乘法的運算，如圖 5.6，由於不涉及量化、QGELU、Multimax 等運算，因此功耗佔據比例最高的便是 PE Cube 中的乘法器與加法器，其餘運算模組僅貢獻了些微的功耗。



MatMul Mode

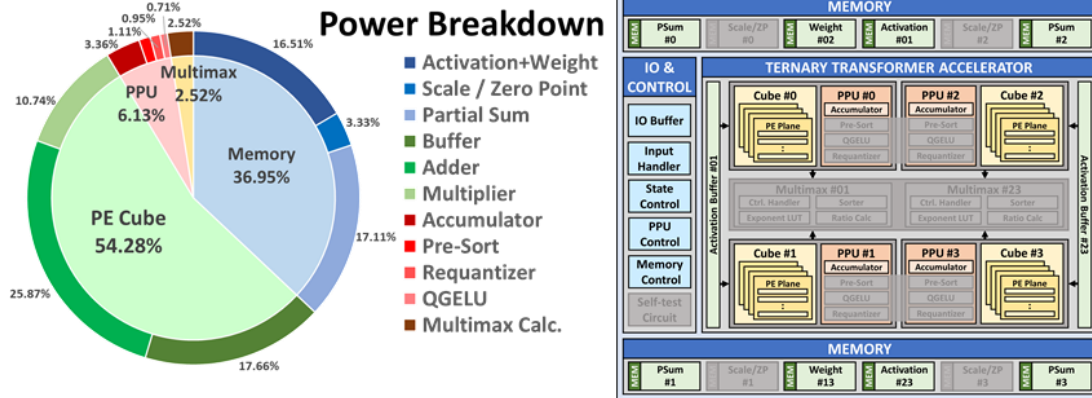


圖 5.6 矩陣運算模式功耗佔比圖與模組啟動示意圖

接著是 Linear Layer 運算中，會涉及矩陣運算以及量化運算，如圖 5.7，可以觀察到負責量化運算的 Requantizer 的功耗佔比顯著上升。

Quantize Mode

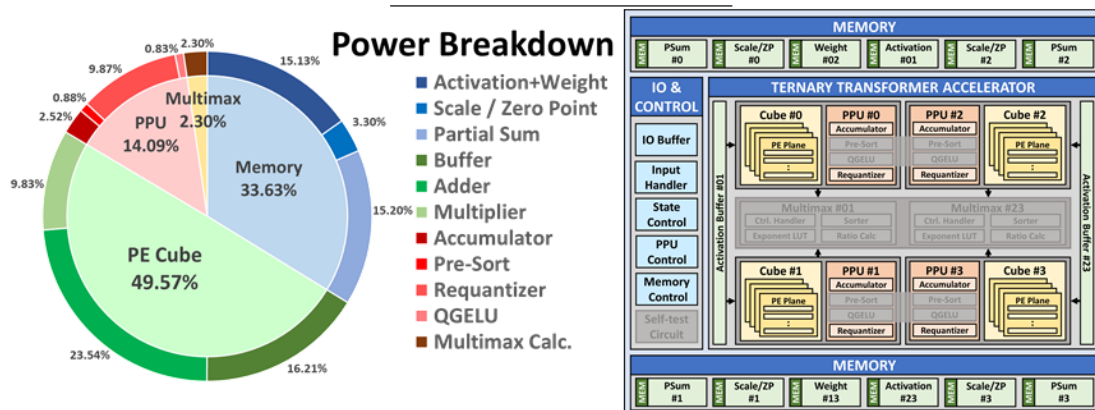


圖 5.7 量化模式功耗佔比圖與模組啟動示意圖

在 MLP 的第一層 layer 中，涉及 Linear Layer 與 Activation Function 的運算，因此可以觀察到 QGELU 模組的功耗佔比上升，此外由於 QGELU 的輸入與輸出皆需要經過 Requantizer，因此該模組之功耗也有佔有顯著比例。



QGELU Mode

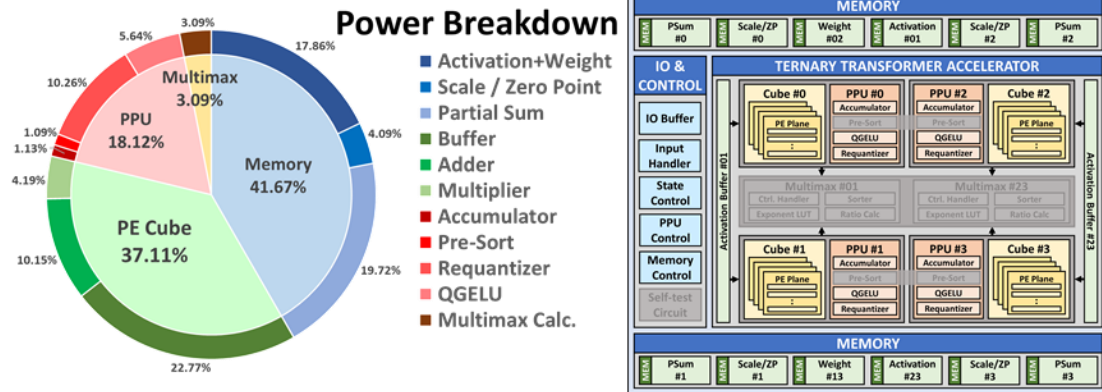


圖 5.8 QGELU 模式功耗佔比圖與模組啟動示意圖

最後是 Multimax 的計算，如圖 5.9，由於 Multimax 需要待所有輸出皆完成運算，並找出 top-N 的數值，才能計算對應之 ratio，因此 PE Cube 的比例便有所下降。

Multimax Mode

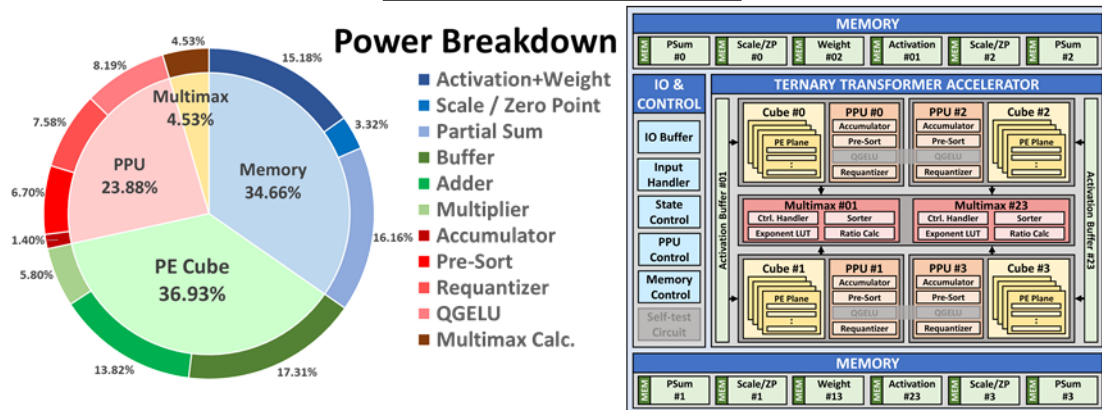


圖 5.9 Multimax 模式功耗佔比圖與模組啟動示意圖

5.3.3 晶片比較表

為了證明我們所提出的加速晶片在算力、面積效率與能源效率皆能超越現有加速器，我們建立了晶片比較表，蒐集了各式神經網路加速電路以進行比較。其中包括與我們相同用於 Transformer 模型加速的晶片、用於 CNN 模型的加速晶片，以及能夠支援多種模型的泛用型加速晶片。這些研究所使用的運算精度與支援的函數形式都有所不同，但我們仍以上個段落介紹的指標進行比較。

不過由於各研究所使用的製程皆有所不同，而較先進的製程在面積與能耗上皆有著非常顯著的優勢，且此部分無關加速電路本身的設計。因此若直接以原始的吞吐量、面積能源效率進行比較，恐怕有失公允。為此，我們參考學術論文中常見的標準化方式，將不同研究的晶片參數標準化至我們所使用的製程(40 奈米)以進行統一的比較。

製程標準化的公式如下所示，首先是(式 5.4)的面積標準化。

$$Normalized\ Area = Area \times \left(\frac{40}{Source\ Technology\ Node} \right)^2 \quad (式\ 5.4)$$

能耗的標準化則須納入工作電壓一併考量，而本研究所進行 power 分析的電壓為 0.81V，如(式 5.5)所示。

$$Normalized\ Power = Power \times \left(\frac{0.81}{Source\ VDD} \right)^2 \times \frac{40}{Source\ Technology\ Node} \quad (式\ 5.5)$$

最後使用標準化後的 area 與 power 計算面積與能源效率，如(式 5.6)所示。

$$\begin{aligned}
 \text{Normalized Area Efficiency} &= \frac{\text{Throughput}}{\text{Normalized Area}} \\
 \text{Normalized Power Efficiency} &= \frac{\text{Throughput}}{\text{Normalized Power}}
 \end{aligned}
 \tag{式 5.6}$$



表 5.4 為晶片比較表，我們擷取了許多篇論文[42-51]進行對照。

在晶片比較表中，我們可以觀察到在將各研究之加速晶片的 energy efficiency 與 area efficiency 標準化至 40 奈米後，本研究的晶片都有著超越其他研究的表現，這歸功於本研究使用的 TW4A 運算精度，大幅降低乘加運算的硬體複雜度，此外加法近似的技巧也進一步降低整體 PE 的功耗。其中[50]提出的晶片由於其較高的工作頻率(>1000MHz)，因此在 area efficiency 上有著超越本研究的表現。[52]設計內容為 Transformer 模型的 on-device 訓練加速晶片，傳統訓練需要複雜的 backward-pass 過程，但歸功於該論文使用的 forward pass only 演算法與運算省略技巧，使得該研究即使是以 FP8 做為運算精度仍能有著很高的 energy efficiency。此外部分研究提出的晶片為了同時處理模型的訓練與推論，例如[48]，造成硬體上的 overhead，使其在 efficiency 的計算上數值較不利，因此不納入比較表中。本研究透過低精度與 QGELU、Multimax 函數近似技巧，降低 Transformer 模型中乘加運算與非線性函數的複雜度，與其他研究相比達到 1.12 – 10.16X 的 energy efficiency 提升，以及 1.27 – 190.7X 的 area efficiency 提升。在比較表的最下方也加入模型訓練結果進行正確率的比較。



表 5.4 晶片比較表

Reference	[42]	[43]	[46]	[44]	[47]	[45]	[49]	[50]	[52]	[51]	
Model Architecture	H100 SXM5	Moon et al. ISSCC'23	Tambe et al. ISSCC'23	Qin et al. ASSC'23	Mun et al. VLSI'23	C-Transformer ISSCC'24	Keller et al. JSSC'23	Mixed-Precision VSA JSSC'25	Li et al. VLSI'25	Hsiung et al. Trans. On CAS'25	This Work
Technology(nm)	General	General	Transformers	Transformers	General	Transformers	Transformers	CNNs	Transformers	Transformers	Transformers
Die Area(mm ²)	41*11	28	12	28	28	28	5	28	40	28	40
Core Area(mm ²)	814	9.61	--	10.76	7.60	20.25	--	1.04	8.66	1.193	4.33
Memory(KB)	--	200	4.59	--	3.85	--	0.153	141	7.09	--	2.43
Supply Voltage(V)	--	0.46-1.13	647	544	398	500	0.46-1.05	0.6-1.2	0.58-1.02	0.9	60.5
Frequency(MHz)	1590	20-400	0.62-1.0	0.68-1.0	0.5-1.05	0.7-1.1	152-1760	172-1024	10-200	1000	200
Precision	FP64/32/16/8 INT32/8	AAQ(N)1-8 AB(W)1-8	FP4/8	INT8/INT16	INT5-INT8	INT8	INT4/INT8/ INT4-VSQ	INT1,2,4,8	Block-FP8	INT8	Activation INT4 Ternary Weight
Power(mW)	700W	2.06-237.1	9-111 @FP4 10-122 @FP8	38.08-395.6	3.3-705	47.5 @50MHz, 0.7V 469.2 @200MHz, 1.1V	--	23-600	16.2-229.3	443.2	200.9
Performance(TOPS)	1979 @ INT8	0.52-5.26 @AQ/B8 1.87-18.9 @AQ/B2	0.734 @FP4 0.367 @FP8	0.17-6.53 @INT8 0.09-3.17 @INT16	4.18	3.41	3.6 @INT4 3.6 @INT4-VSQ 1.8 @INT8	8.26 @1b, 1.2V	10.24	2.66	6.50
Energy Efficiency (TOPS/W)	2.83 @ INT8	8.94-127.8 @AQ/B8 up to 90% sparsity	18.1 @FP4 8.24 @FP8	2.22-49.7 @INT8 1.11-23.9 @INT16	66.8 @0.5V	22.9-47.8 @50MHz, 0.7V	91.1 @INT4 95.6 @INT4-VSQ 39.1 @INT8	63.5 @1b, 0.6V	43.7-157	6.00	32.3 (Die) 32.8 (Core)
Area Efficiency (TOPS/mm ²)	Die Area	0.07-0.26 @AQ/B8 0.72-2.60 @AQ/B2	0.16 @FP4 0.08 @FP8	0.016-0.61 @INT8 0.008-0.29 @INT16	0.55	0.17	--	--	0.06-1.18	0.86	1.50
Core Area	--	0.07-0.26 @AQ/B8 0.72-2.60 @AQ/B2	0.16 @FP4 0.08 @FP8	--	1.09	--	23.3 @INT4 23.3 @INT4-VSQ 11.7 @INT8	7.94 @1b, 1.2V	0.07-1.44	--	2.67
Normalized to 40nm & 0.81V											
Normed Energy Efficiency (TOPS/W)	--	2.02-28.85 @AQ/B8 up to 90% sparsity	3.18 @FP4 1.45 @FP8	1.10-24.5 @INT8 0.55-11.8 @INT16	17.82 @0.5V	11.97-24.99 @50MHz, 0.7V	3.67 @INT4 3.85 @INT4-VSQ 1.58 @INT8	24.39 @1b, 0.6V	43.7-157	5.19	32.3 (Die) 32.8 (Core)
Normed Area Efficiency (TOPS/mm ²)	Die Area	0.02-0.10 @AQ/B8 0.27-0.97 @AQ/B2	--	0.008-0.30 @INT8 0.004-0.14 @INT16	0.27	0.083	--	--	0.06-1.18	--	1.50
Core Area	--	0.03-0.13 @AQ/B8 0.35-1.27 @AQ/B2	0.014 @FP4 0.007 @FP8	--	0.53	--	0.364 @INT4 0.364 @INT4-VSQ 0.183 @INT8	3.89 @1b, 1.2V	0.07-1.44	0.421	2.67
Model/ Accuracy	--	VIT-tiny 73.60%(ImageNet)	BERT 90.30%(SST-2)	--	DeiT-small 79.0%(ImageNet)	GPT-2 14.96(Wikitext)	DeiT-small 76.22%(ImageNet)	VGG-16 70.146%(ImageNet)	BERT 92.60%(SST-2)	DeiT-small 79.80%(ImageNet)	DeiT-Base 82.03%(ImageNet)
Accuracy Loss	--	-1.86%	-1.90%	--	-0.90%	-1.26 Perplexity loss	-3.60%	-5.03%	-0.90%	-1.40%	-2.32%



5.4 效能與功耗分析

為比較本研究提出之加速晶片效能與其他主流運算平台的差異，本段落會進行各項數據的比較。在比較對象的選擇上，首先是高端 CPU，以及 CPU+GPU 的常見機器學習運算組合。邊緣運算裝置則選擇 Jetson Nano 作為比較對象。最後 ARM Cortex-A53 + TTA IC(表中以 TTA IC 稱呼本研究所提出的 ternary transformer accelerator IC)的組合是由於 TTA IC 無法支援所有 transformer 模型中的運算，因此此類不支援運算會以 ARM Cortex-A53 負責。詳細規格如表 5.5 所示。

表 5.5 運算平台設置細節

Setup	Detailed Spec.
CPU	Intel Xeon w7-2475X
CPU + GPU	Intel Xeon w7-2475X + Nvidia 4090
Low Power CPU + GPU	Jetson Nano ARM Cortex-A57 CPU + Maxwell GPU
Low Power CPU	KV260 ARM Cortex-A53 CPU
Low Power CPU + ASIC	KV260 ARM Cortex-A53 CPU + Proposed Accelerator

為公平起見，本段落實驗皆使用 ViT-Base 模型進行一張影像的 inference 運算，功耗量測部分，Intel CPU 使用第三方套件 Scaphandre[53]來獲取 CPU 個執行緒自身的功耗，Nvidia GPU 則使用 nvidia-smi 取得運算當下的功耗。KV260 上的 CPU 我們使用該平台附帶的功率監測裝置 INA260 以獲取功耗數值，不過此方式獲取的功耗是 KV260 全系統的功耗，因此我們會將運算時的功耗減去平台 IDLE 時的功耗以獲取 CPU 單獨的功耗。Jetson Nano 則使用其自帶的 jtop 套件獲取 CPU 與 GPU 的能耗。ASIC 的功耗則是由 post simulation 獲得。由於除 post simulation 外，各種量測方式都是取得當下的瞬時功耗，因此必然存在誤差，以下實驗結果都經過多次擷取功耗後平均已獲得較穩定準確的數據。



各平台的功耗如表 5.6，其中 ARM Cortex-A53 + ASIC 的組合中，由於 CPU 與 ASIC 不會同時啟動，因此 average power 是取總能量除以總時間獲得加權的 power，而非兩個硬體的 power 相加。

表 5.6 各運算平台平均功耗

Hardware Setup	Power (W)			Avg Power (W)
	CPU	GPU	TTA IC	
Intel Xeon	168.4	--	--	168.4
Intel Xeon + Nvidia 4090	110.5	115.4	--	225.4
ARM Cortex-A57 + Maxwell(Jetson Nano)	1.20	0.44	--	1.64
ARM Cortex-A53	0.71	--	--	0.71
ARM Cortex-A53 + TTA IC	0.71	--	0.12	0.49

獲得功耗後，我們接著計算 token/s 以及 token/J，前者可以直觀理解為 inference 的速度，由表 5.7 中的結果也可以觀察到各項數據是符合預期的，CPU + GPU 的組合擁有著最快的 inference 速度，純 CPU 的平台次之。而 ARM Cortex-A53 + ASIC 組合的運算速度相比純粹的 ARM Cortex-A53 有 10 倍的明顯提升。至於 token/J 則可以用於評估平台的能量效率，表 5.7 中的結果也可以觀察到本研究的加速器搭配 ARM Cortex-A53 相比於其他平台有著 8.4~37.8 倍的效率提升，足見 TTA IC 在能耗上的優勢。

表 5.7 各運算平台能量效率計算

Setup	token/s	token/J
Intel Xeon	4477.27	26.56
Intel Xeon + Nvidia 4090	22386.36	99.07
ARM Cortex-A57 + Maxwell	887.39	120.12
ARM Cortex-A53	48.08	69.69
ARM Cortex-A53 + TTA IC	491.52	1003.1

除了上述分析外，為了進一步凸顯 TTA IC 與 low power CPU 間的性能差距，表 5.8 僅針對 inference 過程中 TTA IC 可以支援的運算進行比較，並統整兩者的數據。可以發現不論是運算時間還是功耗，TTA IC 都有的明顯的優勢，這也使得總能量消耗有著 218 倍的提升。

表 5.8 ASIC 與 LP CPU inference 數據比較

Operation	Time (ms)	Avg power (mW)	Energy (mJ)
TTA IC	157.8	122.3	19.3
ARM Cortex-A53	3854.0	710.0	2736.3

以上的分析雖然證明在相同模型下本晶片可以達到更高的能源效率，但由於量化與近似的過程中難免造成 accuracy loss，因此我們希望針對加上參數量化與函數近似的模型與同系列較小(參數量低、運算速度快)的模型間的能耗關係做進一步的討論與分析。延續本段落的討論，我們使用 ViT 系列模型來進行實驗，ViT 各大小模型的架構與參數量如表 5.9。

表 5.9 ViT 系列模型參數細節

Model	Embedded Dim.	Model Depth	# of Heads	Size
ViT-Base	768	12	12	86.6M
ViT-Small	384	12	6	22.1M
ViT-Tiny	192	12	3	5.72M

而 ViT-base 加上 TW4A 量化與 Multimax、QGELU 函數近似後，accuracy 會有所下降，表 5.10 中我們與 FP32 精度的 ViT-small 與 ViT-tiny 做比較。

表 5.10 量化近似 ViT 模型與同系列較小模型 accuracy 比較

Model	Precision & Approximation	ImageNet100 Top-1 Acc.
ViT-Base	FP32	87.20%
ViT-Base	TW4A + Multimax + QGELU	85.60%
ViT-Small	FP32	85.78%
ViT-Small	TW4A + Multimax + QGELU	82.80%
ViT-Tiny	FP32	83.30%

為了驗證本晶片可以使用更低的能耗獲得更高的模型效能，我們使用 ARM Cortex-A53 搭配 TTA IC 執行 ViT-base，並使用 Intel CPU 及 Nvidia GPU 運行 ViT-tiny，並記錄運行功耗。此外，也加入了 accuracy 與 FP32 ViT-tiny 相近的 TW4A ViT-small 的能耗數據做為比較，結果如表 5.11 所示，可以發現 ARM CPU 加上 TTA IC 的組合除了模型效能表現較佳外，在 J/token 的能量效率上也具有顯著的優勢，相比 Intel CPU 以及 CPU + GPU 的對照組有明顯的能源效率提升。

表 5.11 各運算平台效率計算結果

Model	Precision	Acc.	Setup	Energy (J)	Avg. Power (W)	Time (ms)	J/token ↓	token/s	token/J ↑
ViT-Tiny	FP32	83.30%	Intel Xeon	2.12	158.3	13.4	1.08×10^{-2}	14701.5	92.9
			Intel Xeon + 4090	1.61	189.5	8.5	8.17×10^{-3}	23176.5	122.3
			Jetson Nano	0.36	1.64	222.0	1.83×10^{-3}	887.39	541.1
ViT-Small		85.78%	Intel Xeon	3.44	163.2	21.1	1.75×10^{-2}	9336.5	57.2
			Intel Xeon + 4090	1.64	188.8	8.7	8.32×10^{-3}	22643.7	119.9
			Jetson Nano	0.60	2.30	259.4	3.05×10^{-3}	759.4	330.2
ViT-Base		87.20%	Intel Xeon	8.36	180.2	46.4	4.24×10^{-2}	4245.7	23.56
			Intel Xeon + 4090	2.13	226.9	9.4	1.08×10^{-2}	20957.4	92.4
			Jetson Nano	1.68	4.60	366.2	8.53×10^{-3}	538.0	117.0
ViT-Small	TW4A +Multimax +QGELU	82.80%	ARM A53 + TTA IC	0.14	0.55	257.0	7.10×10^{-4}	766.53	1393.7
ViT-Base		85.60%		0.19	0.49	400.8	9.64×10^{-4}	491.52	1003.1

5.5 晶片量測

以上介紹了如何驗證晶片的正確性，以及進行 power 分析。但以晶片設計而言最重要的便是實際下線晶片的量測結果。在撰寫論文的當下雖然晶片尚未完成製作，但此章節仍會介紹預計進行的操作以及期望的結果。

晶片量測的流程與模擬時類似，需使用 Python 生成的輸入與輸出進行比對，差異在於需要實際將晶片使用專業量測機台進行輸入的載入與結果的紀錄。我們會使用 TSRI 架設的 ADVENTEST V93000 邏輯晶片測量儀器進行量測，而流程如圖 5.10 所示。

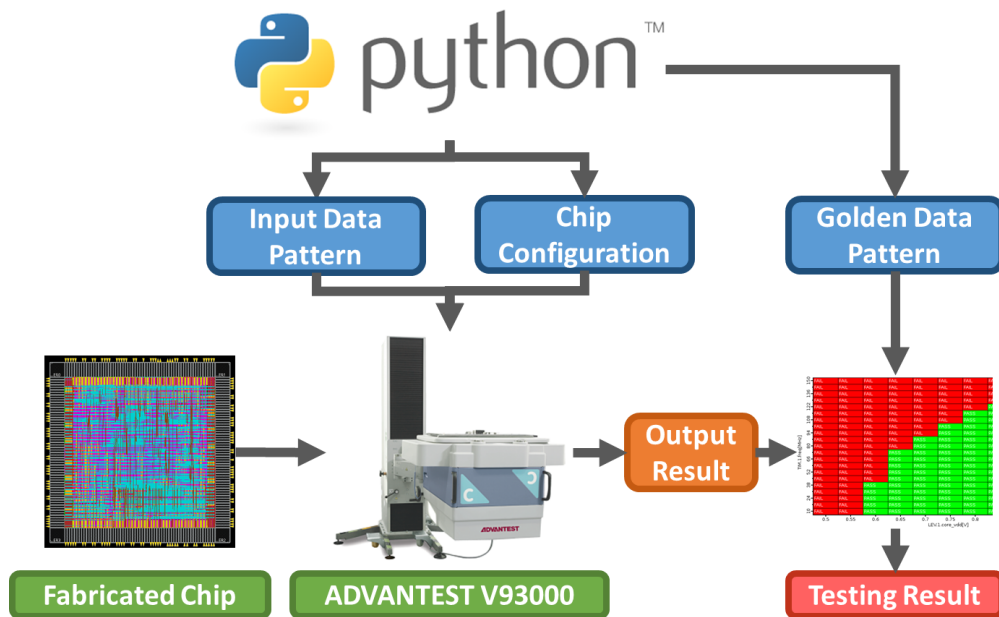


圖 5.10 晶片量測流程圖

由於實際下線的晶片，即使製造過程皆順利，實際運行的最高頻率仍會受當下氣溫、輸入電壓、晶片內部供電情形、晶片個體差異影響，而與使用 tool 進行模擬的結果有所差異，因此，我們需要繪製 shmoo plot。以下會對 shmoo plot 做簡單介紹。

Shmoo Plot 是晶片測試與電路驗證中常用的圖表，用以評估電路在不同工作條件下的功能表現與可靠度。在實際測試中，Shmoo Plot 會選擇兩個關鍵參數作為座標軸，一般常見的選擇是電壓與工作頻率，並針對不同組合的測試條件或測試模式，記錄電路的 Pass/Fail 結果。圖表通常以不同顏色標示功能正常

與異常區域，綠色一般代表測資通過，紅色則代表失敗，可直觀呈現電路在各參數範圍內的可操作區，並觀察其邊界條件。



圖 5.11 是一個範例的晶片量測 shmoo plot。可以觀察到隨著輸入電壓升高，最高的工作頻率也隨之提升，符合對於電路特性的預期。

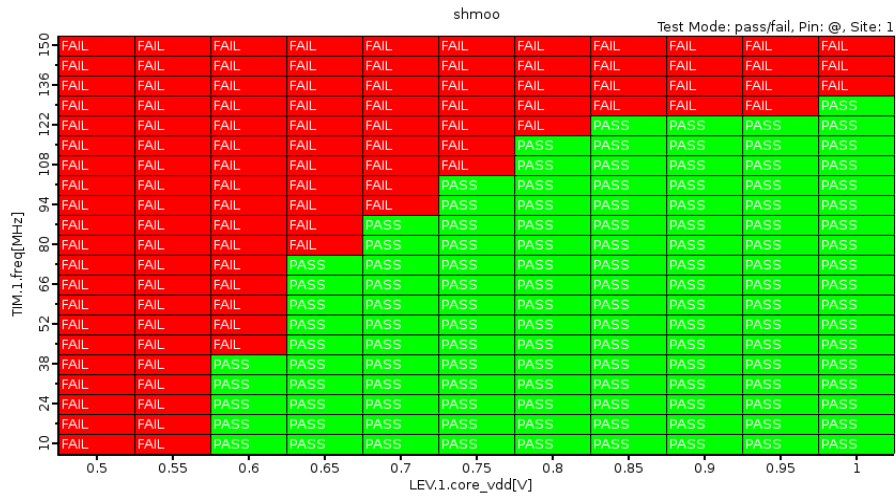


圖 5.11 範例量測結果 shmoo plot

我們會針對各種晶片模式，例如 Linear Layer、Multimax、QGELU 等不同的運算模式繪製 shmoo plot，由於各模式間啟動的運算模組並不相同，因此可預期的是這些 shmoo plot 之間會有著些許的差異。

至於功率的分析，由於量測時無法指定某一區段進行 power 的分析，僅能獲取整個測試過程的平均功耗。因此我們會於模擬時紀錄整個模擬過程的平均功耗，以及重要運算區段功耗的比值，並以此估算實際晶片工作環境下核心運算的工耗為何。最終目標希望與模擬的結果接近。

最後我們進行晶片的 PVT(process, voltage, temperature)特性討論，在不同的 PVT 組合下晶片效能會展現不同的結果。於模擬時，我們會使用三個 PVT corner 進行模擬，分別是 slow(ss)、typical(tt)、fast(ff)，三個 corner 的 voltage 與 temperature 設定如表 5.12。Slow corner 中會使用最低的 voltage 做為電路供電，而溫度也是設定在較高的環境溫度以驗證晶片在最惡劣的環境下的表現，本研究中晶片參數表的數值皆是使用此 corner 下的數據以確保晶片本身最不理想情況下的表現仍符合預期。而 typical corner 則是最貼近實際量測情境的 corner，溫度為室溫的 25°C，電壓為理想電壓 0.9V。至於 fast corner 則是將參數都設定為理想的低溫高電壓情況，去找出晶片最高速的運行頻率，也會用於驗證晶片的 hold time violation。表 5.12 中以 linear layer 的計算模式進行模擬，並找出在各 corner 下晶片可以達到的最高工作頻率與當下的功耗。由於在 APR 過程中我們便是使用 slow corner 進行晶片的 static time analysis，因此 slow corner 的最高頻率便是我們設定的 200MHz，而隨著溫度與電壓上升，晶片的最高工作頻率與功耗也隨之上升，這代表在實際量測時若各條件皆理想的情況下，晶片有機會量測到超過 200MHz 的最高工作頻率。這也是為什麼我們會使用 shmoo plot 來找出各 supply voltage 下的晶片表現。

表 5.12 PVT corner 設定與晶片模擬結果

PVT Corner	Voltage(V)	Temp(°C)	Max Freq.(MHz)	Power(mW)
Slow(ss)	0.81	125	200	239
Typical(tt)	0.9	25	263	309
Fast(ff)	0.99	-40	588	672

5.6 第五章總結

本章針對本研究所提出之加速器設計，從晶片設計流程到驗證方法進行了完整的介紹與分析。首先是晶片的設計流程，從模型軟體驗證開始，到 RTL 的

撰寫、合成、APR，最後進行晶片的下線。接著介紹晶片的驗證方式，如何以軟體與硬體交叉比對的方式確保晶片的正確性。



晶片參數部分，我們介紹了本研究晶片的布局，以及與晶片相關的參數。並介紹了硬體加速晶片的重要指標，以及與其他研究提出的加速晶片相比較的結果。在將面積與能源效率標準化至相同製程後，我們的晶片相比其他加速晶片有著更高的效率，這要歸功於本研究採用的低精度 TW4A 量化技巧，以及硬體友善的非線性函數近似技巧，使得運算的複雜度與硬體成本顯著降低，進而提高晶片的效率。

晶片測量部分，由於當下晶片尚未完成，因此僅列出預計的量測方式與期待的量測結果。

第六章 研究結語與展望



自 Transformer 模型問世以來，其優越的表現已在各項領域中展現出超越傳統模型的能力，並締造了許多卓越的成果。然而，在追求極致效能的同時，Transformer 模型在訓練與推論階段均帶來了極高的運算與記憶體需求，使其在邊緣運算裝置上的部署面臨諸多挑戰。

因此，本研究綜合運用包括低精度量化、複雜函數近似與加法器近似等多項技術，以有效降低 Transformer 模型的整體運算量與硬體負擔，提升其於邊緣運算裝置上的部署可行性。

首先，針對模型參數與運算精度，本研究採用 Ternary Weight 與 INT-4 Activation 的 TW4A 量化策略，能有效減少 93.75%與 87.5%的 on-chip 記憶體需求。此外，TW4A 策略亦大幅降低 MAC 運算的硬體複雜度，從而減少 PE 的電路面積與功耗。此外，本研究進一步於 PE 中引入基於 3-input 邏輯閘的 OAI21 + AAI21 雙層近似加法器，針對加法運算中的 LSB 或末兩位數值進行加法近似運算。實驗結果顯示，該方法在 PE Cube 中分別能節省 4.3%與 21.4%的功耗。

在非線性函數近似方面，我們針對 Transformer 模型中的非線性函數，Softmax 與 GELU，分別提出 Multimax 與 QGELU 技巧。Multimax 利用 Softmax 輸出分佈特性，僅計算單一 row 中 Top-N 輸出以近似 Softmax，顯著降低含有指數運算與除法運算之 Softmax 函數的複雜度。QGELU 則將 GELU 函數以分段二項式進行近似，並將二項式的係數調整為可以透過 shift & add 便可完成的運算，

以達到高運算效率與低硬體成本。兩者皆經過完整實驗驗證，證實能有效取代原始函數而不造成明顯的準確率損失。



本研究在影像辨識、文字生成、語音轉文字等任務上，驗證所提出的 TW4A 量化技巧與函數近似對 Transformer 模型效能的影響，並應用於 ViTs、GPT-2、Whisper 等不同 Transformer 模型架構，展現技巧的通用性。在影像辨識任務中，部分低精度模型在經過量化感知訓練後，能維持與原始全精度模型相近的準確率。然而，隨著任務難度上升，例如 GPT-2 的文字生成任務，儘管量化模型能將效能損失控制在可接受的範圍內，仍不可避免地出現準確率下降的情形，但與 1W4A 的量化策略相比仍有十足進步。

然而，本研究提出之低精度量化與函數近似技巧，在實作上仍需透過軟體層面的量化感知訓練以避免過大的效能損失。此訓練方式雖能有效模擬硬體行為，提升模型在近似運算下的適應性，但同時也造成龐大的運算資源消耗，且由於訓練過程中需額外執行大量模擬近似運算的操作，使得整體訓練時間相較於傳統 FP32 模型顯著增加。隨著模型參數量及結構複雜度的上升，如何進一步優化軟體層之訓練成本，將成為未來在實際應用中必須面對的重要課題。

此外，未來亦可考慮將本研究提出的量化與近似方法，應用於更多元的深度學習任務，如近年興起的多模態學習、影音生成、圖像生成與理解等應用場景，探討本研究介紹之技巧在不同任務與模型結構中對效能的影響。

隨著硬體加速技術的快速發展，記憶體牆(Memory Wall)的問題也變得愈發嚴重，成為限制系統效能提升的主要瓶頸之一。許多研究便朝向記憶體內運算(Compute in Memory)的技術進行研究，以減緩運算核心與記憶體之間溝通所造

成的延遲與能耗。由於本研究的數值已量化至低精度，無論是乘法器或加法器均較傳統全精度實現大幅簡化，未來可進一步結合 CIM 架構，探索低精度量化的運算於記憶體內運算中的設計挑戰與潛在效益。

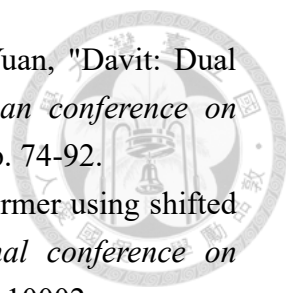


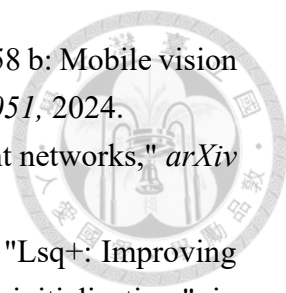
最後，在系統整合層面，本研究所提出的技巧與晶片設計，主要聚焦於加速器本體的運算單元優化。然而，若要於實際應用場域進行部署，仍存在諸多待解決的挑戰。首先，如何建立低延遲且高效率的溝通流程，以串聯 MPU 與 ASIC 加速器，將成為影響整體系統效能的關鍵因素。唯有透過資料傳輸與記憶體訪問流程的最佳化設計，才能充分發揮 ASIC 加速晶片在運算速度與能耗上的優勢。此外，易於開發與部署的 compiler 亦是系統導入時的重要考量之一。隨著神經網路加速電路架構日益多樣化，若缺乏對應的高效編譯器支持，將大幅增加模型部署與維護的複雜度。因此，如何設計可支援低精度運算、函數近似，以及特殊算子加速的編譯工具鏈，將成為未來研究與系統商品化過程中的重要課題。

參考文獻



- [1] A. Vaswani *et al.*, "Attention is all you need," in *Proceedings of Advances in neural information processing systems*, Long Beach, CA, USA, December 2017, vol. 30, pp. 5998-6008.
- [2] N. Jouppi *et al.*, "Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proceedings of the 50th annual international symposium on computer architecture*, Orlando, FL, USA, June 2023, pp. 1-14.
- [3] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin, "Convolutional sequence to sequence learning," in *Proceedings of International conference on machine learning*, Sydney, Australia, August 2017: PMLR, pp. 1243-1252.
- [4] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," *arXiv preprint arXiv:1606.08415*, 2016.
- [5] J. Wang, J. Wu, and L. Huang, "Understanding the failure of batch normalization for transformers in NLP," in *Proceedings of Advances in Neural Information Processing Systems*, New Orleans, LA, USA, November 2022, vol. 35, pp. 37617-37630.
- [6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, November 2002.
- [7] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *Proceedings of 2009 IEEE conference on computer vision and pattern recognition*, Miami Beach, FL, USA, June 2009: Ieee, pp. 248-255.
- [8] S. V. Lab. "ImageNet." <https://www.image-net.org/download.php> (accessed 06-27, 2025).
- [9] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
- [10] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proceedings of International conference on machine learning*, Vienna, Austria, July 2021: PMLR, pp. 10347-10357.

- 
- [11] M. Ding, B. Xiao, N. Codella, P. Luo, J. Wang, and L. Yuan, "Davit: Dual attention vision transformers," in *Proceedings of European conference on computer vision*, Tel Aviv, Israel, October 2022: Springer, pp. 74-92.
 - [12] Z. Liu *et al.*, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proceedings of the IEEE/CVF international conference on computer vision*, Montreal, Canada, October 2021, pp. 9992-10002.
 - [13] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," 2018.
 - [14] C. Raffel *et al.*, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of machine learning research*, vol. 21, no. 1, pp. 5485 - 5551, January 2020, Art no. 140.
 - [15] H. Touvron *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
 - [16] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
 - [17] J. Achiam *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
 - [18] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: an asr corpus based on public domain audio books," in *Proceedings of 2015 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, Brisbane, Queensland, Australia, April 2015: IEEE, pp. 5206-5210.
 - [19] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of International conference on machine learning*, Honolulu, Hawaii, USA, July 2023: PMLR, pp. 28492-28518.
 - [20] H. Wang *et al.*, "Bitnet: Scaling 1-bit transformers for large language models," *arXiv preprint arXiv:2310.11453*, 2023.
 - [21] S. Ma *et al.*, "The era of 1-bit llms: All large language models are in 1.58 bits," *arXiv preprint arXiv:2402.17764*, vol. 1, no. 4, 2024.
 - [22] S. Ma *et al.*, "BitNet b1. 58 2B4T Technical Report," *arXiv preprint arXiv:2504.12285*, 2025.
 - [23] H. Wang, S. Ma, and F. Wei, "BitNet a4. 8: 4-bit Activations for 1-bit LLMs," *arXiv preprint arXiv:2411.04965*, 2024.
 - [24] H. Wang, S. Ma, and F. Wei, "BitNet v2: Native 4-bit Activations with Hadamard Transformation for 1-bit LLMs," *arXiv preprint arXiv:2504.18415*, 2025.

- 
- [25] Z. Yuan, R. Zhou, H. Wang, L. He, Y. Ye, and L. Sun, "Vit-1.58 b: Mobile vision transformers in the 1-bit era," *arXiv preprint arXiv:2406.18051*, 2024.
 - [26] F. Li, B. Liu, X. Wang, B. Zhang, and J. Yan, "Ternary weight networks," *arXiv preprint arXiv:1605.04711*, 2016.
 - [27] Y. Bhalgat, J. Lee, M. Nagel, T. Blankevoort, and N. Kwak, "Lsq+: Improving low-bit quantization through learnable offsets and better initialization," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, Seattle, WA, USA, June 2020, pp. 696-697.
 - [28] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, 2013.
 - [29] W. Byun, J. Woo, and S. Mukhopadhyay, "FPGA Acceleration With Hessian-Based Comprehensive Intra-Layer Mixed-Precision Quantization for Transformer Models," *IEEE Access*, vol. 13, pp. 70282--70297, April 2025.
 - [30] A. Marchisio, D. Dura, M. Capra, M. Martina, G. Masera, and M. Shafique, "SwiftTron: An efficient hardware accelerator for quantized transformers," in *Proceedings of 2023 International Joint Conference on Neural Networks (IJCNN)*, Queensland, Australia, June 2023: IEEE, pp. 1-9.
 - [31] K. Chen, Y. Gao, H. Waris, W. Liu, and F. Lombardi, "Approximate softmax functions for energy-efficient deep neural networks," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 1, pp. 4-16, November 2022.
 - [32] J. Yu *et al.*, "NN-LUT: Neural approximation of non-linear operations for efficient transformer inference," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, San Francisco, CA, USA, July 2022, pp. 577-582.
 - [33] K. Banerjee, R. R. Gupta, K. Vyas, and B. Mishra, "Exploring alternatives to softmax function," *arXiv preprint arXiv:2011.11538*, 2020.
 - [34] C. Peltekis, K. Alexandridis, and G. Dimitrakopoulos, "Reusing softmax hardware unit for gelu computation in transformers," in *Proceedings of 2024 IEEE 6th International Conference on AI Circuits and Systems (AICAS)*, Dubai, UAE, April 2024: IEEE, pp. 159-163.
 - [35] J. Chen, "Design and Implementation of an Energy-Efficient Binary-Weight Transformer Accelerator Chip," M.S. thesis, National Taiwan University, Taipei, 2024.
 - [36] Y.-H. Huang, P.-H. Kuo, and J.-D. Huang, "Hardware-Friendly Activation Function Designs and Its Efficient VLSI Implementations for Transformer-Based Applications," in *Proceedings of 2023 IEEE 5th International*

- Conference on Artificial Intelligence Circuits and Systems (AICAS)*, Hangzhou, China, June 2023: IEEE, pp. 1-5.
- [37] T. Mohaidat, M. R. K. Khan, and K. Khalil, "Curvature-Based Piecewise Linear Approximation Method of GELU Activation Function in Neural Networks," in *Proceedings of 2024 2nd International Conference on Artificial Intelligence, Blockchain, and Internet of Things (AIBThings)*, Mount Pleasant, MI, USA, September 2024: IEEE, pp. 1-5.
- [38] D. Wang, C.-T. Lin, G. K. Chen, P. Knag, R. K. Krishnamurthy, and M. Seok, "DIMC: 2219TOPS/W 2569F2/b digital in-memory computing macro in 28nm based on approximate arithmetic hardware," in *Proceedings of 2022 IEEE international solid-state circuits conference (ISSCC)*, San Francisco, CA, USA, February 2022, vol. 65: IEEE, pp. 266-268.
- [39] Y. He *et al.*, "An RRAM-based digital computing-in-memory macro with dynamic voltage sense amplifier and sparse-aware approximate adder tree," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 2, pp. 416-420, September 2022.
- [40] M.-G. Lin, J.-P. Wang, C.-Y. Chang, and A.-Y. A. Wu, "Approximate Adder Tree Design with Sparsity-Aware Encoding and In-Memory Swapping for SRAM-based Digital Compute-In-Memory Macros," in *Proceedings of 2024 IEEE 6th International Conference on AI Circuits and Systems (AICAS)*, Abu Dhabi, UAE, April 2024: IEEE, pp. 362-366.
- [41] A. Dave, F. Frustaci, F. Spagnolo, M. Yayla, J.-J. Chen, and H. Amrouch, "HW/SW codesign for approximation-aware binary neural networks," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 13, no. 1, pp. 33-47, February 2023.
- [42] N. Corporation. "NVIDIA H100 Tensor Core GPU Architecture." <https://resources.nvidia.com/en-us-data-center-overview/gtc22-whitepaper-hopper> (accessed 06-27, 2025).
- [43] S. Moon, H.-G. Mun, H. Son, and J.-Y. Sim, "A 127.8 TOPS/W arbitrarily quantized 1-to-8b scalable-precision accelerator for general-purpose deep learning with reduction of storage, logic and latency waste," in *Proceedings of 2023 IEEE International Solid-State Circuits Conference (ISSCC)*, San Francisco, CA, USA, February 2023: IEEE, pp. 21-23.
- [44] Y. Qin *et al.*, "A 28nm 49.7 TOPS/W Sparse Transformer Processor with Random-Projection-Based Speculation, Multi-Stationary Dataflow, and Redundant Partial Product Elimination," in *Proceedings of 2023 IEEE Asian Solid-State Circuits Conference (A-SSCC)*, Haikou, China, November 2023: IEEE, pp. 1-3.

- [45] S. Kim, S. Kim, W. Jo, S. Kim, S. Hong, and H.-J. Yoo, "20.5 C-Transformer: A 2.6-18.1 μ J/Token Homogeneous DNN-Transformer/Spiking-Transformer Processor with Big-Little Network and Implicit Weight Generation for Large Language Models," in *Proceedings of 2024 IEEE International Solid-State Circuits Conference (ISSCC)*, San Francisco, CA, USA, February 2024, vol. 67: IEEE, pp. 368-370.
- [46] T. Tambe *et al.*, "22.9 A 12nm 18.1 TFLOPs/W sparse transformer processor with entropy-based early exit, mixed-precision predication and fine-grained power management," in *Proceedings of 2023 IEEE International Solid-State Circuits Conference (ISSCC)*, San Francisco, CA, USA, February 2023: IEEE, pp. 342-344.
- [47] H. Mun, H. Son, S. Moon, J. Park, B. Kim, and J.-Y. Sim, "A 28 nm 66.8 TOPS/W sparsity-aware dynamic-precision deep-learning processor," in *Proceedings of 2023 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*, Kyoto, Japan, June 2023: IEEE, pp. 1-2.
- [48] K. Prabhu *et al.*, "MINOTAUR: A Posit-Based 0.42–0.50-TOPS/W Edge Transformer Inference and Training Accelerator," *IEEE Journal of Solid-State Circuits*, vol. 60, no. 4, pp. 1311 - 1323, March 2025.
- [49] B. Keller *et al.*, "A 95.6-TOPS/W deep learning inference accelerator with per-vector scaled 4-bit quantization in 5 nm," *IEEE Journal of Solid-State Circuits*, April 2023.
- [50] K. Li, M. Huang, A. Li, S. Yang, Q. Cheng, and H. Yu, "A 29.12-TOPS/W Vector Systolic Accelerator With NAS-Optimized DNNs in 28-nm CMOS," *IEEE Journal of Solid-State Circuits*, vol. 60, no. 10, pp. 3790 - 3801, April 2025.
- [51] C.-L. Hsiung and T.-S. Chang, "Low-Power Vision Transformer Accelerator With Hardware-Aware Pruning and Optimized Dataflow," *IEEE Transactions on Circuits and Systems I: Regular Papers*, pp. 1--10, July 2025.
- [52] C.-Y. Li, Y.-F. Shyu, and C.-H. Yang, "An 157TOPS/W Transformer Learning Processor Supporting Forward Pass Only with Zeroth-Order Optimization," in *proceedings of 2025 Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*, Kyoto, Japan, June 2025: IEEE, pp. 1-3.
- [53] "Scaphandre." <https://github.com/hubblo-org/scaphandre> (accessed 08-22, 2025).